

федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет
им. И.И. Ползунова»

Факультет (институт) ФИТ
Кафедра прикладной математики

Отчёт защищён с оценкой

Зав. кафедрой _____
Боровцов Е. Г.
(подпись) (Фамилия И. О.)

Отчёт по практике

Вид	Учебная практика
-----	------------------

Код и наименование направления подготовки (специальности): _____
09.03.04 Программная инженерия

Направленность (профиль, специализация): _____
Разработка программно-информационных систем

Форма обучения: очная

Студента Дворникова Марка Сергеевича
(Фамилия Имя Отчество)

Группа ПИ-54

г. Барнаул

ФГБОУ ВО «Алтайский государственный технический
университет И.И. Ползунова»

Кафедра прикладной математики

Индивидуальное задание

на учебную практику
(вид и тип практики по УП)

студенту Дворникову Марку Сергеевичу Группы ПИ-54
(Ф.И.О.)

График проведения практики:

№п/п	Содержание работ, выполняемых на практике	Сроки выполнения
1	Получение индивидуального задания на практику. Прохождение инструктажа по охране труда и ТБ. Изучение предметной области и формирование технического задания	1 неделя
2	Разработка и документирование архитектуры программного продукта	2 неделя
3	Конструирование, отладка и тестирование программного продукта	2-4 неделя
4	Оформление и защита отчёта о практике	4 неделя

Руководитель практики от университета Егорова Е. В., доцент
(подпись) (Ф.И.О., должность)

Инструктаж по ОТ, ТБ, ПБ, ПВТР

Инструктаж обучающегося по ознакомлению с требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего трудового распорядка проведён «__» _____ 20__ г.

Руководитель практики от университета Егорова Е. В., доцент
(подпись) (Ф.И.О., должность)

Содержание

Введение.....	1
1 Обзор предметной области и постановка задачи	2
1.1 Обзор предметной области	2
1.2 Постановка задачи	3
1.2.1 Общее описание задачи.....	3
1.2.2 Требования к функциональности	3
2 Проектирование	5
2.1 Укрупненный алгоритм решения.....	5
2.2 Структура данных.....	5
2.2.1 Структура записи	5
2.2.2 Структура фильтра.....	6
3 Реализация	7
3.1 Выбор средств реализации.....	7
3.2 Структура программы	8
3.2.1 Модули графического интерфейса (формы).....	8
3.2.2 Игровые модули	9
3.2.3 Вспомогательные модули	9
3.2.4. Взаимодействие модулей	10
3.3 Описание реализации ПО.....	11
3.3.1 Главное меню (FlightMenu.h).....	11
3.3.2 Авторизация администратора (AdminCheck.h).....	11
3.3.3 Работа с базой данных (AdminSite.h / UserSite.h).....	12
3.3.4 Добавление рейса (AddFlight.h).....	13
3.3.5 Изменение/удаление рейсов (EditFlights.h / DeleteFlights.h) .	13

3.3.6 Настройка фильтров (SetFilters.h / EditFilterField.h)	14
3.3.7 Изменение пути к файлу (ChangePath.h)	14
3.3.8 Игровой модуль (StartGame.h / TheZondbie.h)	14
Заключение	18
Список использованных источников	20
Приложение А	21
Приложение Б	27
П.Б.1 Код FlightMenu.cpp	27
П.Б.2 Код FlightMenu.h	27
П.Б.3 Код structure.h	31
П.Б.4 Код structure.cpp	31
П.Б.5 Код file_funcs.h	36
П.Б.6 Код file_funcs.cpp	37
П.Б.7 Код AdminCheck.h	41
П.Б.8 Код AdminSite.h	43
П.Б.9 Код SetFilters.h	50
П.Б.10 Код EditFilterField.h	64
П.Б.11 Код AddFlight.h	72
П.Б.12 Код EditFlights.h	80
П.Б.13 Код DeleteFlights.h	91
П.Б.14 Код ChangePath.h	95
П.Б.15 Код UserSite.h	101
П.Б.16 Код StartGame.h	104
П.Б.17 Код TheZondbie.h	107
Приложение В	117

П.В.1 Системные требования	117
П.В.2 Установка и запуск.....	117
П.В.3 Структура файлов данных	117
П.В.4 Управление файлами данных.....	118
П.В.5 Пароль администратора.....	118
Приложение Г	119
П.Г.1 Начало работы.....	119
П.Г.2 Главное меню	119
П.Г.3 Режим администратора	119
П.Г.4 Работа с фильтрами	120
П.Г.5 Добавление, изменение и удаление записей.....	120
П.Г.6 Игра «Зомби-шутер»	120
П.Г.7 Завершение работы.....	121

Введение

Для проектов самой разной направленности всегда была актуальна проблема хранения информации, ее структуризации и возможности эффективного использования. Сегодня эта задача может быть значительно упрощена за счет работы с электронными таблицами и базами данных.

Работа с пользовательскими данными – это распространенный вид деятельности в сфере информационных технологий, предполагающий большой объем работы и большую ответственность. Поэтому каждому разработчику прикладного программного обеспечения необходимо получить опыт в реализации подобных проектов.

Цель данной работы: изучение принципов разработки графических приложений, получение навыков разработки приложений с графическим интерфейсом в среде Microsoft Visual Studio.

Главная задача: создание программного продукта с удобным графическим интерфейсом, позволяющего администратору в защищенном режиме работать с базой данных определенного назначения, а рядовому пользователю - эффективно взаимодействовать с ней.

1 Обзор предметной области и постановка задачи

1.1 Обзор предметной области

В современном мире информация стала одним из самых ценных ресурсов. Особенно это касается структурированных данных – информации, организованной по определённым правилам и предназначенной для хранения, поиска и обработки. Для работы с такими данными используются информационные системы, одной из разновидностей которых являются базы данных.

База данных (БД) – это совокупность данных, организованная по определённым правилам, обеспечивающая хранение, доступ и представление информации в удобном для пользователя виде. Простейшим примером базы данных является структурированный список, где каждая строка (запись) содержит информацию об одном объекте, а столбцы (поля) – характеристики этих объектов. Такие списки широко применяются на практике: расписание движения поездов и самолётов, каталог товаров, список студентов, база контактов и т.д.

Программное обеспечение для работы со структурированными списками должно решать несколько типовых задач:

- ввод новых данных;
- вывод данных в удобочитаемом виде;
- корректировка (редактирование) существующих данных;
- удаление устаревших или ошибочных данных;
- поиск и фильтрация данных по определённым критериям.

Особую важность приобретает вопрос долговременного хранения данных. Программа не может полагаться только на оперативную память, так как при выключении компьютера все данные теряются. Поэтому данные должны сохраняться в файле на диске. При этом работа с файлом должна

быть организована эффективно: перенос всего файла в оперативную память для выполнения каждой операции (как это часто делается в учебных примерах) становится недопустимым при больших объёмах данных. Вместо этого необходимо выполнять операции непосредственно в файле или с использованием вспомогательных файлов.

Таким образом, разработка программы, позволяющей организовать хранение, редактирование и поиск данных в структурированном списке с сохранением информации в файле, является актуальной задачей, имеющей как учебную, так и практическую ценность.

1.2 Постановка задачи

1.2.1 Общее описание задачи

Требуется разработать программное обеспечение, которое обеспечивает организацию, хранение и обработку структурированного списка данных об авиарейсах. Каждая запись содержит:

- номер рейса;
- тип самолета;
- пункт назначения;
- дни отправления;
- время вылета;
- время прилета;
- цена билета.

Например: 121 Ту-154 Москва 1,3,5 10.00 14.00 5000.20

1.2.2 Требования к функциональности

Программа должна обеспечивать выполнение следующих функций:

- Ввод и корректировка таблицы:
 - Ввод таблицы;
 - Изменение записей;
 - Удаление записей;

- запрос по любой комбинации любых полей;
- Вывод таблицы на экран.

В программе следует организовать необходимые защиты от неправильного ввода данных так, чтобы ни при каких обстоятельствах программа не выдала ошибку. В программе должен быть реализован удобный интерфейс, предполагающий удобное взаимодействие с программой.

В программе нужно реализовать два режима работы: администратор (вход по паролю) и пользователь. Функции создания и редактирования базы доступны только администратору.

В программе может присутствовать игра с графическим интерфейсом.

Итоговое приложение должно содержать минимум две формы: одна основная форма – работа с базой, а вторая форма, с игрой, может активироваться в процессе основной работы по желанию пользователя (пользователь хочет передохнуть, немного поиграть, а потом опять вернуться к основной работе).

2 Проектирование

2.1 Укрупненный алгоритм решения

При реализации программного обеспечения используется метод нисходящего проектирования и модульное программирование. Программа работает в диалоговом режиме.

При входе в программу пользователь сначала попадает в главное меню, с выбором режима работы: администратор (по паролю), пользователь и игра.

Пользователь в режиме пользователя имеет лишь функцию установки фильтров (запрос по любой комбинации любых полей).

Режим администратора предлагает следующие функции по работе с БД:

- Установка фильтров (запрос по любой комбинации любых полей);
- Добавление записи;
- Изменение записей;
- Удаление записей;
- Изменение пути до файла;

2.2 Структура данных

2.2.1 Структура записи

Запись об одном авиарейсе в программе хранится в структуре Flight. Каждая структура имеет поля:

1. `fnum` – переменная целого типа для хранения номера рейса;
2. `name` – символьный массив (строка) для хранения типа самолёта;
3. `dest` – символьный массив (строка) для хранения пункта назначения;
4. `days` – целочисленный массив для хранения чисел от 1 до 7, которые обозначают дни недели (1 – понедельник, 2 – вторник и т.д.);
5. `dep_time` – переменная целого типа для хранения времени вылета в минутах;
6. `arr_time` – переменная целого типа для хранения времени прилёта в минутах;

7. `price` – переменная типа числа двойной точности для хранения цены билета

Данные хранятся в файле `table.csv` в формате CSV (Comma-Separated Values) с разделителем «точка с запятой» (;). Каждая запись соответствует одной строке файла.

Например, запись «121 Ту-154 Москва 1,3,5 10.00 14.00 5000.20» будет иметь следующие данные:

- `fnum = 121;`
- `name = "Ту-154";`
- `dest = "Москва";`
- `days = {1, 3, 5, 0, 0, 0, 0};`
- `dep_time = 600;`
- `arr_time = 840;`
- `price = 5000.20;`

При этом `table.csv` будет хранить строчку:

«121;Ту-154;Москва;135000;600;800;5000.20»

2.2.2 Структура фильтра

Фильтры хранятся в массиве и представляют собой структуру `Flight_filter`. Она имеет те же поля, что и структура `Flight`, но к ним добавляются ещё два:

8. `apply` – целочисленный массив для хранения 0/1 в зависимости от того, на какое поле нужно применять фильтр, а на какое нет. Для некоторых полей (`dep_time`, `arr_time`, `price`) значения массива могут быть числами от 0 до 5 – в таком случае они означают операцию сравнения;
9. `logic` – целочисленный массив для хранения 0/1 в зависимости от того, какой логический оператор нужно применить при сравнении записи с фильтром.

3 Реализация

3.1 Выбор средств реализации

Для разработки был выбран язык **C++/CLI** в среде **Microsoft Visual Studio 2022**. Приложение представляет собой проект типа **Windows Forms App**, что позволило создать графический пользовательский интерфейс (GUI) с использованием стандартных компонентов (кнопки, таблицы, текстовые поля, меню и т.д.).

Для полноценного функционирования программы требуется:

- ОС Windows XP SP3 или выше;
- Оперативная память 30МБ или выше;
- Жесткий диск: 10МБ или выше для хранения программы и дополнительное место для хранения вводимых в базу данных;
- Наличие стандартных устройств ввода-вывода – клавиатура и монитор (дисплей);
- .NET Framework версии 2.0 или выше;
- Разрешение экрана 1200 x 900 или выше.

3.2 Структура программы

Программа реализована по модульному принципу. Весь исходный код разделён на логические модули (файлы .h и .cpp), каждый из которых отвечает за конкретную функциональность. Ниже приведено краткое описание основных модулей.

3.2.1 Модули графического интерфейса (формы)

- **FlightMenu** — главное меню приложения. Содержит кнопки для выбора режима работы: «Администратор», «Пользователь» и «Запустить игру».
- **AdminCheck** — форма для ввода пароля администратора. При успешной авторизации открывает форму AdminSite.
- **AdminSite** — основная форма для работы в режиме администратора. Содержит таблицу (DataGridView) для отображения списка рейсов и набор кнопок для управления базой данных: добавление, изменение, удаление, настройка фильтров, изменение пути к файлу.
- **UserSite** — форма для работы в режиме пользователя. Наследуется от **AdminSite**, но скрывает кнопки, связанные с редактированием базы данных.
- **AddFlight** — форма для добавления нового рейса. Включает поля ввода всех характеристик рейса (номер, тип самолёта, пункт назначения, дни недели, время вылета/прилёта, цена).
- **EditFlights** — форма для массового изменения записей. Позволяет выбрать поля для изменения, указать новые значения и применить изменения либо ко всем записям, подходящим под фильтр, либо к выбранным номерам.
- **DeleteFlights** — форма для удаления записей. Аналогично форме редактирования позволяет удалять записи по фильтру или по списку номеров.

- **SetFilters** — форма для настройки фильтров поиска. Отображает текущие фильтры по каждому полю и позволяет добавлять, редактировать или удалять условия.
- **EditFilterField** — вспомогательная форма для ввода значения конкретного поля фильтра с выбором оператора сравнения и логического оператора.
- **ChangePath** — форма для изменения пути к файлу базы данных. Использует стандартные диалоги `SaveFileDialog` и `OpenFileDialog`.

3.2.2 Игровые модули

- **StartGame** — стартовое меню игры. Отображает сюжетное описание, лучший результат и кнопки запуска игры и выхода.
- **TheZondbie** — основная игровая форма. Реализует механику зомби-шутера: управление оружием, движение противников, систему таймеров, звуковое сопровождение и сохранение рекордов.

3.2.3 Вспомогательные модули

- `structure.h / structure.cpp` — содержат определение структур `Flight` (запись о рейсе) и `Flight_filter` (структура фильтра с полями `apply` и `logic`), а также функции для работы с ними.
- `file_funcs.h / file_funcs.cpp` — функции для работы с файлами: получение пути к БД, изменение пути, чтение и запись строк, чтение и запись фильтров, а также вспомогательные функции для редактирования и удаления записей.

3.2.4. Взаимодействие модулей

Взаимодействие модулей показано на рисунке 1. Главная форма FlightMenu является точкой входа. При выборе режима «Администратор» последовательно вызываются формы AdminCheck → AdminSite. При выборе режима «Пользователь» сразу открывается UserSite. При выборе «Запустить игру» открывается StartGame, а из неё — TheZondbie. Все формы работы с базой данных (AdminSite, UserSite, AddFlight, EditFlights и др.) используют вспомогательные модули structure и file_funcs для доступа к CSV-файлам и выполнения операций с данными.

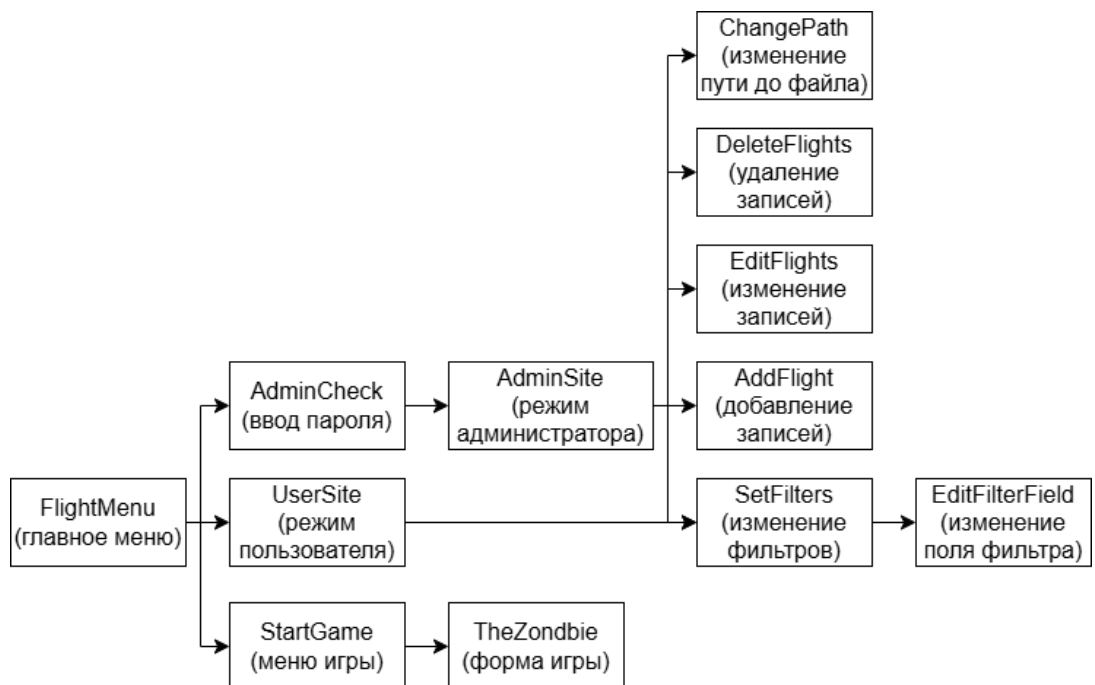


Рисунок 1 – Взаимодействие модулей

3.3 Описание реализации ПО

Разработанное приложение представляет собой многооконный программный комплекс с графическим интерфейсом, реализованный на языке C++/CLI в среде Microsoft Visual Studio. Взаимодействие между формами построено на принципе диалоговых окон и сокрытии родительских форм.

3.3.1 Главное меню (FlightMenu.h)

Главная форма является точкой входа в приложение. На форме расположены три кнопки управления (Button): «Администратор», «Пользователь» и «Запустить игру». Метка Label «Последние новости» и графический элемент PictureBox в текущей версии реализованы лишь в качестве имитации портала с новостями и оставлены для дальнейшего расширения.

При нажатии на кнопку «Администратор» создается экземпляр формы AdminCheck. При нажатии на кнопку «Пользователь» — форма UserSite. При выборе «Запустить игру» аналогичным образом открывается форма StartGame. При любом выборе главное меню скрывается методом Hide().

3.3.2 Авторизация администратора (AdminCheck.h)

Форма AdminCheck предназначена для ввода пароля. Она содержит текстовое поле TextBox и кнопку «Войти». Пароль хранится в файле конфигурации flights_data/settings.txt. При нажатии на кнопку «Войти» содержимое поля ввода сравнивается с паролем, считанным функцией get_password() из файла. При успешной авторизации текущая форма закрывается, а форма AdminSite отображается. При неверном пароле выводится сообщение об ошибке.

3.3.3 Работа с базой данных (AdminSite.h / UserSite.h)

Форма AdminSite и унаследованная от неё UserSite являются центральными для работы с данными. Основной компонент — DataGridView с именем data_base. При загрузке формы вызывается метод SetupDataBase(), который программно создает колонки таблицы («Номер записи», «Номер рейса», «Тип самолёта» и т.д.).

Заполнение таблицы происходит в методе ReadDataBase(). Алгоритм следующий:

1. Функция get_file() получает путь к CSV-файлу из settings.txt.
2. Файл открывается для чтения. Если файла нет, он создается.
3. Функция read_filters() загружает текущие фильтры из файла flights_data/filters.txt.
4. В цикле построчно читается файл с рейсами. Строка парсится в структуру Flight.
5. Вызывается функция compare_flight(), которая проверяет, соответствует ли запись заданным фильтрам.
6. Если запись подходит, данные конвертируются в удобный для отображения формат (например, время из минут в «ЧЧ:ММ», дни недели — в строку через запятую) и добавляются в data_base методом Rows->Add().

Слева от таблицы расположены кнопки действий. В режиме администратора доступны: «Настроить фильтры», «Добавить рейс», «Изменение рейсов», «Удаление рейсов», «Изменить путь до файла», «Обновить таблицу». В режиме пользователя (UserSite) кнопки добавления, изменения и удаления наследуются, но не используются (скрыты). Кнопка «Выйти в главное меню» закрывает текущую форму и показывает FlightMenu.

3.3.4 Добавление рейса (AddFlight.h)

Форма AddFlight отвечает за ввод новой записи. Компоненты ввода:

- TextBox для номера рейса, типа самолёта, пункта назначения и цены.
- MaskedTextBox с маской «00:00» для полей времени вылета и прилёта.
- Группа CheckBox для выбора дней недели (от 1 до 7).

При нажатии на кнопку «Добавить» происходит валидация введенных данных с использованием блоков try-catch (для чисел) и проверок на пустоту. Если все поля корректны, формируется структура Flight. Функция write_line() дописывает эту структуру в конец CSV-файла.

3.3.5 Изменение/удаление рейсов (EditFlights.h / DeleteFlights.h)

Формы EditFlights и DeleteFlights имеют схожую логику работы. Пользователь сначала выбирает, к каким записям применить действие:

- **«Все подходящие под фильтр»** — изменение/удаление применяется ко всем записям, которые удовлетворяют условиям, заданным на форме SetFilters.
- **«По выбранным номерам»** — действие применяется только к конкретным номерам записей, введенным пользователем в TextBox в виде списка через запятую или пробел.

Форма EditFlights дополнительно содержит набор CheckBox для выбора полей, которые необходимо изменить, и соответствующие поля для ввода новых значений. При нажатии «Применить» вызывается функция edit_with_example(), которая с помощью вспомогательного файла последовательно обрабатывает исходный CSV-файл, заменяя нужные поля в выбранных строках.

3.3.6 Настройка фильтров (SetFilters.h / EditFilterField.h)

Механизм фильтрации является ключевой особенностью приложения. Форма SetFilters отображает список всех полей БД и текущее состояние фильтров по каждому полю. При нажатии на кнопку с названием поля (например, «Номер рейса») правая часть формы расширяется, показывая панель управления фильтрами для этого поля.

Для каждого поля можно создать до двух условий, объединенных логическими операторами «И» / «ИЛИ». Форма EditFilterField предназначена для ввода значения фильтра. Для полей с числами и временем дополнительно доступен выбор оператора сравнения (>, <, =, >=, <=) через группу RadioButton. Сохраненные фильтры записываются в файл filters.txt с помощью функции write_filters().

3.3.7 Изменение пути к файлу (ChangePath.h)

Форма ChangePath позволяет администратору сменить файл базы данных. Интерфейс включает:

- Поле для отображения текущего пути.
- Переключатели (RadioButton) для выбора режима: «Создать новый файл» или «Выбрать существующий».
- Кнопки «Создать»/«Выбрать», вызывающие стандартные диалоги Windows (SaveFileDialog / OpenFileDialog).
- CheckBox для опций «Скопировать данные из текущего файла в новый» и «Удалить текущий файл».

После подтверждения изменения сохраняются в settings.txt.

3.3.8 Игровой модуль (StartGame.h / TheZondbie.h)

В рамках дополнительного задания была разработана игра «Зомби-шутер» (The Zondbie). Игровой процесс построен на отражении волн зомби, стремящихся добраться до игрока. Форма StartGame является стартовым

меню игры, содержащим описание сюжета («лор»), кнопку «Начать защиту» для запуска игрового процесса, кнопку «Вернуться в меню» для выхода в главное меню приложения, а также метку `best_time_label` для отображения лучшего времени выживания, загруженного из файла `game_data/score.txt`.

Основная игровая механика реализована в форме `TheZondbie`. Форма содержит следующие ключевые компоненты:

- `PictureBox street_box` — игровое поле (улица), на котором отображаются движущиеся противники.
- `PictureBox Peet_closed_box` и `Peet_open_box` — визуальные элементы, отображающие закрытое и открытое ружьё соответственно. При нажатии на закрытое ружьё открывается доступ к патронам.
- `PictureBox bullet_done_left/right_box` и `bullet_ready_left/right_box` — индикаторы состояния патронов. Для перезарядки игрок должен последовательно нажать на две «пустые» пули, после чего они становятся заряженными и готовыми к выстрелу.
- `Timer` — три таймера обеспечивают динамику игры: `game_timer` (обновление позиций зомби, ~60 FPS), `spawn_timer` (появление новых противников каждые 1,5 секунды), `animation_timer` (смена кадров анимации зомби), `world_time` (подсчёт времени выживания).

Управление оружием. При нажатии на закрытое ружьё (`Peet_closed_box`) воспроизводится звук `open_Peet_sound`, ружьё открывается (`Peet_open_box` становится видимым), становятся доступны для перезарядки индикаторы пуль. Заряжание осуществляется кликом по `bullet_done_left_box` и `bullet_done_right_box` — каждый клик воспроизводит звук `new_bullet_sound`, уменьшает количество доступных для заряжания пуль и увеличивает счётчик оставшихся патронов. После зарядки обеих пуль ружьё нужно закрыть.

Стрельба. Выстрел производится кликом по любому зомби на игровом поле. Обработчик `Zondbie_Click` проверяет: открыто ли ружьё и есть ли патроны. Если условия соблюдены, зомби удаляется с поля, количество патронов уменьшается, воспроизводится звук (`shoot_sound` или `zondbie_death_sound`), а скорость всех оставшихся зомби увеличивается (поле `zondbie_speed`).

Поведение противников. Зомби (объекты `PictureBox`) создаются в методе `SpawnZondbie`. Каждый зомби появляется с левого или правого края игрового поля (случайный выбор). Траектория движения рассчитана таким образом, что зомби движутся по горизонтали и одновременно смещаются по вертикали, стремясь к нижней центральной точке (`centerX` поля). Позиция каждого зомби пересчитывается в методе `GameTick`. Анимация ходьбы реализована через переключение двух кадров (`Zombie_left_1/2.png`, `Zombie_right_1/2.png`) в обработчике `AnimationTick`. Проигрыш наступает, если любой зомби достигает центральной точки (`pos.X >= centerX - ZONDBIE_SIZE_X/2`).

Звуковое сопровождение. Для каждого игрового события создан отдельный объект `SoundPlayer`, загружающий WAV-файлы из папки `game_data/`:

- `shoot_sound` — звук удачного выстрела;
- `open_Peet_sound` — звук открытия/закрытия ружья;
- `new_bullet_sound` — звук заряжания патрона;
- `zondbie_death_sound` — звук уничтожения зомби;
- `game_over_sound` — звук окончания игры.

Сохранение рекордов. В конструкторе формы `StartGame` из файла `game_data/score.txt` считывается лучший результат с помощью `StreamReader` и отображается в метке `best_time_label`. В деструкторе формы `TheZondbie` текущее время выживания (`time_passed`) сравнивается с сохранённым

рекордом. При превышении новое значение записывается в файл через StreamWriter.

Завершение игры. Игра завершается при достижении любого зомби центральной точки поля либо при нажатии кнопки «Сдаться» (exit_button). В обоих случаях вызывается метод Close(), который в деструкторе формы сохраняет рекорд, проигрывает звук окончания игры, выводит сообщение «Вы держались как могли...» и автоматически показывает стартовую форму StartGame.

Заключение

В результате выполнения данного курсового проекта (практики) разработано программное обеспечение «Mark Flights», позволяющее эффективно работать с базой данных авиарейсов в двух режимах: администратора и пользователя. Программа имеет удобный графический интерфейс, реализованный на платформе Windows Forms в среде Microsoft Visual Studio 2022 на языке C++/CLI.

В процессе выполнения работы были решены следующие задачи:

- Разработана структура данных для хранения информации об авиарейсах (номер рейса, тип самолёта, пункт назначения, дни отправления, время вылета и прилёта, цена билета).
- Реализованы все основные функции работы с базой данных: добавление, редактирование, удаление и вывод записей.
- Создана гибкая система фильтрации, позволяющая выполнять запросы по любой комбинации полей с использованием логических операторов «И» и «ИЛИ».
- Реализовано разграничение прав доступа: администратор имеет полный доступ к редактированию базы данных, пользователь — только к просмотру и поиску.
- Разработан механизм изменения пути к файлу базы данных с возможностью копирования данных и удаления старого файла.
- В качестве дополнительного задания реализована игра «Зомби-шутер» с графическим интерфейсом, таймерами, анимацией, звуковым сопровождением и системой сохранения рекордов.

Основные преимущества разработанного приложения:

- Интуитивно понятный графический интерфейс;
- Разделение режимов пользователя и администратора;

- Возможность массового редактирования и удаления записей;
- Изменение пути к файлу базы данных без перекомпиляции программы;
- Наличие механизма фильтрации с поддержкой сложных логических условий;
- Наличие дополнительного игрового модуля для организации досуга.

Основные недостатки приложения:

- Для работы необходимы специально отформатированные файлы (CSV с разделителем «;»);
- Отсутствие возможности шифрования пароля администратора;
- Невозможность работы с несколькими таблицами одновременно;
- Отсутствие экспорта данных в другие форматы (JSON, XML, Excel);
- Графический интерфейс игры требует доработки анимации и баланса сложности.

Возможные направления дальнейшего усовершенствования:

- Добавление шифрования пароля и данных;
- Расширение функциональности для работы с несколькими таблицами одновременно;
- Добавление возможности экспорта данных в другие форматы (например, JSON, XML);
- Разработка модуля экспорта и печати отчётов;
- Улучшение игровой механики (добавление уровней, боссов, систематическое увеличение сложности).

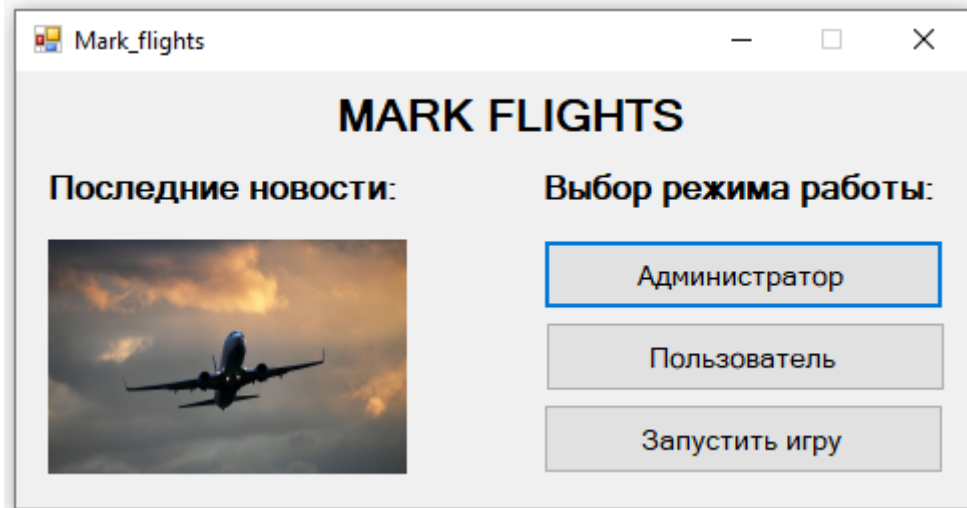
Список использованных источников

- 1 Егорова Е.В. Программирование на языке СИ. Учебное пособие/Алт. Госуд. Технич. Ун-т им. И.И.Ползунова.-Барнаул:2013.-184с.
- 2 “MSDN” – информационный сервис для разработчиков [Электронный ресурс] – Режим доступа: <http://msdn.microsoft.com/ru-ru/ms348103/library>, свободный.
- 3 Онлайн справочник программиста на С и С++ [Электронный ресурс]. Режим доступа: <http://www.c-cpp.ru/>, свободный.

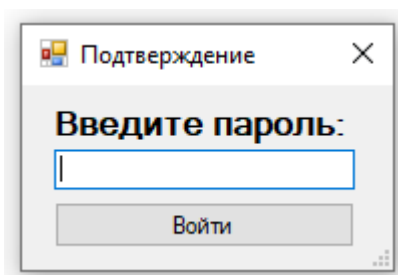
Приложение А

Снимки экранных форм пользовательского интерфейса

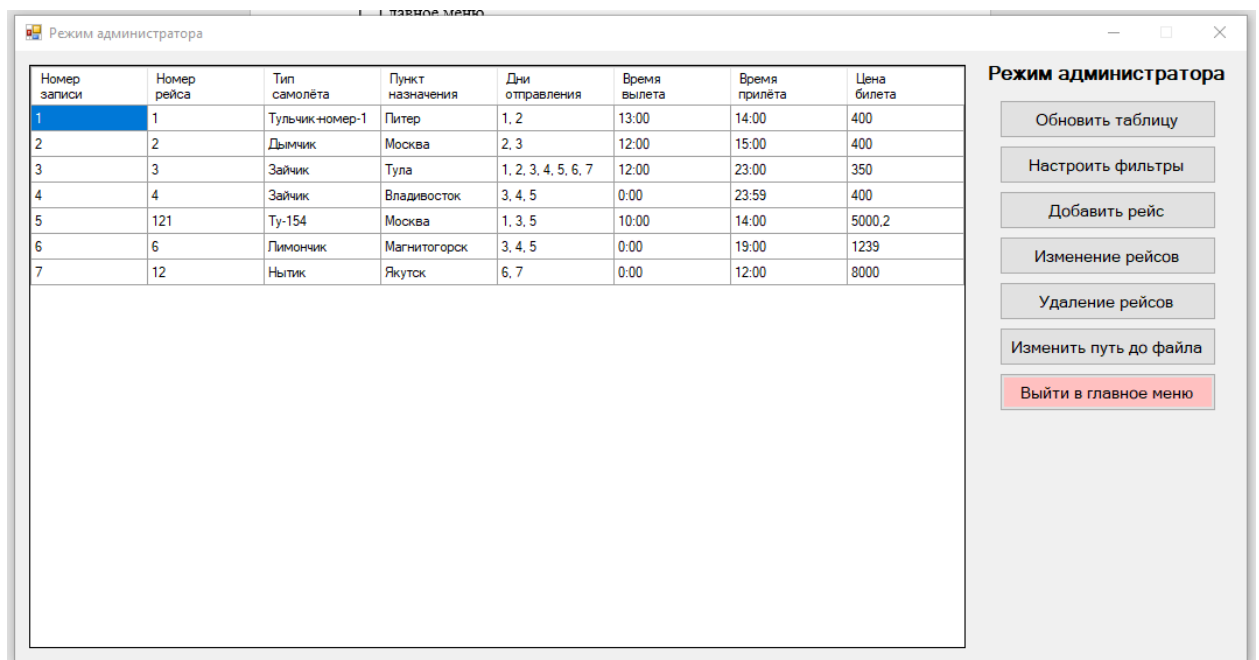
1. Главное меню



2. Ввод пароля для режима администратора



3. Режим администратора



4. Настройка фильтров (+ настройка первого поля)

Настройка фильтров

Нажмите на название поля, чтобы редактировать его фильтр

Номер рейса не используется

Тип самолёта не используется

Пункт назначения не используется

Дни отправления не используется

Время вылета не используется

Время прилёта не используется

Цена билета не используется

Обновить список

Поле номер рейса

1) - не используется Удалить Редактировать

2) - не используется Удалить Редактировать

Добавить новый фильтр Удалить все фильтры

5. Установка нового значения поля фильтра

Новое значение

Новое значение

Сравнение

Логический операнд

☒ И ☐ > ☐ <

☐ ИЛИ ☐ =

☐ >= ☐ <=

Сохранить

6. Добавление записи

Добавление рейса

Номер рейса

Тип самолёта

Пункт назначения

Дни отправления

☐ 1 (понедельник)

☐ 2 (вторник)

☐ 3 (среда)

☐ 4 (четверг)

☐ 5 (пятница)

☐ 6 (суббота)

☐ 7 (воскресенье)

Время вылета 00:00

Время прилёта 00:00

Цена билета

Добавить

7. Изменение записей

Изменение рейсов

×

Новые данные

Выберите нужные поля

☐ Номер рейса

☐ Тип самолёта

☐ Пункт назначения

☐ Дни отправления

☐ - 1 (понедельник)

☐ - 2 (вторник)

☐ - 3 (среда)

☐ - 4 (четверг)

☐ - 5 (пятница)

☐ - 6 (суббота)

☐ - 7 (воскресенье)

☐ Время вылета

☐ Время прилёта

☐ Цена билета

00:00

00:00

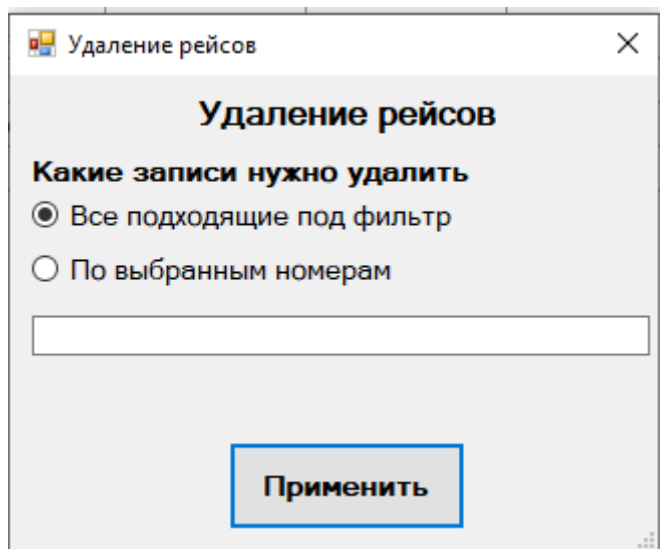
Какие записи нужно изменить

☒ Все подходящие под фильтр

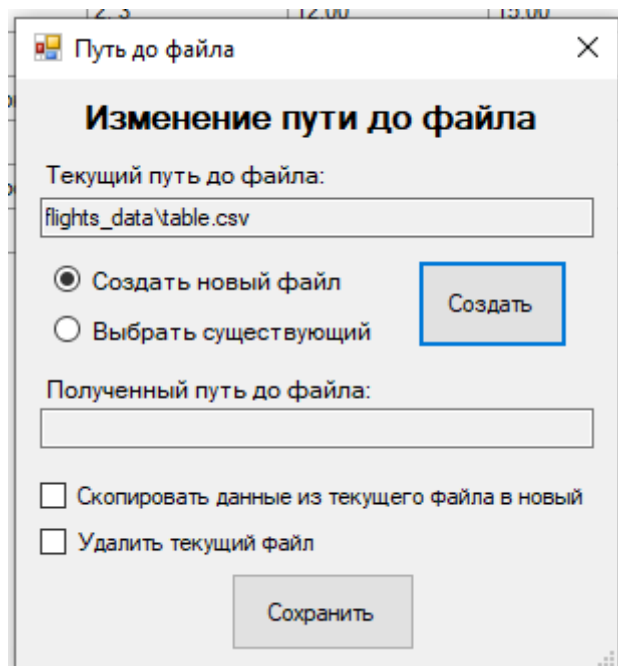
☐ По выбранным номерам

Применить

8. Удаление записей



9. Изменение пути до файла



10.Режим пользователя

Режим пользователя

Номер записи	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	1	Тулчик номер-1	Питер	1, 2	13:00	14:00	400
2	2	Дымчик	Москва	2, 3	12:00	15:00	400
3	3	Зайчик	Тула	1, 2, 3, 4, 5, 6, 7	12:00	23:00	350
4	4	Зайчик	Владивосток	3, 4, 5	0:00	23:59	400
5	121	Ту-154	Москва	1, 3, 5	10:00	14:00	5000,2
6	6	Лимончик	Магнитогорск	3, 4, 5	0:00	19:00	1239
7	12	Нытик	Якутск	6, 7	0:00	12:00	8000

Режим пользователя

Обновить таблицу

Настроить фильтры

Выйти в главное меню

11. Главное меню игры

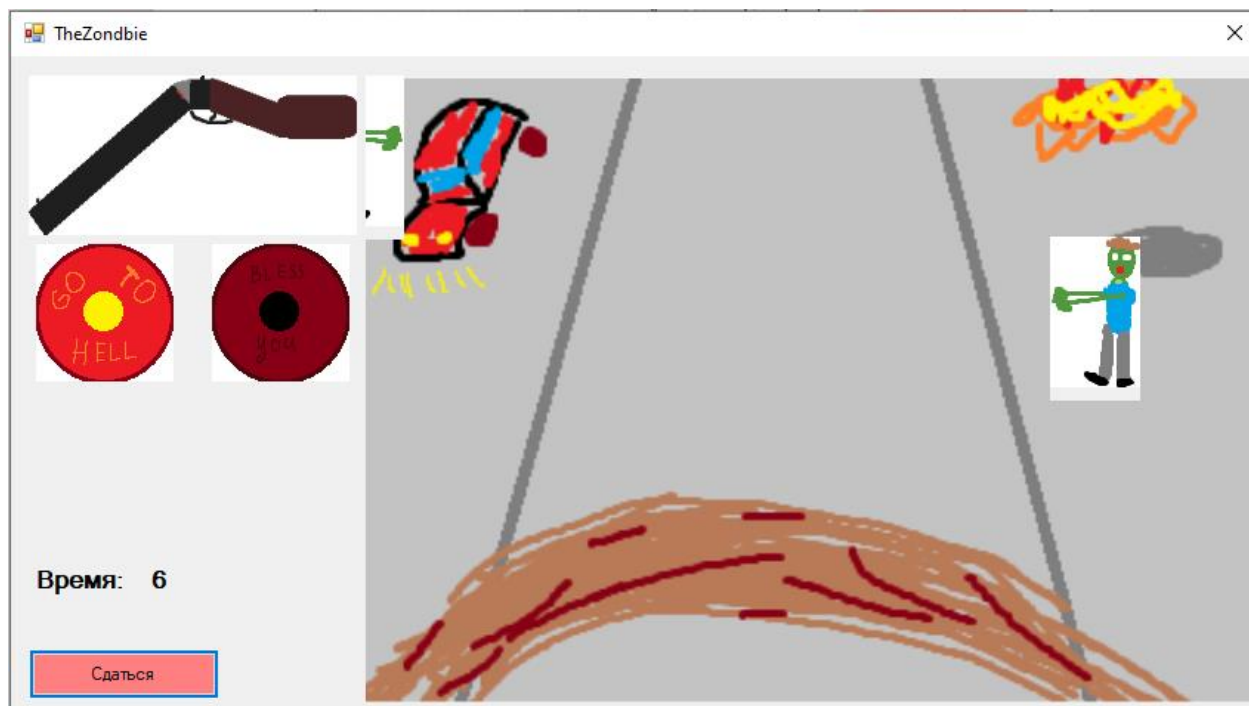
The Zondbie

Обычный вечер июня 2026 года.
Ничего не предвещало беды.
За окном светило солнце, а у вас было отличное настроение.
Неожиданно по радио заиграли предупреждения о новом вирусе и место эвакуации.
Вы взяли своего лучшего друга - двухствольную винтовку и вышли на улицу.
Через толпы заражённых вы пробрались к
назначенному месту, но места на борту уже не было.
Вы согласились подождать следующий вертолёт - осталось лишь разобраться с гостями...
Постарайтесь прожить как можно дольше... Лучшее время: 52

Начать защиту

Вернуться в меню

12.Игра



Приложение Б

Исходный код программы

П.Б.1 Код FlightMenu.cpp

```
#include "FlightMenu.h"
#include <Windows.h>
using namespace MarkflightsUI; // Название проекта

[STAThread]
int WINAPI WinMain(HINSTANCE, HINSTANCE, LPSTR, int) {
    SetConsoleCP(1251); SetConsoleOutputCP(1251);
    Application::EnableVisualStyles();
    Application::SetCompatibleTextRenderingDefault(false);
    Application::Run(gcnew FlightMenu);
    return 0;
}
```

П.Б.2 Код FlightMenu.h

```
#pragma once

#include "AdminCheck.h"
#include "UserSite.h"
#include "StartGame.h"

namespace MarkflightsUI {
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;

    /// <summary>
    /// Сводка для FlightMenu
    /// </summary>
    public ref class FlightMenu : public System::Windows::Forms::Form
    {
    public:
        FlightMenu(void)
        {
            InitializeComponent();
        }

    protected:
        /// <summary>
        /// Освободить все используемые ресурсы.
        /// </summary>
        ~FlightMenu()
        {
            if (components)
            {
                delete components;
            }
        }
    private: System::Windows::Forms::Label^ name_label;
```



```

private: System::Windows::Forms::Label^ news_label;
private: System::Windows::Forms::Label^ mode_label;
private: System::Windows::Forms::Button^ as_admin_button;
private: System::Windows::Forms::Button^ as_player_button;
private: System::Windows::Forms::Button^ as_user_button;
private: System::Windows::Forms::PictureBox^ pictureBox1;

private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container^ components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        System::ComponentModel::ComponentResourceManager^ resources =
(gcnew System::ComponentModel::ComponentResourceManager(FlightMenu::typeid));
        this->name_label = (gcnew System::Windows::Forms::Label());
        this->news_label = (gcnew System::Windows::Forms::Label());
        this->mode_label = (gcnew System::Windows::Forms::Label());
        this->as_admin_button = (gcnew
System::Windows::Forms::Button());
        this->as_player_button = (gcnew
System::Windows::Forms::Button());
        this->as_user_button = (gcnew System::Windows::Forms::Button());
        this->pictureBox1 = (gcnew
System::Windows::Forms::PictureBox());

        (cli::safe_cast<System::ComponentModel::ISupportInitialize>(this-
>pictureBox1))->BeginInit();
        this->SuspendLayout();
        //
        // name_label
        //
        this->name_label->AutoSize = true;
        this->name_label->FlatStyle =
System::Windows::Forms::FlatStyle::Flat;
        this->name_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 16, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->name_label->Location = System::Drawing::Point(155, 9);
        this->name_label->Margin = System::Windows::Forms::Padding(0);
        this->name_label->Name = L"name_label";
        this->name_label->Size = System::Drawing::Size(185, 26);
        this->name_label->TabIndex = 0;
        this->name_label->Text = L"MARK FLIGHTS";
        //
        // news_label
        //
        this->news_label->AutoSize = true;
        this->news_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->news_label->Location = System::Drawing::Point(12, 48);
        this->news_label->Name = L"news_label";
        this->news_label->Size = System::Drawing::Size(183, 20);
    }

```

```

        this->news_label->TabIndex = 1;
        this->news_label->Text = L"Последние новости:";
        //
        // mode_label
        //
        this->mode_label->AutoSize = true;
        this->mode_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->mode_label->Location = System::Drawing::Point(259, 48);
        this->mode_label->Name = L"mode_label";
        this->mode_label->Size = System::Drawing::Size(204, 20);
        this->mode_label->TabIndex = 2;
        this->mode_label->Text = L"Выбор режима работы:";
        //
        // as_admin_button
        //
        this->as_admin_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->as_admin_button->Location = System::Drawing::Point(263,
84);

        this->as_admin_button->Name = L"as_admin_button";
        this->as_admin_button->Size = System::Drawing::Size(200, 35);
        this->as_admin_button->TabIndex = 3;
        this->as_admin_button->Text = L"Администратор";
        this->as_admin_button->UseVisualStyleBackColor = true;
        this->as_admin_button->Click += gcnew System::EventHandler(this,
&FlightMenu::as_admin_button_Click);
        //
        // as_player_button
        //
        this->as_player_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->as_player_button->Location = System::Drawing::Point(263,
166);

        this->as_player_button->Name = L"as_player_button";
        this->as_player_button->Size = System::Drawing::Size(200, 35);
        this->as_player_button->TabIndex = 5;
        this->as_player_button->Text = L"Запустить игру";
        this->as_player_button->UseVisualStyleBackColor = true;
        this->as_player_button->Click += gcnew
System::EventHandler(this, &FlightMenu::as_player_button_Click);
        //
        // as_user_button
        //
        this->as_user_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->as_user_button->Location = System::Drawing::Point(264,
125);

        this->as_user_button->Name = L"as_user_button";
        this->as_user_button->Size = System::Drawing::Size(200, 35);
        this->as_user_button->TabIndex = 7;
        this->as_user_button->Text = L"Пользователь";
        this->as_user_button->UseVisualStyleBackColor = true;
        this->as_user_button->Click += gcnew System::EventHandler(this,
&FlightMenu::as_user_button_Click);

```

```

        //
        // pictureBox1
        //
        this->pictureBox1->Image =
(cli::safe_cast<System::Drawing::Image^>(resources-
>GetObject(L"pictureBox1.Image")));
        this->pictureBox1->Location = System::Drawing::Point(16, 84);
        this->pictureBox1->Name = L"pictureBox1";
        this->pictureBox1->Size = System::Drawing::Size(179, 117);
        this->pictureBox1->SizeMode =
System::Windows::Forms::PictureBoxSizeMode::StretchImage;
        this->pictureBox1->TabIndex = 8;
        this->pictureBox1->TabStop = false;
        //
        // FlightMenu
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(476, 218);
        this->Controls->Add(this->pictureBox1);
        this->Controls->Add(this->as_user_button);
        this->Controls->Add(this->as_player_button);
        this->Controls->Add(this->as_admin_button);
        this->Controls->Add(this->mode_label);
        this->Controls->Add(this->news_label);
        this->Controls->Add(this->name_label);
        this->MaximizeBox = false;
        this->MinimumSize = System::Drawing::Size(492, 257);
        this->Name = L"FlightMenu";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"Mark_flights";

        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>pictureBox1))->EndInit();
        this->ResumeLayout(false);
        this->PerformLayout();

    }
#pragma endregion
    private: System::Void as_admin_button_Click(System::Object^ sender,
System::EventArgs^ e) {
        AdminCheck^ p = gcnew AdminCheck();
        p->ShowDialog();
    }
    private: System::Void as_user_button_Click(System::Object^ sender,
System::EventArgs^ e) {
        UserSite^ p = gcnew UserSite();
        p->Show();
        this->Hide();
    }
    private: System::Void as_player_button_Click(System::Object^ sender,
System::EventArgs^ e) {
        StartGame^ p = gcnew StartGame();
        p->Show();
        this->Hide();
    }
}
};
}

```

П.Б.3 Код structure.h

```
#pragma once

#include <stdio.h>

#define TEXT_LEN 32      // Сколько памяти нужно выделить на хранение текстовых
                          // полей
#define FIELDS_NUM 7     // Количество полей структуры
#define FILTERS_NUM 2    // Сколько фильтров можно одновременно использовать

// Структура информации о полёте
typedef struct {
    int fnum;              // Номер рейса (flight number)
    char name[TEXT_LEN];   // Тип самолёта (название модели)
    char dest[TEXT_LEN];   // Пункт назначения (destination)
    int days[7];           // Дни отправления
    int dep_time;          // Время вылета в минутах (departure time)
    int arr_time;          // Время прилёта в минутах (arrival time)
    double price;          // Цена билета (десятичная дробь)
} Flight;

// Структура для фильтров записей
typedef struct {
    int fnum;              // Номер рейса (flight number)
    char name[TEXT_LEN];   // Тип самолёта (название модели)
    char dest[TEXT_LEN];   // Пункт назначения (destination)
    int days[7];           // Дни отправления
    int dep_time;          // Время вылета в минутах (departure time)
    int arr_time;          // Время прилёта в минутах (arrival time)
    double price;          // Цена билета (десятичная дробь)

    int apply[FIELDS_NUM]; // Какие столбцы нужно фильтровать (и каким образом)
    int logic[FIELDS_NUM]; // Какую операцию нужно применять при сложении
                          // фильтров
} Flight_filter;

// Соответствует ли запись всем заданным фильтрам
int compare_flight(Flight flight, Flight_filter filters[]);

// Удаление и изменение записей
// Открыть текущий файл table.csv для чтения и dable.csv для записи
int open_dable(char *file_path, char *new_file_path, FILE **new_table, FILE
**old_table);

// Изменение всех записей по фильтру или по указанным номерам
int edit_with_example(Flight_filter example, int filtered, int* to_edit, int
edit_len);

// Удаление всех записей по фильтру или по указанным номерам
int delete_flights(int filtered, int* to_del, int del_len);
```

П.Б.4 Код structure.cpp

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>          // Для FILE, fopen, fclose, rewind, rename, remove
#include <stdlib.h>         // Для _itoa
#include <string.h>         // Для strcpy, strstr
```

```

#include "structure.h"
#include "file_funcs.h"          // Для FILE_NAME_LEN, get_file, read_line,
write_line, count_lines

#include <errno.h> // Обязательно подключить

// Стандартный strlwr не работает(
char* mystrlwr(char* string) {
    // Преобразовать буквы верхнего регистра строки string в буквы нижнего
    регистра
    // string: изменяемая строка
    for (int i = 0; string[i]; i++)
        if (('A' <= string[i]) && (string[i] <= 'Z')) string[i] += 'a' - 'A';
        else if (('A' <= string[i]) && (string[i] <= 'Я')) string[i] += 'a' -
'A';
    return string;
}

// Соответствует ли запись всем заданным фильтрам
int compare_flight(Flight flight, Flight_filter filters[]) {
    int is_good = 1; // Запись изначально подходит
    for (int i = 0; i < FIELDS_NUM && is_good; i++)
        for (int fil = 0; fil < FILTERS_NUM; fil++) {
            Flight_filter cur_filter = filters[fil]; // Текущий фильтр
            if (!cur_filter.apply[i]) continue; // Пропускаем,
если его не нужно применить
            int cur_good = 1; // Подходит ли текущее поле под текущий
фильтр
            if (i == 0) { // Поле fnum
                char flight_fnum[16], filter_fnum[16];
                _itoa(flight.fnum, flight_fnum, 10);
                _itoa(cur_filter.fnum, filter_fnum, 10);
                if (strstr(flight_fnum, filter_fnum) == NULL) cur_good =
0;
            }
            if (i == 1) { // Поле name
                char flight_name[16], filter_name[16];
                mystrlwr(strcpy(flight_name, flight.name));
                mystrlwr(strcpy(filter_name, cur_filter.name));
                if (strstr(flight_name, filter_name) == NULL) cur_good =
0;
            }
            if (i == 2) { // Поле dest
                char flight_dest[16], filter_dest[16];
                mystrlwr(strcpy(flight_dest, flight.dest));
                mystrlwr(strcpy(filter_dest, cur_filter.dest));
                if (strstr(flight_dest, filter_dest) == NULL) cur_good =
0;
            }
            if (i == 3) { // Поле days
                int common = 0;
                for (int i = 0; i < 7 && cur_filter.days[i]; i++)
                    for (int j = 0; j < 7 && flight.days[j]; j++)
                        if (cur_filter.days[i] == flight.days[j])
common += 1;
                if (!common) cur_good = 0;
            }
            if (i == 4) { // Поле dep_time
                if (cur_filter.apply[4] == 1) // flight > filter
                    if (flight.dep_time <= cur_filter.dep_time) cur_good
= 0;
                if (cur_filter.apply[4] == 2) // flight < filter

```

```

        if (flight.dep_time >= cur_filter.dep_time) cur_good
= 0;
        if (cur_filter.apply[4] == 3) // flight = filter
            if (flight.dep_time != cur_filter.dep_time) cur_good
= 0;
        if (cur_filter.apply[4] == 4) // flight >= filter
            if (flight.dep_time < cur_filter.dep_time) cur_good
= 0;
        if (cur_filter.apply[4] == 5) // flight <= filter
            if (flight.dep_time > cur_filter.dep_time) cur_good
= 0;
    }
    if (i == 5) { // Поле arr_time
        if (cur_filter.apply[5] == 1) // flight > filter
            if (flight.arr_time <= cur_filter.arr_time) cur_good
= 0;
        if (cur_filter.apply[5] == 2) // flight < filter
            if (flight.arr_time >= cur_filter.arr_time) cur_good
= 0;
        if (cur_filter.apply[5] == 3) // flight = filter
            if (flight.arr_time != cur_filter.arr_time) cur_good
= 0;
        if (cur_filter.apply[5] == 4) // flight >= filter
            if (flight.arr_time < cur_filter.arr_time) cur_good
= 0;
        if (cur_filter.apply[5] == 5) // flight <= filter
            if (flight.arr_time > cur_filter.arr_time) cur_good
= 0;
    }
    if (i == 6) { // Поле price
        if (cur_filter.apply[6] == 1) // flight > filter
            if (flight.price <= cur_filter.price) cur_good = 0;
        if (cur_filter.apply[6] == 2) // flight < filter
            if (flight.price >= cur_filter.price) cur_good = 0;
        if (cur_filter.apply[6] == 3) // flight = filter
            if (flight.price != cur_filter.price) cur_good = 0;
        if (cur_filter.apply[6] == 4) // flight >= filter
            if (flight.price < cur_filter.price) cur_good = 0;
        if (cur_filter.apply[6] == 5) // flight <= filter
            if (flight.price > cur_filter.price) cur_good = 0;
    }
    // Применяем логическую операцию
    if (cur_filter.logic[i] == 0) is_good = is_good &&
cur_good;
    else if (cur_filter.logic[i] == 1) is_good = is_good ||
cur_good;

    }
    return is_good;
}

// Открыть текущий файл table.csv для чтения и dable.csv для записи
int open_dable(char *file_path, char *new_file_path, FILE **new_table, FILE
**old_table) {
    /*
    Возвращает:
        -1: Недостаток прав
        0: Успешное открытие
    */
    *strstr(strcpy(new_file_path, file_path), "table") = 'd'; // Тот же путь,
только ../dable.csv
    // Открываем dable.csv для записи
    *new_table = fopen(new_file_path, "w");

```

```

    if (new_table == NULL) return -1;
    // Открываем текущий файл table.csv для чтения
    *old_table = fopen(file_path, "r");
    if (old_table == NULL) {
        fclose(*new_table);
        return -1;
    }
    return 0;
}

// Изменение всех записей по фильтру или по указанным номерам
int edit_with_example(Flight_filter example, int filtered,
    int* to_edit, int edit_len) {
    /*
    Возвращает:
        -2: Ошибка чтения
        -1: Недостаток прав
        0: Успешное изменение
    */
    int is_error = 0; // Ошибка чтения
    // Получим путь до файла
    char file_path[FILE_NAME_LEN];
    if (get_file(file_path) == -1) return -1;
    // Создаём новый файл для записи
    char new_file_path[FILE_NAME_LEN];
    FILE* new_table, * old_table;
    if (open_dable(file_path, new_file_path, &new_table, &old_table) == -1)
return -1;
    // Получаем фильтры
    Flight_filter filters[FILTERS_NUM];
    read_filters(filters);

    Flight buffer; // Буфер для чтения полёта
    int len, fil_len; // Количество записей в файле
    count_lines(old_table, filters, &len, &fil_len);
    rewind(old_table);

    if (filtered) { // Если нужно изменить все подходящие под фильтр записи
        // Если строка не подходит под фильтр, то переписываем её
        // Иначе изменяем её по шаблону и записываем
        for (int cur_line = 0; cur_line < len && !is_error; cur_line++)
            if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
            else if (!compare_flight(buffer, filters)) write_line(new_table,
buffer);
        else {
            for (int i = 0; i < FIELDS_NUM; i++)
                if (i == 0 && example.apply[i])
buffer.fnum = example.fnum;
                else if (i == 1 && example.apply[i])
strcpy(buffer.name, example.name);
                else if (i == 2 && example.apply[i])
strcpy(buffer.dest, example.dest);
                else if (i == 3 && example.apply[i]) for (int j =
0; j < 7; j++) buffer.days[j] = example.days[j];
                else if (i == 4 && example.apply[i])
buffer.dep_time = example.dep_time;
                else if (i == 5 && example.apply[i])
buffer.arr_time = example.arr_time;
                else if (i == 6 && example.apply[i]) buffer.price =
example.price;
                write_line(new_table, buffer);
            }
        }
    }
}

```

```

else { // Если изменение по указанным номерам
    int last_edit = 0; // Для прохода по массиву to_edit
    int cur_line = 0; // Для прохода по файлу
    for (; cur_line < len && last_edit < edit_len && !is_error; cur_line++)
        if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
        else if (cur_line + 1 < to_edit[last_edit])
write_line(new_table, buffer);
        else {
            for (int i = 0; i < FIELDS_NUM; i++)
                if (i == 0 && example.apply[i])
buffer.fnum = example.fnum;
                else if (i == 1 && example.apply[i])
strcpy(buffer.name, example.name);
                else if (i == 2 && example.apply[i])
strcpy(buffer.dest, example.dest);
                else if (i == 3 && example.apply[i]) for (int j =
0; j < 7; j++) buffer.days[j] = example.days[j];
                else if (i == 4 && example.apply[i])
buffer.dep_time = example.dep_time;
                else if (i == 5 && example.apply[i])
buffer.arr_time = example.arr_time;
                else if (i == 6 && example.apply[i]) buffer.price =
example.price;
                write_line(new_table, buffer);
                last_edit++;
            }
        for (; cur_line < len && !is_error; cur_line++)
            if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
            else write_line(new_table, buffer);
    }

    // Удаляем прошлый файл и переименовываем новый
    fclose(new_table);
    fclose(old_table);
    remove(file_path);
    rename(new_file_path, file_path);

    if (is_error) return -2;
    return 0;
}

// Удаление всех записей по фильтру или по указанным номерам
int delete_flights(int filtered, int* to_del, int del_len) {
    /*
    Возвращает:
        -2: Ошибка чтения
        -1: Недостаток прав
        0: Успешное изменение
    */

    int is_error = 0; // Ошибка чтения
    // Получим путь до файла
    char file_path[FILE_NAME_LEN];
    if (get_file(file_path) == -1) return -1;
    // Создаём новый файл для записи
    char new_file_path[FILE_NAME_LEN];
    FILE* new_table, * old_table;
    if (open_dable(file_path, new_file_path, &new_table, &old_table) == -1)
return -1;
    // Получаем фильтры
    Flight_filter filters[FILTERS_NUM];
    read_filters(filters);

```



```

Flight buffer; // Буфер для чтения полёта
int len, fil_len; // Количество записей в файле
count_lines(old_table, filters, &len, &fil_len);
rewind(old_table);

if (filtered) { // Если нужно удалить все подходящие под фильтр записи
    // Если строка не подходит под фильтр, то переписываем её
    for (int cur_line = 0; cur_line < len && !is_error; cur_line++)
        if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
        else if (!compare_flight(buffer, filters)) write_line(new_table,
buffer);
}
else { // Если удаление по указанным номерам
    // Удаление будем совершать следующим образом:
    // Если номер строки в массиве – пропускаем её
    int last_del = 0; // Для прохода по массиву to_edit
    int cur_line = 0; // Для прохода по файлу
    for (; cur_line < len && last_del < del_len && !is_error; cur_line++)
        if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
        else if (cur_line + 1 < to_del[last_del]) write_line(new_table,
buffer);
        else last_del++;
    for (; cur_line < len && !is_error; cur_line++)
        if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
        else write_line(new_table, buffer);
}

// Удаляем прошлый файл и переименовываем новый
fclose(new_table);
fclose(old_table);
remove(file_path);
rename(new_file_path, file_path);

if (is_error) return -2;
return 0;
}

```

П.Б.5 Код file_funcs.h

```

#pragma once

#define DIR_NAME "flights_data" // Название папки со всеми данными
#define FILE_NAME "table.csv" // Название файла с таблицей
#define FILE_NAME_LEN 512 // Максимальная длина названия файла
#define CONF_NAME "settings.txt" // Название файла с конфигурацией
#define PASSWORD_LEN 256 // Максимальная длина пароля

#include <stdio.h> // Для FILE
#include "structure.h" // Для Flight, Flight_filter

// Проверка, влезло ли название файла FILE_NAME в строку
int correct_name(char *file_path);

// Получить путь до файла, в который нужно писать данные
int get_file(char *file_path, int buffer_len = FILE_NAME_LEN);

// Получить пароль
int get_password(char* password, int buffer_len = PASSWORD_LEN);

// Записать одну строку файла в структуру
int read_line(FILE *table, Flight *flight);

```

```

// Записать одну структуру таблицы в файл
int write_line(FILE *table, Flight flight);

// Посчитать количество подходящих под фильтр строк
int count_lines(FILE *table, Flight_filter filters[], int *len, int *fil_len);

// Прочитать фильтры из файла DIR_NAME/filters.csv в массив
int read_filters(Flight_filter filterts[FILTERS_NUM]);

// Записать фильтры массива в файл DIR_NAME/filters.csv
int write_filters(Flight_filter filterts[FILTERS_NUM]);

// Перенести данные из String^ в char*
void copy_char_from_string(char* buffer, int len, System::String^ string);

// Перевести данные из String^ формата ЧЧ:ММ в int
int convert_string_time(System::String^ string);

```

П.Б.6 Код file_funcs.cpp

```

#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h> // Для FILE, fopen, fclose, fgets, fputs, printf,
remove
#include <string.h> // Для strlen, strcat, strstr, _strnset
#include <direct.h> // Для _mkdir

#include "file_funcs.h"
#include "structure.h" // Для FIELDS_NUM, Flight, Flight_filter

// Проверка, влезло ли название файла FILE_NAME в строку
int correct_name(char *file_path) {
    char* file_pntr = strstr(file_path, ".csv"); // Указатель на расширение
    if (file_pntr == NULL)
        return 0;
    return 1;
}

// Получить путь до файла, в который нужно напечатывать данные
int get_file(char *file_path, int buffer_len) {
    /*
    Возвращает:
        -1: У пользователя нет прав даже для папки DIR_NAME
        0: Путь до файла успешно получен
        1: Слишком длинный путь (кто-то вручную поменял файл) – запись
        произойдёт по пути DIR_NAME/FILE_NAME
        2: У пользователя нет прав для создания файла – запись произойдёт по
        пути DIR_NAME/FILE_NAME
    */
    _mkdir(DIR_NAME); // Пробуем создать папку на случай, если её нету
    FILE* settings = fopen(DIR_NAME "/" CONF_NAME, "a+");
    if (!settings) return -1;
    rewind(settings);

    // Если файл пустой, то возвращаем путь DIR_NAME/FILE_NAME
    char password[256];
    if (fgets(password, 256, settings) == NULL) {
        strcpy(password, "root");
        strcpy(file_path, DIR_NAME "/" FILE_NAME);
        fclose(settings);
        return 0;
    }
}

```

```

    }
    else if (fgets(file_path, FILE_NAME_LEN, settings) == NULL) {
        strcpy(file_path, DIR_NAME "/" FILE_NAME);
        fclose(settings);
        return 0;
    }
    else fclose(settings);

    // Меняем \n на \0
    if (file_path[strlen(file_path) - 1] == '\n')
        file_path[strlen(file_path) - 1] = '\0';

    if (!correct_name(file_path)) {
        strcpy(file_path, DIR_NAME "/" FILE_NAME);
        return 1; // Возвращаем путь DIR_NAME / FILE_NAME и код "путь не влез"
    }

    // Создаём файл, если его нет
    FILE* table = fopen(file_path, "r");
    if (table == NULL) {
        table = fopen(file_path, "w");
        if (table == NULL) {
            strcpy(file_path, DIR_NAME "/" FILE_NAME);
            return -2;
        }
        fclose(table);
    }
    else fclose(table);
    return 0;
}

// Получить пароль
int get_password(char* password, int buffer_len) {
    /*
    Возвращает:
        -1: У пользователя нет прав даже для папки DIR_NAME
        0: Пароль успешно получен
    */
    _mkdir(DIR_NAME); // Пробуем создать папку на случай, если её нету
    FILE* settings = fopen(DIR_NAME "/" CONF_NAME, "a+");
    if (!settings) return -1;
    rewind(settings);

    // Если файл пустой, то возвращаем пароль root
    if (fgets(password, PASSWORD_LEN, settings) == NULL) {
        strcpy(password, "root");
        fclose(settings);
        return 0;
    }
    else fclose(settings);

    // Меняем \n на \0
    if (password[strlen(password) - 1] == '\n')
        password[strlen(password) - 1] = '\0';
    return 0;
}

// Записать одну строчку файла в структуру
int read_line(FILE *table, Flight *flight) {
    int days = 0; // Хранение дней в int, например [1, 2, 4] -> 1240000
    int res = fscanf(table, "%d;%31[^\n];%31[^\n];%d;%d;%d;%lf",
        &flight->fnum,
        flight->name,

```

```

        flight->dest,
        &days,
        &flight->dep_time,
        &flight->arr_time,
        &flight->price);
    for (int i = 6; i >= 0; i--) { // Переводим int days в массив
        flight->days[i] = days % 10;
        days /= 10;
    }
    return res;
}

// Записать одну структуру таблицы в файл
int write_line(FILE *table, Flight flight) {
    int days = 0; // Хранение дней в int, например [1, 2, 4] -> 1240000
    // Переводим массив в int days
    for (int i = 0; i < 7; i++) days = days * 10 + flight.days[i];
    int res = fprintf(table, "%d;%s;%s;%d;%d;%d;%lf\n",
        flight.fnum,
        flight.name,
        flight.dest,
        days,
        flight.dep_time,
        flight.arr_time,
        flight.price);
    return res;
}

// Посчитать количество подходящих под фильтр строк
int count_lines(FILE *table, Flight_filter filters[], int *len, int *fil_len) {
    *len = 0; // Сколько записей получилось успешно прочитать
    *fil_len = 0; // Сколько записей, подходящих фильтрам, получилось
    успешно прочитать
    /*
    После использования рекомендуется использовать rewind()
    Возвращает:
    0: все строки успешно прочитаны или файл пустой
    1: не все строки были прочитаны и файл не пустой
    -1: ни одна строка не была прочитана, но файл не пустой
    */

    Flight flight_buffer; // Буфер для чтения
    int col_num; // Сколько столбцов успешно прочитано

    while ((col_num = read_line(table, &flight_buffer)) == FIELDS_NUM) {
        *len = *len + 1;
        if (compare_flight(flight_buffer, filters))
            *fil_len = *fil_len + 1; // Только если подходит под фильтр
    }
    if (*len != 0)
        return (col_num != EOF);
    return -(col_num != EOF);
}

// Прочитать фильтры из файла DIR_NAME/filters.csv в массив
int read_filters(Flight_filter filters[FILTERS_NUM]) {
    FILE* filters_table = fopen(DIR_NAME "/filters.csv", "r");
    if (filters_table == NULL) return -2;
    for (int f = 0; f < FILTERS_NUM; f++) {
        int days = 0; // Хранение дней в int, например
        [1, 2, 4] -> 1240000
        int apply = 0, logic = 0; // Похожее для apply и logic: [1, 0, 1] ->
101

```

```

        int res = fscanf(filters_table,
"%d;%31[^\n];%31[^\n];%d;%d;%d;%d;%d;%d;%d;%d",
        &(filters[f].fnum),
        filters[f].name,
        filters[f].dest,
        &days,
        &(filters[f].dep_time),
        &(filters[f].arr_time),
        &(filters[f].price),
        &apply,
        &logic);
        if (res != FIELDS_NUM + 2) {
            fclose(filters_table);
            return res;
        }
        for (int i = 6; i >= 0; i--) { // Переводим int days в массив
            filters[f].days[i] = days % 10;
            days /= 10;
        }
        for (int i = FIELDS_NUM - 1; i >= 0; i--) { // Переводим int apply и
logic в массивы
            filters[f].apply[i] = apply % 10;
            apply /= 10;
            filters[f].logic[i] = logic % 10;
            logic /= 10;
        }
    }
    fclose(filters_table);
    return 0;
}

// Записать фильтры массива в файл DIR_NAME/filters.csv
int write_filters(Flight_filter filters[FILTERS_NUM]) {
    FILE* filters_table = fopen(DIR_NAME "/filters.csv", "w");
    if (filters_table == NULL) return -2;
    for (int f = 0; f < FILTERS_NUM; f++) {
        int days = 0; // Хранение дней в int, например
[1, 2, 4] -> 1240000
        int apply = 0, logic = 0; // Похожее для apply и logic: [1, 0, 1] ->
101
        for (int i = 0; i < 7; i++) days = days * 10 + filters[f].days[i];
        for (int i = 0; i < FIELDS_NUM; i++) {
            apply = apply * 10 + filters[f].apply[i];
            logic = logic * 10 + filters[f].logic[i];
        }
        int res = fprintf(filters_table, "%d;%s;%s;%d;%d;%d;%d;%d;%d;%d\n",
            filters[f].fnum,
            filters[f].name,
            filters[f].dest,
            days,
            filters[f].dep_time,
            filters[f].arr_time,
            filters[f].price,
            apply,
            logic);
    }
    fclose(filters_table);
    return 0;
}

// Перенести данные из String^ в char*
void copy_char_from_string(char* buffer, int len, System::String^ string) {

```

```

        System::IntPtr ptr =
System::Runtime::InteropServices::Marshal::StringToHGlobalAnsi(string);
        char* char_data = (char*)ptr.ToPointer();
        strncpy(buffer, char_data, len);
        buffer[len - 1] = '\0';
        System::Runtime::InteropServices::Marshal::FreeHGlobal(ptr);
    }

// Перевести данные из String^ формата ЧЧ:ММ в int
int convert_string_time(System::String^ string) {
    // Возвращает результат перевода (-1, при ошибке

    int time[5]; // Буфер для хранения времени

    // Перебор двух символов ЧЧ (которые обязаны присутствовать)
    for (int i = 0; i < 2; i++)
        if (string[i] != ' ')
            time[i] = string[i] - '0'; // Делаем переход char->int
        else time[i] = 0; // Стоит пробел

    // Перебор двух символов ММ (которых может не быть)
    for (int i = 3; i < 5; i++)
        if (i < string->Length) // Если ещё есть символы
            if (string[i] != ' ')
                time[i] = string[i] - '0'; // Делаем переход char->int
            else time[i] = 0; // Стоит пробел
        else
            time[i] = 0; // Проставляем нолики

    // Проверяем на корректность формата
    if ((time[0] * 10 + time[1] > 23) || (time[3] > 5)) return -1;
    // Возвращаем время
    return (time[0] * 10 + time[1]) * 60 + time[3] * 10 + time[4];
}

```

П.Б.7 Код AdminCheck.h

```

#pragma once

#include "AdminSite.h"
#include "file_funcs.h"

namespace MarkflightsUI {

    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;

    /// <summary>
    /// Сводка для AdminCheck
    /// </summary>
    public ref class AdminCheck : public System::Windows::Forms::Form
    {
    public:
        AdminCheck(void)
        {
            InitializeComponent();
        }

    protected:

```

```

    /// <summary>
    /// Освободить все используемые ресурсы.
    /// </summary>
    ~AdminCheck()
    {
        if (components)
        {
            delete components;
        }
    }
private: System::Windows::Forms::Label^ enter_label;
private: System::Windows::Forms::TextBox^ password_box;
private: System::Windows::Forms::Button^ conf_button;
private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container^ components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        this->enter_label = (gcnew System::Windows::Forms::Label());
        this->password_box = (gcnew System::Windows::Forms::TextBox());
        this->conf_button = (gcnew System::Windows::Forms::Button());
        this->SuspendLayout();
        //
        // enter_label
        //
        this->enter_label->AutoSize = true;
        this->enter_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->enter_label->Location = System::Drawing::Point(12, 9);
        this->enter_label->Name = L"enter_label";
        this->enter_label->Size = System::Drawing::Size(154, 20);
        this->enter_label->TabIndex = 0;
        this->enter_label->Text = L"Введите пароль:";
        //
        // password_box
        //
        this->password_box->Location = System::Drawing::Point(16, 32);
        this->password_box->Name = L"password_box";
        this->password_box->Size = System::Drawing::Size(150, 20);
        this->password_box->TabIndex = 1;
        //
        // conf_button
        //
        this->conf_button->Location = System::Drawing::Point(16, 58);
        this->conf_button->Name = L"conf_button";
        this->conf_button->Size = System::Drawing::Size(150, 23);
        this->conf_button->TabIndex = 2;
        this->conf_button->Text = L"Войти";
        this->conf_button->UseVisualStyleBackColor = true;
        this->conf_button->Click += gcnew System::EventHandler(this,
&AdminCheck::conf_button_Click);
        //
        // AdminCheck

```

```

        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(185, 92);
        this->Controls->Add(this->conf_button);
        this->Controls->Add(this->password_box);
        this->Controls->Add(this->enter_label);
        this->MaximizeBox = false;
        this->MinimizeBox = false;
        this->Name = L"AdminCheck";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"Подтверждение";
        this->ResumeLayout(false);
        this->PerformLayout();

    }
#pragma endregion
    private: System::Void conf_button_Click(System::Object^ sender,
System::EventArgs^ e) {
        char password[PASSWORD_LEN];
        get_password(password, PASSWORD_LEN);
        if (this->password_box->Text != gcnew String(password)) {
            MessageBox::Show(L"Неверный пароль");
            return;
        }
        AdminSite^ p = gcnew AdminSite();
        p->Show();
        this->Close();
        for each (Form ^ form in Application::OpenForms) {
            if (form->GetType()->Name == "FlightMenu") {
                form->Hide();
                break;
            }
        }
    }
};
}

```

П.Б.8 Код AdminSite.h

```

#pragma once

#include <stdio.h>

#include "SetFilters.h"
#include "AddFlight.h"
#include "EditFlights.h"
#include "DeleteFlights.h"
#include "ChangePath.h"

#include "structure.h"
#include "file_funcs.h"

namespace MarkflightsUI {
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;

```



```

using namespace System::Drawing;

/// <summary>
/// Сводка для AdminSite
/// </summary>
public ref class AdminSite : public System::Windows::Forms::Form
{
public:
    AdminSite(void)
    {
        InitializeComponent();
        SetupDataBase();
        ReadDataBase();
    }

protected:
    /// <summary>
    /// Освободить все используемые ресурсы.
    /// </summary>
    ~AdminSite()
    {
        if (components)
        {
            delete components;
        }
        // Открываем главную форму
        for each (Form ^ form in Application::OpenForms) {
            if (form->GetType()->Name == "FlightMenu") {
                form->Show();
                break;
            }
        }
    }

private: System::Windows::Forms::DataGridView^ data_base;
private: System::Windows::Forms::Label^ mode_label;
private: System::Windows::Forms::Button^ set_filters_button;
private: System::Windows::Forms::Button^ add_flight_button;
private: System::Windows::Forms::Button^ change_flights_button;
private: System::Windows::Forms::Button^ delete_flights_button;
private: System::Windows::Forms::Button^ change_path_button;
private: System::Windows::Forms::Button^ exit_button;
private: System::Windows::Forms::Button^ update_button;

private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container^ components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        this->data_base = (gcnew
System::Windows::Forms::DataGridView());
        this->mode_label = (gcnew System::Windows::Forms::Label());
        this->set_filters_button = (gcnew
System::Windows::Forms::Button());
        this->add_flight_button = (gcnew
System::Windows::Forms::Button());
    }

```

```

        this->change_flights_button = (gcnew
System::Windows::Forms::Button());
        this->delete_flights_button = (gcnew
System::Windows::Forms::Button());
        this->change_path_button = (gcnew
System::Windows::Forms::Button());
        this->exit_button = (gcnew System::Windows::Forms::Button());
        this->update_button = (gcnew System::Windows::Forms::Button());

        (cli::safe_cast<System::ComponentModel::ISupportInitialize^(this-
>data_base))->BeginInit();
        this->SuspendLayout();
        //
        // data_base
        //
        this->data_base->AllowUserToAddRows = false;
        this->data_base->AllowUserToDeleteRows = false;
        this->data_base->AllowUserToResizeColumns = false;
        this->data_base->AllowUserToResizeRows = false;
        this->data_base->BackColor =
System::Drawing::SystemColors::ButtonHighlight;
        this->data_base->ColumnHeadersHeightSizeMode =
System::Windows::Forms::DataGridViewColumnHeadersHeightSizeMode::AutoSize;
        this->data_base->Location = System::Drawing::Point(12, 12);
        this->data_base->Name = L"data_base";
        this->data_base->ReadOnly = true;
        this->data_base->RowHeadersVisible = false;
        this->data_base->Size = System::Drawing::Size(803, 500);
        this->data_base->TabIndex = 0;
        //
        // mode_label
        //
        this->mode_label->AutoSize = true;
        this->mode_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->mode_label->Location = System::Drawing::Point(830, 9);
        this->mode_label->Name = L"mode_label";
        this->mode_label->Size = System::Drawing::Size(212, 20);
        this->mode_label->TabIndex = 1;
        this->mode_label->Text = L"Режим администратора";
        //
        // set_filters_button
        //
        this->set_filters_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->set_filters_button->Location = System::Drawing::Point(844,
81);
        this->set_filters_button->Name = L"set_filters_button";
        this->set_filters_button->Size = System::Drawing::Size(186, 33);
        this->set_filters_button->TabIndex = 2;
        this->set_filters_button->Text = L"Настроить фильтры";
        this->set_filters_button->UseVisualStyleBackColor = true;
        this->set_filters_button->Click += gcnew
System::EventHandler(this, &AdminSite::set_filters_button_Click);
        //
        // add_flight_button
        //

```

```

        this->add_flight_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->add_flight_button->Location = System::Drawing::Point(844,
120);

        this->add_flight_button->Name = L"add_flight_button";
        this->add_flight_button->Size = System::Drawing::Size(186, 33);
        this->add_flight_button->TabIndex = 3;
        this->add_flight_button->Text = L"Добавить рейс";
        this->add_flight_button->UseVisualStyleBackColor = true;
        this->add_flight_button->Click += gcnew
System::EventHandler(this, &AdminSite::add_flight_button_Click);
        //
        // change_flights_button
        //
        this->change_flights_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular,
        System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
        this->change_flights_button->Location =
System::Drawing::Point(844, 159);
        this->change_flights_button->Name = L"change_flights_button";
        this->change_flights_button->Size = System::Drawing::Size(186,
33);

        this->change_flights_button->TabIndex = 4;
        this->change_flights_button->Text = L"Изменение рейсов";
        this->change_flights_button->UseVisualStyleBackColor = true;
        this->change_flights_button->Click += gcnew
System::EventHandler(this, &AdminSite::change_flights_button_Click);
        //
        // delete_flights_button
        //
        this->delete_flights_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular,
        System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
        this->delete_flights_button->Location =
System::Drawing::Point(844, 198);
        this->delete_flights_button->Name = L"delete_flights_button";
        this->delete_flights_button->Size = System::Drawing::Size(186,
33);

        this->delete_flights_button->TabIndex = 5;
        this->delete_flights_button->Text = L"Удаление рейсов";
        this->delete_flights_button->UseVisualStyleBackColor = true;
        this->delete_flights_button->Click += gcnew
System::EventHandler(this, &AdminSite::delete_flights_button_Click);
        //
        // change_path_button
        //
        this->change_path_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->change_path_button->Location = System::Drawing::Point(844,
237);

        this->change_path_button->Name = L"change_path_button";
        this->change_path_button->Size = System::Drawing::Size(186, 33);
        this->change_path_button->TabIndex = 6;
        this->change_path_button->Text = L"Изменить путь до файла";
        this->change_path_button->UseVisualStyleBackColor = true;

```

```

        this->change_path_button->Click += gcnw
System::EventHandler(this, &AdminSite::change_path_button_Click);
        //
        // exit_button
        //
        this->exit_button->BackColor =
System::Drawing::Color::FromArgb(static_cast<System::Int32>(static_cast<System::Byte>
>(255)), static_cast<System::Int32>(static_cast<System::Byte>(192)),

        static_cast<System::Int32>(static_cast<System::Byte>(192)));
        this->exit_button->Font = (gcnw
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->exit_button->Location = System::Drawing::Point(844, 276);
        this->exit_button->Name = L"exit_button";
        this->exit_button->Size = System::Drawing::Size(186, 33);
        this->exit_button->TabIndex = 7;
        this->exit_button->Text = L"Выйти в главное меню";
        this->exit_button->UseVisualStyleBackColor = false;
        this->exit_button->Click += gcnw System::EventHandler(this,
&AdminSite::exit_button_Click);
        //
        // update_button
        //
        this->update_button->Font = (gcnw
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->update_button->Location = System::Drawing::Point(844, 42);
        this->update_button->Name = L"update_button";
        this->update_button->Size = System::Drawing::Size(186, 33);
        this->update_button->TabIndex = 8;
        this->update_button->Text = L"Обновить таблицу";
        this->update_button->UseVisualStyleBackColor = true;
        this->update_button->Click += gcnw System::EventHandler(this,
&AdminSite::update_button_Click);
        //
        // AdminSite
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(1056, 524);
        this->Controls->Add(this->update_button);
        this->Controls->Add(this->exit_button);
        this->Controls->Add(this->change_path_button);
        this->Controls->Add(this->delete_flights_button);
        this->Controls->Add(this->change_flights_button);
        this->Controls->Add(this->add_flight_button);
        this->Controls->Add(this->set_filters_button);
        this->Controls->Add(this->mode_label);
        this->Controls->Add(this->data_base);
        this->MaximizeBox = false;
        this->Name = L"AdminSite";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"Режим администратора";

        (cli::safe_cast<System::ComponentModel::ISupportInitialize>(this-
>data_base))->EndInit();

```

```

        this->ResumeLayout(false);
        this->PerformLayout();
    }
#pragma endregion
private: Void SetupDataBase() {
    this->data_base->Columns->Add("id", L"Номер\пзаписи");
    this->data_base->Columns->Add("fnum", L"Номер\прейса");
    this->data_base->Columns->Add("name", L"Тип\псамолёта");
    this->data_base->Columns->Add("dest", L"Пункт\пназначения");
    this->data_base->Columns->Add("days", L"Дни\потправления");
    this->data_base->Columns->Add("dep_time", L"Время\пвылета");
    this->data_base->Columns->Add("arr_time", L"Время\пприлёта");
    this->data_base->Columns->Add("price", L"Цена\пбилета");
}
private: Void ReadDataBase() {
    // Очищаем БД
    this->data_base->Rows->Clear();

    char file_path[FILE_NAME_LEN]; // Название файла БД
    if (get_file(file_path) == -1) {
        MessageBox::Show(L"Не удалось открыть файл с информацией для
чтения");
        return;
    }

    // Пробуем открыть, либо создаём
    FILE* table = fopen(file_path, "r");
    if (table == NULL) {
        table = fopen(file_path, "w");
        if (table == NULL) {
            MessageBox::Show(L"Не удалось открыть файл с БД");
            return;
        }
        fclose(table);
        table = fopen(file_path, "r");
    }

    // Читаем фильтры в массив
    Flight_filter filters[FILTERS_NUM];
    int fil_err;
    if (fil_err = read_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл с фильтрами для
чтения - ошибка " + fil_err);
        fclose(table);
        return;
    }

    // Считываем файл с БД
    Flight flight_buffer; // Буфер для чтения
    int col_num; // Сколько столбцов успешно прочитано
    int id = 0; // Номер записи
    while ((col_num = read_line(table, &flight_buffer)) == FIELDS_NUM) {
        id++;
        if (!compare_flight(flight_buffer, filters)) continue;
        String^ str_dep_time = (flight_buffer.dep_time % 60).ToString();
        // Получаем минуты dep_time
        if (str_dep_time->Length < 2) str_dep_time = "0" + str_dep_time;
        // Дополняем до формата "ММ"
        str_dep_time = (flight_buffer.dep_time / 60).ToString() + ":" +
str_dep_time; // Получаем часы
    }
}

```

```

        String^ str_arr_time = (flight_buffer.arr_time % 60).ToString();
        // Получаем минуты arr_time
        if (str_arr_time->Length < 2) str_arr_time = "0" + str_arr_time;
        // Дополняем до формата "MM"
        str_arr_time = (flight_buffer.arr_time / 60).ToString() + ":" +
str_arr_time;
        // Получаем дни отправления
        String^ str_days = "";
        if (flight_buffer.days[0] != 0)
            str_days += flight_buffer.days[0].ToString();
        for (int i = 1; i < 7 && flight_buffer.days[i]; i++)
            str_days += ", " + flight_buffer.days[i].ToString();

        this->data_base->Rows->Add(
            id,
            flight_buffer.fnum,
            gcnew String(flight_buffer.name),
            gcnew String(flight_buffer.dest),
            str_days,
            str_dep_time,
            str_arr_time,
            flight_buffer.price

        );
    }
    fclose(table);
}
public: System::Void set_filters_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    SetFilters^ p = gcnew SetFilters();
    p->ShowDialog();
    ReadDataBase();
}
private: System::Void add_flight_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    AddFlight^ p = gcnew AddFlight();
    p->ShowDialog();
    ReadDataBase();
}
private: System::Void change_flights_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    EditFlights^ p = gcnew EditFlights();
    p->ShowDialog();
    ReadDataBase();
}
private: System::Void delete_flights_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    DeleteFlights^ p = gcnew DeleteFlights();
    p->ShowDialog();
    ReadDataBase();
}
private: System::Void change_path_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    char file_path[FILE_NAME_LEN];
    if (get_file(file_path) == -1) {
        MessageBox::Show(L"Недостаточно прав");
        return;
    }
    ChangePath^ p = gcnew ChangePath();
    p->ShowDialog(this);
    ReadDataBase();
}
}

```

```

        public: System::Void exit_button_Click(System::Object^ sender,
System::EventArgs^ e) {
            for each (Form ^ form in Application::OpenForms) {
                if (form->GetType()->Name == "FlightMenu") {
                    form->Show();
                    break;
                }
            }
            this->Close();
        }
        public: System::Void update_button_Click(System::Object^ sender,
System::EventArgs^ e) {
            ReadDataBase();
        }
    };
}

```

П.Б.9 Код SetFilters.h

```

#pragma once

#include "file_funcs.h"
#include "structure.h"

#include "EditFilterField.h"

#define NOT_IN_USE "не используется"

namespace MarkflightsUI {
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;

    /// <summary>
    /// Сводка для SetFilters
    /// </summary>
    public ref class SetFilters : public System::Windows::Forms::Form
    {
        array<String^>^ field_names;    // Названия полей
        array<String^>^ comp_ops;       // Операторы сравнения
        int cur_field;                  // Какое поле нужно
    };

    public:
        SetFilters(void)
        {
            InitializeComponent();
            // Скрываем часть для изменения конкретного фильтра
            this->ClientSize = System::Drawing::Size(440-16, 296);

            this->field_names = gcnew array<String^>{ // Заполняем названия
                L"номер рейса",
                L"тип самолёта",
                L"пункт назначения",
                L"дни отправления",
                L"время вылета",
                L"время прилёта",
                L"цена билета"
            };
        }
    };
}

```

```

        this->comp_ops = gcnew array<String^>{ // Заполняем операторы
сравнения
            L">",
            L"<",
            L"=",
            L">=",
            L"<="
        };
        ReadFilters();
        this->cur_field = -1;
    }

protected:
    /// <summary>
    /// Освободить все используемые ресурсы.
    /// </summary>
    ~SetFilters()
    {
        if (components)
        {
            delete components;
        }
    }

private: System::Windows::Forms::Label^ setting_label;
private: System::Windows::Forms::Label^ fnum_use_label;
private: System::Windows::Forms::Label^ name_use_label;
private: System::Windows::Forms::Label^ dest_use_label;
private: System::Windows::Forms::Label^ days_use_label;
private: System::Windows::Forms::Label^ dep_time_use_label;
private: System::Windows::Forms::Label^ arr_time_use_label;
private: System::Windows::Forms::Label^ price_use_label;
private: System::Windows::Forms::Button^ fnum_button;
private: System::Windows::Forms::Button^ name_button;
private: System::Windows::Forms::Button^ dest_button;
private: System::Windows::Forms::Button^ days_button;
private: System::Windows::Forms::Button^ dep_time_button;
private: System::Windows::Forms::Button^ arr_time_button;
private: System::Windows::Forms::Button^ price_button;
private: System::Windows::Forms::Label^ hint_label;
private: System::Windows::Forms::Button^ update_button;
private: System::Windows::Forms::Label^ field_label;
private: System::Windows::Forms::Label^ field_filter_1_label;
private: System::Windows::Forms::Label^ num_1_label;
private: System::Windows::Forms::Label^ num_2_label;
private: System::Windows::Forms::Label^ field_filter_2_label;
private: System::Windows::Forms::Button^ field_filter_1_delete_button;
private: System::Windows::Forms::Button^ field_filter_1_edit_button;
private: System::Windows::Forms::Button^ field_filter_2_delete_button;
private: System::Windows::Forms::Button^ field_filter_2_edit_button;
private: System::Windows::Forms::Button^ add_new_filter_button;
private: System::Windows::Forms::Button^ delete_all_filters_button;
private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container ^components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)

```



```

        {
            this->setting_label = (gcnew System::Windows::Forms::Label());
            this->fnum_use_label = (gcnew System::Windows::Forms::Label());
            this->name_use_label = (gcnew System::Windows::Forms::Label());
            this->dest_use_label = (gcnew System::Windows::Forms::Label());
            this->days_use_label = (gcnew System::Windows::Forms::Label());
            this->dep_time_use_label = (gcnew
System::Windows::Forms::Label());
            this->arr_time_use_label = (gcnew
System::Windows::Forms::Label());
            this->price_use_label = (gcnew System::Windows::Forms::Label());
            this->fnum_button = (gcnew System::Windows::Forms::Button());
            this->name_button = (gcnew System::Windows::Forms::Button());
            this->dest_button = (gcnew System::Windows::Forms::Button());
            this->days_button = (gcnew System::Windows::Forms::Button());
            this->dep_time_button = (gcnew
System::Windows::Forms::Button());
            this->arr_time_button = (gcnew
System::Windows::Forms::Button());
            this->price_button = (gcnew System::Windows::Forms::Button());
            this->hint_label = (gcnew System::Windows::Forms::Label());
            this->update_button = (gcnew System::Windows::Forms::Button());
            this->field_label = (gcnew System::Windows::Forms::Label());
            this->field_filter_1_label = (gcnew
System::Windows::Forms::Label());
            this->num_1_label = (gcnew System::Windows::Forms::Label());
            this->num_2_label = (gcnew System::Windows::Forms::Label());
            this->field_filter_2_label = (gcnew
System::Windows::Forms::Label());
            this->field_filter_1_delete_button = (gcnew
System::Windows::Forms::Button());
            this->field_filter_1_edit_button = (gcnew
System::Windows::Forms::Button());
            this->field_filter_2_delete_button = (gcnew
System::Windows::Forms::Button());
            this->field_filter_2_edit_button = (gcnew
System::Windows::Forms::Button());
            this->add_new_filter_button = (gcnew
System::Windows::Forms::Button());
            this->delete_all_filters_button = (gcnew
System::Windows::Forms::Button());
            this->SuspendLayout();
            //
            // setting_label
            //
            this->setting_label->AutoSize = true;
            this->setting_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
                static_cast<System::Byte>(204)));
            this->setting_label->Location = System::Drawing::Point(139, 9);
            this->setting_label->Name = L"setting_label";
            this->setting_label->Size = System::Drawing::Size(192, 20);
            this->setting_label->TabIndex = 1;
            this->setting_label->Text = L"Настройка фильтров";
            //
            // fnum_use_label
            //
            this->fnum_use_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
                static_cast<System::Byte>(204)));

```

```

        this->fnum_use_label->Location = System::Drawing::Point(178,
62);
        this->fnum_use_label->Name = L"fnum_use_label";
        this->fnum_use_label->Size = System::Drawing::Size(232, 17);
        this->fnum_use_label->TabIndex = 34;
        this->fnum_use_label->Text = L"не используется";
        //
        // name_use_label
        //
        this->name_use_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->name_use_label->Location = System::Drawing::Point(178,
91);
        this->name_use_label->Name = L"name_use_label";
        this->name_use_label->Size = System::Drawing::Size(232, 17);
        this->name_use_label->TabIndex = 35;
        this->name_use_label->Text = L"не используется";
        //
        // dest_use_label
        //
        this->dest_use_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->dest_use_label->Location = System::Drawing::Point(177,
120);
        this->dest_use_label->Name = L"dest_use_label";
        this->dest_use_label->Size = System::Drawing::Size(233, 17);
        this->dest_use_label->TabIndex = 36;
        this->dest_use_label->Text = L"не используется";
        //
        // days_use_label
        //
        this->days_use_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->days_use_label->Location = System::Drawing::Point(177,
149);
        this->days_use_label->Name = L"days_use_label";
        this->days_use_label->Size = System::Drawing::Size(233, 17);
        this->days_use_label->TabIndex = 37;
        this->days_use_label->Text = L"не используется";
        //
        // dep_time_use_label
        //
        this->dep_time_use_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->dep_time_use_label->Location = System::Drawing::Point(177,
178);
        this->dep_time_use_label->Name = L"dep_time_use_label";
        this->dep_time_use_label->Size = System::Drawing::Size(233, 17);
        this->dep_time_use_label->TabIndex = 38;
        this->dep_time_use_label->Text = L"не используется";
        //
        // arr_time_use_label
        //

```

```

        this->arr_time_use_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->arr_time_use_label->Location = System::Drawing::Point(177,
207);

        this->arr_time_use_label->Name = L"arr_time_use_label";
        this->arr_time_use_label->Size = System::Drawing::Size(233, 17);
        this->arr_time_use_label->TabIndex = 39;
        this->arr_time_use_label->Text = L"не используется";
        //
        // price_use_label
        //
        this->price_use_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->price_use_label->Location = System::Drawing::Point(177,
236);

        this->price_use_label->Name = L"price_use_label";
        this->price_use_label->Size = System::Drawing::Size(233, 17);
        this->price_use_label->TabIndex = 40;
        this->price_use_label->Text = L"не используется";
        //
        // fnum_button
        //
        this->fnum_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->fnum_button->Location = System::Drawing::Point(12, 59);
        this->fnum_button->Name = L"fnum_button";
        this->fnum_button->Size = System::Drawing::Size(159, 23);
        this->fnum_button->TabIndex = 41;
        this->fnum_button->Text = L"Номер пейса";
        this->fnum_button->UseVisualStyleBackColor = true;
        this->fnum_button->Click += gcnew System::EventHandler(this,
&SetFilters::fnum_button_Click);
        //
        // name_button
        //
        this->name_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->name_button->Location = System::Drawing::Point(12, 88);
        this->name_button->Name = L"name_button";
        this->name_button->Size = System::Drawing::Size(159, 23);
        this->name_button->TabIndex = 42;
        this->name_button->Text = L"Тип самолёта";
        this->name_button->UseVisualStyleBackColor = true;
        this->name_button->Click += gcnew System::EventHandler(this,
&SetFilters::name_button_Click);
        //
        // dest_button
        //
        this->dest_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->dest_button->Location = System::Drawing::Point(12, 117);
        this->dest_button->Name = L"dest_button";
        this->dest_button->Size = System::Drawing::Size(159, 23);

```

```

        this->dest_button->TabIndex = 43;
        this->dest_button->Text = L"Пункт назначения";
        this->dest_button->UseVisualStyleBackColor = true;
        this->dest_button->Click += gcnew System::EventHandler(this,
&SetFilters::dest_button_Click);
        //
        // days_button
        //
        this->days_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->days_button->Location = System::Drawing::Point(12, 146);
        this->days_button->Name = L"days_button";
        this->days_button->Size = System::Drawing::Size(159, 23);
        this->days_button->TabIndex = 44;
        this->days_button->Text = L"Дни отправления";
        this->days_button->UseVisualStyleBackColor = true;
        this->days_button->Click += gcnew System::EventHandler(this,
&SetFilters::days_button_Click);
        //
        // dep_time_button
        //
        this->dep_time_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->dep_time_button->Location = System::Drawing::Point(12,
175);
        this->dep_time_button->Name = L"dep_time_button";
        this->dep_time_button->Size = System::Drawing::Size(159, 23);
        this->dep_time_button->TabIndex = 45;
        this->dep_time_button->Text = L"Время вылета";
        this->dep_time_button->UseVisualStyleBackColor = true;
        this->dep_time_button->Click += gcnew System::EventHandler(this,
&SetFilters::dep_time_button_Click);
        //
        // arr_time_button
        //
        this->arr_time_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->arr_time_button->Location = System::Drawing::Point(12,
204);
        this->arr_time_button->Name = L"arr_time_button";
        this->arr_time_button->Size = System::Drawing::Size(159, 23);
        this->arr_time_button->TabIndex = 46;
        this->arr_time_button->Text = L"Время прилёта";
        this->arr_time_button->UseVisualStyleBackColor = true;
        this->arr_time_button->Click += gcnew System::EventHandler(this,
&SetFilters::arr_time_button_Click);
        //
        // price_button
        //
        this->price_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->price_button->Location = System::Drawing::Point(12, 233);
        this->price_button->Name = L"price_button";
        this->price_button->Size = System::Drawing::Size(159, 23);
        this->price_button->TabIndex = 47;

```

```

        this->price_button->Text = L"Цена билета";
        this->price_button->UseVisualStyleBackColor = true;
        this->price_button->Click += gcnew System::EventHandler(this,
&SetFilters::price_button_Click);
        //
        // hint_label
        //
        this->hint_label->AutoSize = true;
        this->hint_label->Location = System::Drawing::Point(67, 29);
        this->hint_label->Name = L"hint_label";
        this->hint_label->Size = System::Drawing::Size(322, 13);
        this->hint_label->TabIndex = 48;
        this->hint_label->Text = L"Нажмите на название поля, чтобы
редактировать его фильтр";
        //
        // update_button
        //
        this->update_button->Location = System::Drawing::Point(155,
262);
        this->update_button->Name = L"update_button";
        this->update_button->Size = System::Drawing::Size(165, 23);
        this->update_button->TabIndex = 49;
        this->update_button->Text = L"Обновить список";
        this->update_button->UseVisualStyleBackColor = true;
        this->update_button->Click += gcnew System::EventHandler(this,
&SetFilters::update_button_Click);
        //
        // field_label
        //
        this->field_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->field_label->Location = System::Drawing::Point(427, 9);
        this->field_label->Name = L"field_label";
        this->field_label->Size = System::Drawing::Size(345, 17);
        this->field_label->TabIndex = 50;
        this->field_label->Text = L"Поле \"номер рейса\"";
        this->field_label->TextAlign =
System::Drawing::ContentAlignment::MiddleCenter;
        //
        // field_filter_1_label
        //
        this->field_filter_1_label->AutoSize = true;
        this->field_filter_1_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular,
        System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->field_filter_1_label->Location =
System::Drawing::Point(456, 35);
        this->field_filter_1_label->Name = L"field_filter_1_label";
        this->field_filter_1_label->Size = System::Drawing::Size(118,
17);
        this->field_filter_1_label->TabIndex = 51;
        this->field_filter_1_label->Text = L"не используется";
        //
        // num_1_label
        //
        this->num_1_label->AutoSize = true;
        this->num_1_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,

```

```

        static_cast<System::Byte>(204)));
this->num_1_label->Location = System::Drawing::Point(427, 35);
this->num_1_label->Name = L"num_1_label";
this->num_1_label->Size = System::Drawing::Size(30, 17);
this->num_1_label->TabIndex = 52;
this->num_1_label->Text = L"1) -";
//
// num_2_label
//
this->num_2_label->AutoSize = true;
this->num_2_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
this->num_2_label->Location = System::Drawing::Point(427, 59);
this->num_2_label->Name = L"num_2_label";
this->num_2_label->Size = System::Drawing::Size(30, 17);
this->num_2_label->TabIndex = 54;
this->num_2_label->Text = L"2) -";
//
// field_filter_2_label
//
this->field_filter_2_label->AutoSize = true;
this->field_filter_2_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular,
        System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->field_filter_2_label->Location =
System::Drawing::Point(456, 59);
this->field_filter_2_label->Name = L"field_filter_2_label";
this->field_filter_2_label->Size = System::Drawing::Size(118,
17);
this->field_filter_2_label->TabIndex = 53;
this->field_filter_2_label->Text = L"не используется";
//
// field_filter_1_delete_button
//
this->field_filter_1_delete_button->Location =
System::Drawing::Point(580, 32);
this->field_filter_1_delete_button->Name =
L"field_filter_1_delete_button";
this->field_filter_1_delete_button->Size =
System::Drawing::Size(75, 23);
this->field_filter_1_delete_button->TabIndex = 55;
this->field_filter_1_delete_button->Text = L"Удалить";
this->field_filter_1_delete_button->UseVisualStyleBackColor =
true;
this->field_filter_1_delete_button->Click += gcnew
System::EventHandler(this, &SetFilters::field_filter_1_delete_button_Click);
//
// field_filter_1_edit_button
//
this->field_filter_1_edit_button->Location =
System::Drawing::Point(661, 32);
this->field_filter_1_edit_button->Name =
L"field_filter_1_edit_button";
this->field_filter_1_edit_button->Size =
System::Drawing::Size(102, 23);
this->field_filter_1_edit_button->TabIndex = 56;
this->field_filter_1_edit_button->Text = L"Редактировать";
this->field_filter_1_edit_button->UseVisualStyleBackColor =
true;

```

```

        this->field_filter_1_edit_button->Click += gcnw
System::EventHandler(this, &SetFilters::field_filter_1_edit_button_Click);
        //
        // field_filter_2_delete_button
        //
        this->field_filter_2_delete_button->Location =
System::Drawing::Point(580, 56);
        this->field_filter_2_delete_button->Name =
L"field_filter_2_delete_button";
        this->field_filter_2_delete_button->Size =
System::Drawing::Size(75, 23);
        this->field_filter_2_delete_button->TabIndex = 57;
        this->field_filter_2_delete_button->Text = L"Удалить";
        this->field_filter_2_delete_button->UseVisualStyleBackColor =
true;
        this->field_filter_2_delete_button->Click += gcnw
System::EventHandler(this, &SetFilters::field_filter_2_delete_button_Click);
        //
        // field_filter_2_edit_button
        //
        this->field_filter_2_edit_button->Location =
System::Drawing::Point(661, 56);
        this->field_filter_2_edit_button->Name =
L"field_filter_2_edit_button";
        this->field_filter_2_edit_button->Size =
System::Drawing::Size(102, 23);
        this->field_filter_2_edit_button->TabIndex = 58;
        this->field_filter_2_edit_button->Text = L"Редактировать";
        this->field_filter_2_edit_button->UseVisualStyleBackColor =
true;
        this->field_filter_2_edit_button->Click += gcnw
System::EventHandler(this, &SetFilters::field_filter_2_edit_button_Click);
        //
        // add_new_filter_button
        //
        this->add_new_filter_button->Location =
System::Drawing::Point(438, 91);
        this->add_new_filter_button->Name = L"add_new_filter_button";
        this->add_new_filter_button->Size = System::Drawing::Size(149,
32);
        this->add_new_filter_button->TabIndex = 59;
        this->add_new_filter_button->Text = L"Добавить новый фильтр";
        this->add_new_filter_button->UseVisualStyleBackColor = true;
        this->add_new_filter_button->Click += gcnw
System::EventHandler(this, &SetFilters::add_new_filter_button_Click);
        //
        // delete_all_filters_button
        //
        this->delete_all_filters_button->Location =
System::Drawing::Point(607, 91);
        this->delete_all_filters_button->Name =
L"delete_all_filters_button";
        this->delete_all_filters_button->Size =
System::Drawing::Size(151, 32);
        this->delete_all_filters_button->TabIndex = 60;
        this->delete_all_filters_button->Text = L"Удалить все фильтры";
        this->delete_all_filters_button->UseVisualStyleBackColor = true;
        this->delete_all_filters_button->Click += gcnw
System::EventHandler(this, &SetFilters::delete_all_filters_button_Click);
        //
        // SetFilters
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);

```

```

        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(784, 296);
        this->Controls->Add(this->delete_all_filters_button);
        this->Controls->Add(this->add_new_filter_button);
        this->Controls->Add(this->field_filter_2_edit_button);
        this->Controls->Add(this->field_filter_2_delete_button);
        this->Controls->Add(this->field_filter_1_edit_button);
        this->Controls->Add(this->field_filter_1_delete_button);
        this->Controls->Add(this->num_2_label);
        this->Controls->Add(this->field_filter_2_label);
        this->Controls->Add(this->num_1_label);
        this->Controls->Add(this->field_filter_1_label);
        this->Controls->Add(this->field_label);
        this->Controls->Add(this->update_button);
        this->Controls->Add(this->hint_label);
        this->Controls->Add(this->price_button);
        this->Controls->Add(this->arr_time_button);
        this->Controls->Add(this->dep_time_button);
        this->Controls->Add(this->days_button);
        this->Controls->Add(this->dest_button);
        this->Controls->Add(this->name_button);
        this->Controls->Add(this->fnum_button);
        this->Controls->Add(this->price_use_label);
        this->Controls->Add(this->arr_time_use_label);
        this->Controls->Add(this->dep_time_use_label);
        this->Controls->Add(this->days_use_label);
        this->Controls->Add(this->dest_use_label);
        this->Controls->Add(this->name_use_label);
        this->Controls->Add(this->fnum_use_label);
        this->Controls->Add(this->setting_label);
        this->MaximizeBox = false;
        this->Name = L"SetFilters";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"Настройка фильтров";
        this->ResumeLayout(false);
        this->PerformLayout();

    }
#pragma endregion
private: System::Void ReadFilters() {
    Flight_filter filters[FILTERS_NUM];
    // Обнуляем поля
    this->fnum_use_label->Text = "";
    this->name_use_label->Text = "";
    this->dest_use_label->Text = "";
    this->days_use_label->Text = "";
    this->dep_time_use_label->Text = "";
    this->arr_time_use_label->Text = "";
    this->price_use_label->Text = "";
    if (read_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл с фильтрами для чтения");
        return;
    }
    for (int f = 0; f < FILTERS_NUM; f++) {
        // Поле fnum
        if (filters[f].apply[0]) {
            if (this->fnum_use_label->Text != "")
                if (filters[f].logic[0] == 0) this->fnum_use_label->Text
+= " && ";

```



```

        else this->fnum_use_label->Text += " || ";
        this->fnum_use_label->Text += filters[f].fnum;
    }
    // Поне name
    if (filters[f].apply[1]) {
        if (this->name_use_label->Text != "")
            if (filters[f].logic[1] == 0) this->name_use_label->Text
+= " && ";
            else this->name_use_label->Text += " || ";
            this->name_use_label->Text += gcnew String(filters[f].name);
        }
    // Поне dest
    if (filters[f].apply[2]) {
        if (this->dest_use_label->Text != "")
            if (filters[f].logic[2] == 0) this->dest_use_label->Text
+= " && ";
            else this->dest_use_label->Text += " || ";
            this->dest_use_label->Text += gcnew String(filters[f].dest);
        }
    // Поне days
    if (filters[f].apply[3]) {
        if (this->days_use_label->Text != "")
            if (filters[f].logic[3] == 0) this->days_use_label->Text
+= " && ";
            else this->days_use_label->Text += " || ";
            this->days_use_label->Text += filters[f].days[0];
            for (int i = 1; i < 7 && filters[f].days[i]; i++)
                this->days_use_label->Text += ", " + filters[f].days[i];
        }
    // Поне dep_time
    if (filters[f].apply[4]) {
        if (this->dep_time_use_label->Text != "")
            if (filters[f].logic[4] == 0) this->dep_time_use_label-
>Text += " && ";
            else this->dep_time_use_label->Text += " || ";
            this->dep_time_use_label->Text += this-
>comp_ops[filters[f].apply[4] - 1];
            this->dep_time_use_label->Text += filters[f].dep_time / 60 +
";";
            this->dep_time_use_label->Text += filters[f].dep_time % 60 / 10;
            this->dep_time_use_label->Text += filters[f].dep_time % 60 % 10;
        }
    // Поне arr_time
    if (filters[f].apply[5]) {
        if (this->arr_time_use_label->Text != "")
            if (filters[f].logic[5] == 0) this->arr_time_use_label-
>Text += " && ";
            else this->arr_time_use_label->Text += " || ";
            this->arr_time_use_label->Text += this-
>comp_ops[filters[f].apply[5] - 1];
            this->arr_time_use_label->Text += filters[f].arr_time / 60 +
";";
            this->arr_time_use_label->Text += filters[f].arr_time % 60 / 10;
            this->arr_time_use_label->Text += filters[f].arr_time % 60 % 10;
        }
    // Поне price
    if (filters[f].apply[6]) {
        if (this->price_use_label->Text != "")
            if (filters[f].logic[6] == 0) this->price_use_label->Text
+= " && ";
            else this->price_use_label->Text += " || ";
            this->price_use_label->Text += filters[f].price;
        }
    }

```

```

    }
    // Если поля пустые, то ставим NOT_IN_USE
    if (this->fnum_use_label->Text == "") this->fnum_use_label->Text =
NOT_IN_USE;
    if (this->name_use_label->Text == "") this->name_use_label->Text =
NOT_IN_USE;
    if (this->dest_use_label->Text == "") this->dest_use_label->Text =
NOT_IN_USE;
    if (this->days_use_label->Text == "") this->days_use_label->Text =
NOT_IN_USE;
    if (this->dep_time_use_label->Text == "") this->dep_time_use_label->Text =
NOT_IN_USE;
    if (this->arr_time_use_label->Text == "") this->arr_time_use_label->Text =
NOT_IN_USE;
    if (this->price_use_label->Text == "") this->price_use_label->Text =
NOT_IN_USE;
}
private: System::Void filter_field_show() {
    int n = this->cur_field;
    if (n == -1) return;
    // Читаем фильтры в массив
    Flight_filter filters[FILTERS_NUM];
    if (read_filters(filters)) return;
    // Открываем скрытую часть
    this->ClientSize = System::Drawing::Size(784, 296);
    this->field_label->Text = "Поле " + this->field_names[n];
    // Обнуляем поля
    this->field_filter_1_label->Text = "";
    this->field_filter_2_label->Text = "";
    // Открываем кнопки для всех полей
    this->field_filter_1_delete_button->Enabled = 1;
    this->field_filter_1_edit_button->Enabled = 1;
    this->field_filter_2_delete_button->Enabled = 1;
    this->field_filter_2_edit_button->Enabled = 1;
    this->add_new_filter_button->Enabled = 1;
    this->delete_all_filters_button->Enabled = 1;
    // Создаём массив строчек для удобства работы
    array<String^>^ labels = gcnew array<String^>{"", ""};
    int working_filters = 0; // Счётчик рабочих фильтров
    for (int f = 0; f < FILTERS_NUM; f++)
        if (filters[f].apply[n]) {
            working_filters++;
            if (n == 0) labels[f] += filters[f].fnum;
            else if (n == 1) labels[f] += gcnew String(filters[f].name);
            else if (n == 2) labels[f] += gcnew String(filters[f].dest);
            else if (n == 3) {
                labels[f] += filters[f].days[0];
                for (int i = 1; i < 7 && filters[f].days[i]; i++)
                    labels[f] += ", " + filters[f].days[i];
            }
            else if (n == 4) {
                labels[f] += this->comp_ops[filters[f].apply[n] - 1];
                labels[f] += filters[f].dep_time / 60 + ":";
                labels[f] += filters[f].dep_time % 60 / 10;
                labels[f] += filters[f].dep_time % 60 % 10;
            }
            else if (n == 5) {
                labels[f] += this->comp_ops[filters[f].apply[n] - 1];
                labels[f] += filters[f].arr_time / 60 + ":";
                labels[f] += filters[f].arr_time % 60 / 10;
                labels[f] += filters[f].arr_time % 60 % 10;
            }
            else if (n == 6) labels[f] += filters[f].price;
        }
}

```

```

        if (f > 0) // Проставим логические операции
            if (filters[f].logic[n] == 0) labels[f] = "&& " +
labels[f];
                else labels[f] = "|| " + labels[f];
        }
        else {
            labels[f] += NOT_IN_USE;
            // Блокируем кнопки
            if (f == 0) {
                this->field_filter_1_delete_button->Enabled = 0;
                this->field_filter_1_edit_button->Enabled = 0;
            }
            else {
                this->field_filter_2_delete_button->Enabled = 0;
                this->field_filter_2_edit_button->Enabled = 0;
            }
        }
        this->field_filter_1_label->Text = labels[0];
        this->field_filter_2_label->Text = labels[1];
        // Теперь определим, будут ли доступны общие кнопки
        if (working_filters == FILTERS_NUM) this->add_new_filter_button->Enabled = 0;
        if (working_filters == 0) this->delete_all_filters_button->Enabled = 0;
    }
    private: System::Void fnum_button_Click(System::Object^ sender, System::EventArgs^
e) {
        this->cur_field = 0;
        filter_field_show();
    }
    private: System::Void name_button_Click(System::Object^ sender, System::EventArgs^
e) {
        this->cur_field = 1;
        filter_field_show();
    }
    private: System::Void dest_button_Click(System::Object^ sender, System::EventArgs^
e) {
        this->cur_field = 2;
        filter_field_show();
    }
    private: System::Void days_button_Click(System::Object^ sender, System::EventArgs^
e) {
        this->cur_field = 3;
        filter_field_show();
    }
    private: System::Void dep_time_button_Click(System::Object^ sender,
System::EventArgs^ e) {
        this->cur_field = 4;
        filter_field_show();
    }
    private: System::Void arr_time_button_Click(System::Object^ sender,
System::EventArgs^ e) {
        this->cur_field = 5;
        filter_field_show();
    }
    private: System::Void price_button_Click(System::Object^ sender, System::EventArgs^
e) {
        this->cur_field = 6;
        filter_field_show();
    }
    private: System::Void update_button_Click(System::Object^ sender, System::EventArgs^
e) {
        ReadFilters();
    }

```

```

        filter_field_show();
    }
private: System::Void field_filter_1_delete_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    // Читаем фильтры в массив
    Flight_filter filters[FILTERS_NUM];
    if (read_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл для чтения");
        return;
    }
    filters[0].apply[this->cur_field] = 0;
    if (write_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл для записи");
        return;
    }
    ReadFilters();
    filter_field_show();
}
private: System::Void field_filter_1_edit_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    EditFilterField^ p = gcnew EditFilterField(0, this->cur_field);
    p->ShowDialog();
    ReadFilters();
    filter_field_show();
}
private: System::Void field_filter_2_delete_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    // Читаем фильтры в массив
    Flight_filter filters[FILTERS_NUM];
    if (read_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл с фильтрами для чтения");
        return;
    }
    filters[1].apply[this->cur_field] = 0;
    if (write_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл с фильтрами для записи");
        return;
    }
    ReadFilters();
    filter_field_show();
}
private: System::Void field_filter_2_edit_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    EditFilterField^ p = gcnew EditFilterField(1, this->cur_field);
    p->ShowDialog();
    ReadFilters();
    filter_field_show();
}
private: System::Void add_new_filter_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    // Читаем фильтры в массив
    Flight_filter filters[FILTERS_NUM];
    if (read_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл с фильтрами для чтения");
        return;
    }
    int free_filter = -1; // Индекс свободного фильтра
    for (int f = 0; f < FILTERS_NUM && free_filter == -1; f++)
        if (filters[f].apply[this->cur_field] == 0)
            free_filter = f;
    if (free_filter == -1) {
        MessageBox::Show(L"ОШИБКА: Больше нет неиспользуемых полей");
        return;
    }
}

```

```

    }
    EditFilterField^ p = gcnew EditFilterField(free_filter, this->cur_field);
    p->ShowDialog();
    ReadFilters();
    filter_field_show();
}
private: System::Void delete_all_filters_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    // Читаем фильтры в массив
    Flight_filter filters[FILTERS_NUM];
    if (read_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл с фильтрами для чтения");
        return;
    }
    for (int f = 0; f < FILTERS_NUM; f++) filters[f].apply[this->cur_field] = 0;
    if (write_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл с фильтрами для записи");
        return;
    }
    ReadFilters();
    this->ClientSize = System::Drawing::Size(440 - 16, 296);
    this->cur_field = -1;
}
};
}

```

П.Б.10 Код EditFilterField.h

```

#pragma once

#include <msclr/marshal.h>

#include "structure.h"
#include "file_funcs.h"

namespace MarkflightsUI {
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;

    /// <summary>
    /// Сводка для EditFilterField
    /// </summary>
    public ref class EditFilterField : public System::Windows::Forms::Form
    {
        int filter_num;           // Номер фильтра в списке
        int field_num;            // Номер поля фильтра

    public:
        EditFilterField(int f, int n)
        {
            this->filter_num = f;
            this->field_num = n;
            InitializeComponent();
            if (f == 0) {
                this->logic_label->Visible = 0;
                this->and_radio->Visible = 0;
                this->or_radio->Visible = 0;
            }
            if (n < 4)

```

```

        this->ClientSize = System::Drawing::Size(280 - 16, 143);
    }

protected:
    /// <summary>
    /// Освободить все используемые ресурсы.
    /// </summary>
    ~EditFilterField()
    {
        if (components)
        {
            delete components;
        }
    }

private: System::Windows::Forms::Label^ compare_label;
private: System::Windows::Forms::RadioButton^ more_radio;
private: System::Windows::Forms::RadioButton^ less_radio;
private: System::Windows::Forms::RadioButton^ more_equals_radio;
private: System::Windows::Forms::RadioButton^ less_equals_radio;
private: System::Windows::Forms::RadioButton^ equals_radio;
private: System::Windows::Forms::Panel^ logic_panel;
private: System::Windows::Forms::Panel^ apply_panel;
private: System::Windows::Forms::Button^ save_button;
private: System::Windows::Forms::Label^ new_data_label;
private: System::Windows::Forms::TextBox^ new_data_textbox;
private: System::Windows::Forms::Label^ logic_label;
private: System::Windows::Forms::RadioButton^ and_radio;
private: System::Windows::Forms::RadioButton^ or_radio;
private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container ^components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        this->save_button = (gcnew System::Windows::Forms::Button());
        this->new_data_label = (gcnew System::Windows::Forms::Label());
        this->new_data_textbox = (gcnew
System::Windows::Forms::TextBox());
        this->logic_label = (gcnew System::Windows::Forms::Label());
        this->and_radio = (gcnew System::Windows::Forms::RadioButton());
        this->or_radio = (gcnew System::Windows::Forms::RadioButton());
        this->compare_label = (gcnew System::Windows::Forms::Label());
        this->more_radio = (gcnew
System::Windows::Forms::RadioButton());
        this->less_radio = (gcnew
System::Windows::Forms::RadioButton());
        this->more_equals_radio = (gcnew
System::Windows::Forms::RadioButton());
        this->less_equals_radio = (gcnew
System::Windows::Forms::RadioButton());
        this->equals_radio = (gcnew
System::Windows::Forms::RadioButton());
        this->logic_panel = (gcnew System::Windows::Forms::Panel());
        this->apply_panel = (gcnew System::Windows::Forms::Panel());
        this->logic_panel->SuspendLayout();
        this->apply_panel->SuspendLayout();
    }

```

```

        this->SuspendLayout();
        //
        // save_button
        //
        this->save_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->save_button->Location = System::Drawing::Point(12, 70);
        this->save_button->Name = L"save_button";
        this->save_button->Size = System::Drawing::Size(109, 31);
        this->save_button->TabIndex = 0;
        this->save_button->Text = L"Сохранить";
        this->save_button->UseVisualStyleBackColor = true;
        this->save_button->Click += gcnew System::EventHandler(this,
&EditFilterField::save_button_Click);
        //
        // new_data_label
        //
        this->new_data_label->AutoSize = true;
        this->new_data_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->new_data_label->Location = System::Drawing::Point(12, 9);
        this->new_data_label->Name = L"new_data_label";
        this->new_data_label->Size = System::Drawing::Size(130, 17);
        this->new_data_label->TabIndex = 1;
        this->new_data_label->Text = L"Новое значение";
        //
        // new_data_textbox
        //
        this->new_data_textbox->ForeColor =
System::Drawing::Color::Black;
        this->new_data_textbox->Location = System::Drawing::Point(148,
9);

        this->new_data_textbox->Name = L"new_data_textbox";
        this->new_data_textbox->Size = System::Drawing::Size(100, 20);
        this->new_data_textbox->TabIndex = 2;
        //
        // logic_label
        //
        this->logic_label->AutoSize = true;
        this->logic_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->logic_label->Location = System::Drawing::Point(12, 38);
        this->logic_label->Name = L"logic_label";
        this->logic_label->Size = System::Drawing::Size(163, 17);
        this->logic_label->TabIndex = 3;
        this->logic_label->Text = L"Логический операнд";
        //
        // and_radio
        //
        this->and_radio->AutoSize = true;
        this->and_radio->Checked = true;
        this->and_radio->Location = System::Drawing::Point(0, 0);
        this->and_radio->Name = L"and_radio";
        this->and_radio->Size = System::Drawing::Size(33, 17);
        this->and_radio->TabIndex = 4;
        this->and_radio->TabStop = true;
        this->and_radio->Text = L"И";

```

```

this->and_radio->UseVisualStyleBackColor = true;
//
// or_radio
//
this->or_radio->AutoSize = true;
this->or_radio->Location = System::Drawing::Point(0, 23);
this->or_radio->Name = L"or_radio";
this->or_radio->Size = System::Drawing::Size(49, 17);
this->or_radio->TabIndex = 5;
this->or_radio->Text = L"ИЛИ";
this->or_radio->UseVisualStyleBackColor = true;
//
// compare_label
//
this->compare_label->AutoSize = true;
this->compare_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->compare_label->Location = System::Drawing::Point(273, 9);
this->compare_label->Name = L"compare_label";
this->compare_label->Size = System::Drawing::Size(89, 17);
this->compare_label->TabIndex = 6;
this->compare_label->Text = L"Сравнение";
//
// more_radio
//
this->more_radio->AutoSize = true;
this->more_radio->Checked = true;
this->more_radio->Location = System::Drawing::Point(0, 0);
this->more_radio->Name = L"more_radio";
this->more_radio->Size = System::Drawing::Size(31, 17);
this->more_radio->TabIndex = 7;
this->more_radio->TabStop = true;
this->more_radio->Text = L">";
this->more_radio->UseVisualStyleBackColor = true;
//
// less_radio
//
this->less_radio->AutoSize = true;
this->less_radio->Location = System::Drawing::Point(37, 0);
this->less_radio->Name = L"less_radio";
this->less_radio->Size = System::Drawing::Size(31, 17);
this->less_radio->TabIndex = 8;
this->less_radio->Text = L"<";
this->less_radio->UseVisualStyleBackColor = true;
//
// more_equals_radio
//
this->more_equals_radio->AutoSize = true;
this->more_equals_radio->Location = System::Drawing::Point(0,
46);

this->more_equals_radio->Name = L"more_equals_radio";
this->more_equals_radio->Size = System::Drawing::Size(37, 17);
this->more_equals_radio->TabIndex = 9;
this->more_equals_radio->Text = L">=";
this->more_equals_radio->UseVisualStyleBackColor = true;
//
// less_equals_radio
//
this->less_equals_radio->AutoSize = true;
this->less_equals_radio->Location = System::Drawing::Point(43,
46);

```



```

this->less_equals_radio->Name = L"less_equals_radio";
this->less_equals_radio->Size = System::Drawing::Size(37, 17);
this->less_equals_radio->TabIndex = 10;
this->less_equals_radio->Text = L"<=";
this->less_equals_radio->UseVisualStyleBackColor = true;
//
// equals_radio
//
this->equals_radio->AutoSize = true;
this->equals_radio->Location = System::Drawing::Point(0, 23);
this->equals_radio->Name = L"equals_radio";
this->equals_radio->Size = System::Drawing::Size(31, 17);
this->equals_radio->TabIndex = 11;
this->equals_radio->Text = L"=";
this->equals_radio->UseVisualStyleBackColor = true;
//
// logic_panel
//
this->logic_panel->Controls->Add(this->or_radio);
this->logic_panel->Controls->Add(this->and_radio);
this->logic_panel->Location = System::Drawing::Point(180, 38);
this->logic_panel->Name = L"logic_panel";
this->logic_panel->Size = System::Drawing::Size(50, 51);
this->logic_panel->TabIndex = 13;
//
// apply_panel
//
this->apply_panel->Controls->Add(this->more_radio);
this->apply_panel->Controls->Add(this->less_radio);
this->apply_panel->Controls->Add(this->less_equals_radio);
this->apply_panel->Controls->Add(this->equals_radio);
this->apply_panel->Controls->Add(this->more_equals_radio);
this->apply_panel->Location = System::Drawing::Point(276, 38);
this->apply_panel->Name = L"apply_panel";
this->apply_panel->Size = System::Drawing::Size(84, 63);
this->apply_panel->TabIndex = 14;
//
// EditFilterField
//
this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
this->ClientSize = System::Drawing::Size(372, 143);
this->Controls->Add(this->apply_panel);
this->Controls->Add(this->logic_panel);
this->Controls->Add(this->compare_label);
this->Controls->Add(this->logic_label);
this->Controls->Add(this->new_data_textbox);
this->Controls->Add(this->new_data_label);
this->Controls->Add(this->save_button);
this->MaximizeBox = false;
this->MinimizeBox = false;
this->Name = L"EditFilterField";
this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
this->Text = L"Новое значение";
this->logic_panel->ResumeLayout(false);
this->logic_panel->PerformLayout();
this->apply_panel->ResumeLayout(false);
this->apply_panel->PerformLayout();
this->ResumeLayout(false);

```

```

        this->PerformLayout();
    }
#pragma endregion
private: System::Void save_button_Click(System::Object^ sender, System::EventArgs^ e) {
    // Читаем фильтры в массив
    Flight_filter filters[FILTERS_NUM];
    if (read_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл для чтения");
        return;
    }

    int f = this->filter_num; // Номер фильтра
    int n = this->field_num; // Номер поля

    // Для начала проверим корректность информации
    String^ data = this->new_data_textbox->Text;
    if (n == 0) {
        try {
            filters[f].fnum = Convert::ToInt32(data);
        }
        catch (...) {
            MessageBox::Show(L"Некорректный номер рейса");
            return;
        }
        if (filters[f].fnum <= 0) {
            MessageBox::Show(L"Некорректный номер рейса");
            return;
        }
    }
    else if (n == 1) {
        copy_char_from_string(filters[f].name, TEXT_LEN, data);
        if (!filters[f].name[0]) {
            MessageBox::Show(L"Некорректный тип самолёта");
            return;
        }
    }
    else if (n == 2) {
        copy_char_from_string(filters[f].dest, TEXT_LEN, data);
        if (!filters[f].dest[0]) {
            MessageBox::Show(L"Некорректный пункт назначения");
            return;
        }
    }
    else if (n == 3) {
        char user_input[30]; // Буфер для хранения String в char[]

        if (data->Length == 0) { // Проверим длину строки
            MessageBox::Show(L"Некорректные дни отправления");
            return;
        }
        copy_char_from_string(user_input, 30, data);

        // Сохраним все допустимые цифры
        char symbols[30]; // Массив всех допустимых цифр
        int last_ind = 0; // Индекс последнего элемента массива
        for (int i = 1; i <= 7; i++)
            symbols[last_ind++] = '0' + i;
        symbols[last_ind++] = ' '; symbols[last_ind++] = ',';
        symbols[last_ind++] = '\\0';

        // Проверка на лишние символы
    }
}

```

```

if (strspn(user_input, symbols) != strlen(user_input)) {
    MessageBox::Show(L"Некорректные дни отправления");
    return;
}

char* number; // Указатель на цифру для strtok
number = strtok(user_input, " ,");
last_ind = 0; // Обнуляем счётчик

if ((number == NULL) || (strlen(number) > 1)) {
    MessageBox::Show(L"Некорректные дни отправления");
    return;
}
filters[f].days[last_ind++] = *number - '0';
while ((number = strtok(NULL, " ,")) != NULL) {
    if (strlen(number) > 1) {
        MessageBox::Show(L"Некорректные дни отправления");
        return;
    }
    int cur_number = *number - '0';
    // Проверим, есть ли эта цифра в списке
    int in_array = 0;
    for (int i = 0; i < last_ind; i++)
        if (filters[f].days[i] == cur_number) in_array = 1;
    if (!in_array) filters[f].days[last_ind++] = cur_number;
}
// Сортируем пузырьком
for (int k = 1; k < last_ind; k++)
    for (int i = 0; i < last_ind - k; i++)
        if (filters[f].days[i] > filters[f].days[i + 1]) {
            int buffer = filters[f].days[i];
            filters[f].days[i] = filters[f].days[i + 1];
            filters[f].days[i + 1] = buffer;
        }
}
else if (n == 4) {
    String^ str_dep_time = data->Replace(":", "");
    // Проверим длину
    if (str_dep_time->Length < 3) { // Длина не менее 3-ух символов, время
некорректное
        MessageBox::Show(L"Некорректное время вылета – используйте
формат 0:00");
        return;
    }
    else if (str_dep_time->Length == 3) // Если 3 символа, то добавляем 0
и возвращаем ":"
        str_dep_time = "0" + str_dep_time[0].ToString() +
        ":" + str_dep_time[1].ToString() + str_dep_time[2].ToString();
    else if (str_dep_time->Length == 4) // Если 4 символа, то просто
возвращаем ":"
        str_dep_time = str_dep_time[0].ToString() +
str_dep_time[1].ToString()
        + ":" + str_dep_time[2].ToString() + str_dep_time[3].ToString();
    else { // Длина не более 4-ёх символов, время некорректное
        MessageBox::Show(L"Некорректное время вылета – используйте
формат 00:00");
        return;
    }
}
// Конвертируем время в int
int dep_time_try = convert_string_time(str_dep_time);
if (dep_time_try == -1) {
    MessageBox::Show(L"Некорректное время вылета");
    return;
}

```

```

        }
        filters[f].dep_time = dep_time_try;
    }
    else if (n == 5) {
        String^ str_arr_time = data->Replace(":", "");
        // Проверим длину
        if (str_arr_time->Length < 3) { // Длина не менее 3-ух символов, время
некорректное
            MessageBox::Show(L"Некорректное время прилёта – используйте
формат 0:00");
            return;
        }
        else if (str_arr_time->Length == 3) // Если 3 символа, то добавляем 0
и возвращаем ":"
            str_arr_time = "0" + str_arr_time[0].ToString()
            + ":" + str_arr_time[1].ToString() + str_arr_time[2].ToString();
        else if (str_arr_time->Length == 4) // Если 4 символа, то просто
возвращаем ":"
            str_arr_time = str_arr_time[0].ToString() +
str_arr_time[1].ToString()
            + ":" + str_arr_time[2].ToString() + str_arr_time[3].ToString();
        else { // Длина не более 4-ёх символов, время некорректное
            MessageBox::Show(L"Некорректное время прилёта – используйте
формат 00:00");
            return;
        }
        // Конвертируем время в int
        int arr_time_try = convert_string_time(str_arr_time);
        if (arr_time_try == -1) {
            MessageBox::Show(L"Некорректное время прилёта");
            return;
        }
        filters[f].arr_time = arr_time_try;
    }
    else if (n == 6) {
        try {
            filters[f].price = Convert::ToDouble(data);
        }
        catch (...) {
            MessageBox::Show(L"Некорректная цена билета");
            return;
        }
        if (filters[f].price <= 0) {
            MessageBox::Show(L"Некорректная цена билета");
            return;
        }
    }
}
// Если это не первый по счёту фильтр, то у него можно задать логический
оператор
if (f != 0)
    if (this->or_radio->Checked) filters[f].logic[n] = 1;
    else filters[f].logic[n] = 0;
else filters[f].logic[n] = 0;
// Если это время или цена, то можно задать оператор сравнения
if (4 <= n && n <= 6) {
    if (this->more_radio->Checked) filters[f].apply[n] = 1;
    // >
    else if (this->less_radio->Checked) filters[f].apply[n] = 2;
    // <
    else if (this->equals_radio->Checked) filters[f].apply[n] = 3;
    // =
    else if (this->more_equals_radio->Checked) filters[f].apply[n] = 4;
    // >=

```

```

        else if (this->less_equals_radio->Checked) filters[f].apply[n] = 5;
    // <=
    }
    else filters[f].apply[n] = 1;
    // Пробуем сохранить фильтры
    int err;
    if (err = write_filters(filters)) {
        MessageBox::Show(L"Не удалось открыть файл для записи - ошибка " +
err);
        return;
    }
    this->Close();
}
};
}

```

П.Б.11 Код AddFlight.h

```

#pragma once

#include <msclr/marshal.h>    // Создание char* из String^

#include "structure.h"
#include "file_funcs.h"

namespace MarkflightsUI {
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;

    /// <summary>
    /// Сводка для AddFlight
    /// </summary>
    public ref class AddFlight : public System::Windows::Forms::Form
    {
    public:
        AddFlight(void)
        {
            InitializeComponent();
        }

    protected:
        /// <summary>
        /// Освободить все используемые ресурсы.
        /// </summary>
        ~AddFlight()
        {
            if (components)
            {
                delete components;
            }
        }

    private: System::Windows::Forms::Label^ adding_label;
    private: System::Windows::Forms::Label^ fnum_label;
    private: System::Windows::Forms::Label^ name_label;
    private: System::Windows::Forms::Label^ dest_label;
    private: System::Windows::Forms::Label^ days_label;
    private: System::Windows::Forms::CheckBox^ checkbox_1;
    private: System::Windows::Forms::CheckBox^ checkbox_2;
    private: System::Windows::Forms::CheckBox^ checkbox_3;
    }
}

```

```

private: System::Windows::Forms::CheckBox^ checkbox_4;
private: System::Windows::Forms::CheckBox^ checkbox_5;
private: System::Windows::Forms::CheckBox^ checkbox_6;
private: System::Windows::Forms::CheckBox^ checkbox_7;
private: System::Windows::Forms::MaskedTextBox^ dep_time_textbox;
private: System::Windows::Forms::Label^ dep_time_label;
private: System::Windows::Forms::MaskedTextBox^ arr_time_textbox;
private: System::Windows::Forms::Label^ arr_time_label;
private: System::Windows::Forms::Label^ price_label;
private: System::Windows::Forms::Button^ add_button;
private: System::Windows::Forms::TextBox^ fnum_textbox;
private: System::Windows::Forms::TextBox^ name_textbox;
private: System::Windows::Forms::TextBox^ dest_textbox;
private: System::Windows::Forms::TextBox^ price_textbox;
private: System::Windows::Forms::Label^ error_label;
private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container ^components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        this->adding_label = (gcnew System::Windows::Forms::Label());
        this->fnum_label = (gcnew System::Windows::Forms::Label());
        this->name_label = (gcnew System::Windows::Forms::Label());
        this->dest_label = (gcnew System::Windows::Forms::Label());
        this->days_label = (gcnew System::Windows::Forms::Label());
        this->checkbox_1 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_2 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_3 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_4 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_5 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_6 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_7 = (gcnew System::Windows::Forms::CheckBox());
        this->dep_time_textbox = (gcnew
System::Windows::Forms::MaskedTextBox());
        this->dep_time_label = (gcnew System::Windows::Forms::Label());
        this->arr_time_textbox = (gcnew
System::Windows::Forms::MaskedTextBox());
        this->arr_time_label = (gcnew System::Windows::Forms::Label());
        this->price_label = (gcnew System::Windows::Forms::Label());
        this->add_button = (gcnew System::Windows::Forms::Button());
        this->fnum_textbox = (gcnew System::Windows::Forms::TextBox());
        this->name_textbox = (gcnew System::Windows::Forms::TextBox());
        this->dest_textbox = (gcnew System::Windows::Forms::TextBox());
        this->price_textbox = (gcnew System::Windows::Forms::TextBox());
        this->error_label = (gcnew System::Windows::Forms::Label());
        this->SuspendLayout();
        //
        // adding_label
        //
        this->adding_label->AutoSize = true;
        this->adding_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
            static_cast<System::Byte>(204)));
        this->adding_label->Location = System::Drawing::Point(70, 9);

```

```

this->adding_label->Name = L"adding_label";
this->adding_label->Size = System::Drawing::Size(167, 20);
this->adding_label->TabIndex = 0;
this->adding_label->Text = L"Добавление рейса";
//
// fnum_label
//
this->fnum_label->AutoSize = true;
this->fnum_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->fnum_label->Location = System::Drawing::Point(12, 50);
this->fnum_label->Name = L"fnum_label";
this->fnum_label->Size = System::Drawing::Size(94, 17);
this->fnum_label->TabIndex = 1;
this->fnum_label->Text = L"Номер рейса";
//
// name_label
//
this->name_label->AutoSize = true;
this->name_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->name_label->Location = System::Drawing::Point(12, 76);
this->name_label->Name = L"name_label";
this->name_label->Size = System::Drawing::Size(100, 17);
this->name_label->TabIndex = 3;
this->name_label->Text = L"Тип самолёта";
//
// dest_label
//
this->dest_label->AutoSize = true;
this->dest_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->dest_label->Location = System::Drawing::Point(12, 102);
this->dest_label->Name = L"dest_label";
this->dest_label->Size = System::Drawing::Size(130, 17);
this->dest_label->TabIndex = 5;
this->dest_label->Text = L"Пункт назначения";
//
// days_label
//
this->days_label->AutoSize = true;
this->days_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->days_label->Location = System::Drawing::Point(12, 133);
this->days_label->Name = L"days_label";
this->days_label->Size = System::Drawing::Size(125, 17);
this->days_label->TabIndex = 7;
this->days_label->Text = L"Дни отправления";
//
// checkbox_1
//
this->checkbox_1->AutoSize = true;
this->checkbox_1->Location = System::Drawing::Point(15, 163);
this->checkbox_1->Name = L"checkbox_1";
this->checkbox_1->Size = System::Drawing::Size(113, 17);

```

```

this->checkbox_1->TabIndex = 8;
this->checkbox_1->Text = L"- 1 (понедельник)";
this->checkbox_1->UseVisualStyleBackColor = true;
//
// checkbox_2
//
this->checkbox_2->AutoSize = true;
this->checkbox_2->Location = System::Drawing::Point(15, 186);
this->checkbox_2->Name = L"checkbox_2";
this->checkbox_2->Size = System::Drawing::Size(88, 17);
this->checkbox_2->TabIndex = 9;
this->checkbox_2->Text = L"- 2 (вторник)";
this->checkbox_2->UseVisualStyleBackColor = true;
//
// checkbox_3
//
this->checkbox_3->AutoSize = true;
this->checkbox_3->Location = System::Drawing::Point(15, 209);
this->checkbox_3->Name = L"checkbox_3";
this->checkbox_3->Size = System::Drawing::Size(77, 17);
this->checkbox_3->TabIndex = 10;
this->checkbox_3->Text = L"- 3 (среда)";
this->checkbox_3->UseVisualStyleBackColor = true;
//
// checkbox_4
//
this->checkbox_4->AutoSize = true;
this->checkbox_4->Location = System::Drawing::Point(15, 232);
this->checkbox_4->Name = L"checkbox_4";
this->checkbox_4->Size = System::Drawing::Size(86, 17);
this->checkbox_4->TabIndex = 11;
this->checkbox_4->Text = L"- 4 (четверг)";
this->checkbox_4->UseVisualStyleBackColor = true;
//
// checkbox_5
//
this->checkbox_5->AutoSize = true;
this->checkbox_5->Location = System::Drawing::Point(15, 255);
this->checkbox_5->Name = L"checkbox_5";
this->checkbox_5->Size = System::Drawing::Size(88, 17);
this->checkbox_5->TabIndex = 12;
this->checkbox_5->Text = L"- 5 (пятница)";
this->checkbox_5->UseVisualStyleBackColor = true;
//
// checkbox_6
//
this->checkbox_6->AutoSize = true;
this->checkbox_6->Location = System::Drawing::Point(15, 278);
this->checkbox_6->Name = L"checkbox_6";
this->checkbox_6->Size = System::Drawing::Size(87, 17);
this->checkbox_6->TabIndex = 13;
this->checkbox_6->Text = L"- 6 (суббота)";
this->checkbox_6->UseVisualStyleBackColor = true;
//
// checkbox_7
//
this->checkbox_7->AutoSize = true;
this->checkbox_7->Location = System::Drawing::Point(15, 301);
this->checkbox_7->Name = L"checkbox_7";
this->checkbox_7->Size = System::Drawing::Size(113, 17);
this->checkbox_7->TabIndex = 14;
this->checkbox_7->Text = L"- 7 (воскресенье)";
this->checkbox_7->UseVisualStyleBackColor = true;

```



```

//
// dep_time_textbox
//
this->dep_time_textbox->Location = System::Drawing::Point(183,
330);
this->dep_time_textbox->Mask = L"00:00";
this->dep_time_textbox->Name = L"dep_time_textbox";
this->dep_time_textbox->PromptChar = '0';
this->dep_time_textbox->Size = System::Drawing::Size(100, 20);
this->dep_time_textbox->TabIndex = 16;
this->dep_time_textbox->ValidatingType =
System::DateTime::typeid;
//
// dep_time_label
//
this->dep_time_label->AutoSize = true;
this->dep_time_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->dep_time_label->Location = System::Drawing::Point(12,
330);
this->dep_time_label->Name = L"dep_time_label";
this->dep_time_label->Size = System::Drawing::Size(102, 17);
this->dep_time_label->TabIndex = 15;
this->dep_time_label->Text = L"Время вылета";
//
// arr_time_textbox
//
this->arr_time_textbox->Location = System::Drawing::Point(183,
356);
this->arr_time_textbox->Mask = L"00:00";
this->arr_time_textbox->Name = L"arr_time_textbox";
this->arr_time_textbox->PromptChar = '0';
this->arr_time_textbox->Size = System::Drawing::Size(100, 20);
this->arr_time_textbox->TabIndex = 18;
this->arr_time_textbox->ValidatingType =
System::DateTime::typeid;
//
// arr_time_label
//
this->arr_time_label->AutoSize = true;
this->arr_time_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->arr_time_label->Location = System::Drawing::Point(12,
356);
this->arr_time_label->Name = L"arr_time_label";
this->arr_time_label->Size = System::Drawing::Size(109, 17);
this->arr_time_label->TabIndex = 17;
this->arr_time_label->Text = L"Время прилёта";
//
// price_label
//
this->price_label->AutoSize = true;
this->price_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->price_label->Location = System::Drawing::Point(12, 382);
this->price_label->Name = L"price_label";
this->price_label->Size = System::Drawing::Size(94, 17);

```

```

        this->price_label->TabIndex = 19;
        this->price_label->Text = L"Цена билета";
        //
        // add_button
        //
        this->add_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->add_button->Location = System::Drawing::Point(88, 458);
        this->add_button->Name = L"add_button";
        this->add_button->Size = System::Drawing::Size(118, 44);
        this->add_button->TabIndex = 21;
        this->add_button->Text = L"Добавить";
        this->add_button->UseVisualStyleBackColor = true;
        this->add_button->Click += gcnew System::EventHandler(this,
&AddFlight::add_button_Click);
        //
        // fnum_textbox
        //
        this->fnum_textbox->Location = System::Drawing::Point(183, 50);
        this->fnum_textbox->Name = L"fnum_textbox";
        this->fnum_textbox->Size = System::Drawing::Size(100, 20);
        this->fnum_textbox->TabIndex = 22;
        //
        // name_textbox
        //
        this->name_textbox->Location = System::Drawing::Point(183, 76);
        this->name_textbox->Name = L"name_textbox";
        this->name_textbox->Size = System::Drawing::Size(100, 20);
        this->name_textbox->TabIndex = 23;
        //
        // dest_textbox
        //
        this->dest_textbox->Location = System::Drawing::Point(183, 102);
        this->dest_textbox->Name = L"dest_textbox";
        this->dest_textbox->Size = System::Drawing::Size(100, 20);
        this->dest_textbox->TabIndex = 24;
        //
        // price_textbox
        //
        this->price_textbox->Location = System::Drawing::Point(183,
382);

        this->price_textbox->Name = L"price_textbox";
        this->price_textbox->Size = System::Drawing::Size(100, 20);
        this->price_textbox->TabIndex = 25;
        //
        // error_label
        //
        this->error_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->error_label->ForeColor = System::Drawing::Color::Red;
        this->error_label->Location = System::Drawing::Point(12, 418);
        this->error_label->Name = L"error_label";
        this->error_label->Size = System::Drawing::Size(275, 37);
        this->error_label->TabIndex = 26;
        this->error_label->TextAlign =
System::Drawing::ContentAlignment::MiddleCenter;
        //
        // AddFlight
        //

```

```

        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(304, 516);
        this->Controls->Add(this->error_label);
        this->Controls->Add(this->price_textbox);
        this->Controls->Add(this->dest_textbox);
        this->Controls->Add(this->name_textbox);
        this->Controls->Add(this->fnum_textbox);
        this->Controls->Add(this->add_button);
        this->Controls->Add(this->price_label);
        this->Controls->Add(this->arr_time_textbox);
        this->Controls->Add(this->arr_time_label);
        this->Controls->Add(this->dep_time_textbox);
        this->Controls->Add(this->dep_time_label);
        this->Controls->Add(this->checkbox7);
        this->Controls->Add(this->checkbox6);
        this->Controls->Add(this->checkbox5);
        this->Controls->Add(this->checkbox4);
        this->Controls->Add(this->checkbox3);
        this->Controls->Add(this->checkbox2);
        this->Controls->Add(this->checkbox1);
        this->Controls->Add(this->days_label);
        this->Controls->Add(this->dest_label);
        this->Controls->Add(this->name_label);
        this->Controls->Add(this->fnum_label);
        this->Controls->Add(this->adding_label);
        this->MaximizeBox = false;
        this->MinimizeBox = false;
        this->Name = L"AddFlight";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"Добавление рейса";
        this->ResumeLayout(false);
        this->PerformLayout();
    }
#pragma endregion
private: System::Void add_button_Click(System::Object^ sender, System::EventArgs^ e)
{
    Flight flight_buffer;    // Структура для хранения информации
    // Получаем и проверяем fnum
    try {
        flight_buffer.fnum = Convert::ToInt32(this->fnum_textbox->Text);
    }
    catch (...) {
        this->error_label->Text = L"Некорректный номер рейса";
        return;
    }
    if (flight_buffer.fnum <= 0) {
        this->error_label->Text = L"Некорректный номер рейса";
        return;
    }
    // Получаем и проверяем name
    copy_char_from_string(flight_buffer.name, TEXT_LEN, this->name_textbox->Text);
    if (!flight_buffer.name[0]) {
        this->error_label->Text = L"Некорректный тип самолёта";
        return;
    }
    // Получаем и проверяем dest

```

```

        copy_char_from_string(flight_buffer.dest, TEXT_LEN, this->dest_textbox-
>Text);
        if (!flight_buffer.dest[0]) {
            this->error_label->Text = L"Некорректный пункт назначения";
            return;
        }
        // Получаем и проверяем days
        int day = 0;
        if (this->checkbox_1->Checked) flight_buffer.days[day++] = 1;
        if (this->checkbox_2->Checked) flight_buffer.days[day++] = 2;
        if (this->checkbox_3->Checked) flight_buffer.days[day++] = 3;
        if (this->checkbox_4->Checked) flight_buffer.days[day++] = 4;
        if (this->checkbox_5->Checked) flight_buffer.days[day++] = 5;
        if (this->checkbox_6->Checked) flight_buffer.days[day++] = 6;
        if (this->checkbox_7->Checked) flight_buffer.days[day++] = 7;
        if (day == 0) {
            this->error_label->Text = L"Нужно выбрать хотя бы один день
отправления";
            return;
        }
        for (int i = day; i < 7; i++) flight_buffer.days[i] = 0;
        // Получаем и проверяем dep_time
        int dep_time_try = convert_string_time(this->dep_time_textbox->Text);
        if (dep_time_try == -1) {
            this->error_label->Text = L"Некорректное время вылета";
            return;
        }
        flight_buffer.dep_time = dep_time_try;
        // Получаем и проверяем arr_time
        int arr_time_try = convert_string_time(this->arr_time_textbox->Text);
        if (arr_time_try == -1) {
            this->error_label->Text = L"Некорректное время прилёта";
            return;
        }
        flight_buffer.arr_time = arr_time_try;
        // Получаем и проверяем price
        try {
            flight_buffer.price = Convert::ToDouble(this->price_textbox->Text-
>Replace(".", ","));
        }
        catch (...) {
            this->error_label->Text = L"Некорректная цена билета";
            return;
        }
        if (flight_buffer.price <= 0) {
            this->error_label->Text = L"Некорректная цена билета";
            return;
        }
        this->error_label->Text = "";
        // Записываем в БД
        char file_path[FILE_NAME_LEN]; // Название файла БД
        if (get_file(file_path) == -1) {
            MessageBox::Show("Ошибка открытия файла для записи: недостаточно прав
для получения пути");
            return;
        }
        FILE* table = fopen(file_path, "a");
        if (table == NULL) {
            MessageBox::Show("Недостаточно прав для редактирования файла.
Добавление отменено");
            return;
        }
        write_line(table, flight_buffer);

```

```

        fclose(table);
        this->Close();
    }
};
}

```

П.Б.12 Код EditFlights.h

```

#pragma once

#include <msclr/marshal.h>
#include <locale.h>
#include <windows.h>
#include <stdlib.h>

#include "structure.h"
#include "file_funcs.h"

namespace MarkflightsUI {
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;

    /// <summary>
    /// Сводка для EditFlights
    /// </summary>
    public ref class EditFlights : public System::Windows::Forms::Form
    {
    public:
        EditFlights(void)
        {
            InitializeComponent();
        }

    protected:
        /// <summary>
        /// Освободить все используемые ресурсы.
        /// </summary>
        ~EditFlights()
        {
            if (components)
            {
                delete components;
            }
        }

    private: System::Windows::Forms::Label^ editing_label;
    private: System::Windows::Forms::Label^ fnum_label;
    private: System::Windows::Forms::Label^ name_label;
    private: System::Windows::Forms::Label^ dest_label;
    private: System::Windows::Forms::Label^ days_label;
    private: System::Windows::Forms::CheckBox^ checkbox_1;
    private: System::Windows::Forms::CheckBox^ checkbox_2;
    private: System::Windows::Forms::CheckBox^ checkbox_3;
    private: System::Windows::Forms::CheckBox^ checkbox_4;
    private: System::Windows::Forms::CheckBox^ checkbox_5;
    private: System::Windows::Forms::CheckBox^ checkbox_6;
    private: System::Windows::Forms::CheckBox^ checkbox_7;
    private: System::Windows::Forms::MaskedTextBox^ dep_time_textbox;
    private: System::Windows::Forms::Label^ dep_time_label;
    private: System::Windows::Forms::MaskedTextBox^ arr_time_textbox;
    private: System::Windows::Forms::Label^ arr_time_label;
    }
}

```

```

private: System::Windows::Forms::Label^ price_label;
private: System::Windows::Forms::Button^ edit_button;
private: System::Windows::Forms::TextBox^ fnum_textbox;
private: System::Windows::Forms::TextBox^ name_textbox;
private: System::Windows::Forms::TextBox^ dest_textbox;
private: System::Windows::Forms::TextBox^ price_textbox;
private: System::Windows::Forms::Label^ error_label;
private: System::Windows::Forms::CheckBox^ fnum_checkbox;
private: System::Windows::Forms::CheckBox^ name_checkbox;
private: System::Windows::Forms::CheckBox^ dest_checkbox;
private: System::Windows::Forms::CheckBox^ days_checkbox;
private: System::Windows::Forms::CheckBox^ dep_time_checkbox;
private: System::Windows::Forms::CheckBox^ arr_time_checkbox;
private: System::Windows::Forms::CheckBox^ price_checkbox;
private: System::Windows::Forms::Label^ check_label;
private: System::Windows::Forms::RadioButton^ from_array_radio;
private: System::Windows::Forms::RadioButton^ filtered_radio;
private: System::Windows::Forms::Label^ apply_label;
private: System::Windows::Forms::TextBox^ to_edit_textbox;

private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container^ components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        this->editing_label = (gcnew System::Windows::Forms::Label());
        this->fnum_label = (gcnew System::Windows::Forms::Label());
        this->name_label = (gcnew System::Windows::Forms::Label());
        this->dest_label = (gcnew System::Windows::Forms::Label());
        this->days_label = (gcnew System::Windows::Forms::Label());
        this->checkbox_1 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_2 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_3 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_4 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_5 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_6 = (gcnew System::Windows::Forms::CheckBox());
        this->checkbox_7 = (gcnew System::Windows::Forms::CheckBox());
        this->dep_time_textbox = (gcnew
System::Windows::Forms::MaskedTextBox());
        this->dep_time_label = (gcnew System::Windows::Forms::Label());
        this->arr_time_textbox = (gcnew
System::Windows::Forms::MaskedTextBox());
        this->arr_time_label = (gcnew System::Windows::Forms::Label());
        this->price_label = (gcnew System::Windows::Forms::Label());
        this->edit_button = (gcnew System::Windows::Forms::Button());
        this->fnum_textbox = (gcnew System::Windows::Forms::TextBox());
        this->name_textbox = (gcnew System::Windows::Forms::TextBox());
        this->dest_textbox = (gcnew System::Windows::Forms::TextBox());
        this->price_textbox = (gcnew System::Windows::Forms::TextBox());
        this->error_label = (gcnew System::Windows::Forms::Label());
        this->fnum_checkbox = (gcnew
System::Windows::Forms::CheckBox());
        this->name_checkbox = (gcnew
System::Windows::Forms::CheckBox());

```

```

        this->dest_checkbox = (gcnew
System::Windows::Forms::CheckBox());
        this->days_checkbox = (gcnew
System::Windows::Forms::CheckBox());
        this->dep_time_checkbox = (gcnew
System::Windows::Forms::CheckBox());
        this->arr_time_checkbox = (gcnew
System::Windows::Forms::CheckBox());
        this->price_checkbox = (gcnew
System::Windows::Forms::CheckBox());
        this->check_label = (gcnew System::Windows::Forms::Label());
        this->from_array_radio = (gcnew
System::Windows::Forms::RadioButton());
        this->filtered_radio = (gcnew
System::Windows::Forms::RadioButton());
        this->apply_label = (gcnew System::Windows::Forms::Label());
        this->to_edit_textbox = (gcnew
System::Windows::Forms::TextBox());
        this->SuspendLayout();
        //
        // editing_label
        //
        this->editing_label->AutoSize = true;
        this->editing_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->editing_label->Location = System::Drawing::Point(101, 9);
        this->editing_label->Name = L"editing_label";
        this->editing_label->Size = System::Drawing::Size(133, 20);
        this->editing_label->TabIndex = 0;
        this->editing_label->Text = L"Новые данные";
        //
        // fnum_label
        //
        this->fnum_label->AutoSize = true;
        this->fnum_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->fnum_label->Location = System::Drawing::Point(32, 68);
        this->fnum_label->Name = L"fnum_label";
        this->fnum_label->Size = System::Drawing::Size(94, 17);
        this->fnum_label->TabIndex = 1;
        this->fnum_label->Text = L"Номер рейса";
        //
        // name_label
        //
        this->name_label->AutoSize = true;
        this->name_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->name_label->Location = System::Drawing::Point(32, 94);
        this->name_label->Name = L"name_label";
        this->name_label->Size = System::Drawing::Size(100, 17);
        this->name_label->TabIndex = 3;
        this->name_label->Text = L"Тип самолёта";
        //
        // dest_label
        //
        this->dest_label->AutoSize = true;

```

```

        this->dest_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->dest_label->Location = System::Drawing::Point(32, 120);
        this->dest_label->Name = L"dest_label";
        this->dest_label->Size = System::Drawing::Size(130, 17);
        this->dest_label->TabIndex = 5;
        this->dest_label->Text = L"Пункт назначения";
        //
        // days_label
        //
        this->days_label->AutoSize = true;
        this->days_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->days_label->Location = System::Drawing::Point(32, 151);
        this->days_label->Name = L"days_label";
        this->days_label->Size = System::Drawing::Size(125, 17);
        this->days_label->TabIndex = 7;
        this->days_label->Text = L"Дни отправления";
        //
        // checkbox_1
        //
        this->checkbox_1->AutoSize = true;
        this->checkbox_1->Location = System::Drawing::Point(35, 181);
        this->checkbox_1->Name = L"checkbox_1";
        this->checkbox_1->Size = System::Drawing::Size(113, 17);
        this->checkbox_1->TabIndex = 8;
        this->checkbox_1->Text = L"- 1 (понедельник)";
        this->checkbox_1->UseVisualStyleBackColor = true;
        //
        // checkbox_2
        //
        this->checkbox_2->AutoSize = true;
        this->checkbox_2->Location = System::Drawing::Point(35, 204);
        this->checkbox_2->Name = L"checkbox_2";
        this->checkbox_2->Size = System::Drawing::Size(88, 17);
        this->checkbox_2->TabIndex = 9;
        this->checkbox_2->Text = L"- 2 (вторник)";
        this->checkbox_2->UseVisualStyleBackColor = true;
        //
        // checkbox_3
        //
        this->checkbox_3->AutoSize = true;
        this->checkbox_3->Location = System::Drawing::Point(35, 227);
        this->checkbox_3->Name = L"checkbox_3";
        this->checkbox_3->Size = System::Drawing::Size(77, 17);
        this->checkbox_3->TabIndex = 10;
        this->checkbox_3->Text = L"- 3 (среда)";
        this->checkbox_3->UseVisualStyleBackColor = true;
        //
        // checkbox_4
        //
        this->checkbox_4->AutoSize = true;
        this->checkbox_4->Location = System::Drawing::Point(35, 250);
        this->checkbox_4->Name = L"checkbox_4";
        this->checkbox_4->Size = System::Drawing::Size(86, 17);
        this->checkbox_4->TabIndex = 11;
        this->checkbox_4->Text = L"- 4 (четверг)";
        this->checkbox_4->UseVisualStyleBackColor = true;
        //

```



```

// checkbox_5
//
this->checkbox_5->AutoSize = true;
this->checkbox_5->Location = System::Drawing::Point(35, 273);
this->checkbox_5->Name = L"checkbox_5";
this->checkbox_5->Size = System::Drawing::Size(88, 17);
this->checkbox_5->TabIndex = 12;
this->checkbox_5->Text = L"- 5 (пятница)";
this->checkbox_5->UseVisualStyleBackColor = true;
//
// checkbox_6
//
this->checkbox_6->AutoSize = true;
this->checkbox_6->Location = System::Drawing::Point(35, 296);
this->checkbox_6->Name = L"checkbox_6";
this->checkbox_6->Size = System::Drawing::Size(87, 17);
this->checkbox_6->TabIndex = 13;
this->checkbox_6->Text = L"- 6 (суббота)";
this->checkbox_6->UseVisualStyleBackColor = true;
//
// checkbox_7
//
this->checkbox_7->AutoSize = true;
this->checkbox_7->Location = System::Drawing::Point(35, 319);
this->checkbox_7->Name = L"checkbox_7";
this->checkbox_7->Size = System::Drawing::Size(113, 17);
this->checkbox_7->TabIndex = 14;
this->checkbox_7->Text = L"- 7 (воскресенье)";
this->checkbox_7->UseVisualStyleBackColor = true;
//
// dep_time_textbox
//
this->dep_time_textbox->Location = System::Drawing::Point(203,
348);

this->dep_time_textbox->Mask = L"00:00";
this->dep_time_textbox->Name = L"dep_time_textbox";
this->dep_time_textbox->PromptChar = '0';
this->dep_time_textbox->Size = System::Drawing::Size(100, 20);
this->dep_time_textbox->TabIndex = 16;
this->dep_time_textbox->ValidatingType =
System::DateTime::typeid;
//
// dep_time_label
//
this->dep_time_label->AutoSize = true;
this->dep_time_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->dep_time_label->Location = System::Drawing::Point(32,
348);

this->dep_time_label->Name = L"dep_time_label";
this->dep_time_label->Size = System::Drawing::Size(102, 17);
this->dep_time_label->TabIndex = 15;
this->dep_time_label->Text = L"Время вылета";
//
// arr_time_textbox
//
this->arr_time_textbox->Location = System::Drawing::Point(203,
374);

this->arr_time_textbox->Mask = L"00:00";
this->arr_time_textbox->Name = L"arr_time_textbox";
this->arr_time_textbox->PromptChar = '0';

```

```

        this->arr_time_textbox->Size = System::Drawing::Size(100, 20);
        this->arr_time_textbox->TabIndex = 18;
        this->arr_time_textbox->ValidatingType =
System::DateTime::typeid;
        //
        // arr_time_label
        //
        this->arr_time_label->AutoSize = true;
        this->arr_time_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->arr_time_label->Location = System::Drawing::Point(32,
374);
        this->arr_time_label->Name = L"arr_time_label";
        this->arr_time_label->Size = System::Drawing::Size(109, 17);
        this->arr_time_label->TabIndex = 17;
        this->arr_time_label->Text = L"Время прилёта";
        //
        // price_label
        //
        this->price_label->AutoSize = true;
        this->price_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->price_label->Location = System::Drawing::Point(32, 400);
        this->price_label->Name = L"price_label";
        this->price_label->Size = System::Drawing::Size(94, 17);
        this->price_label->TabIndex = 19;
        this->price_label->Text = L"Цена билета";
        //
        // edit_button
        //
        this->edit_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->edit_button->Location = System::Drawing::Point(105, 584);
        this->edit_button->Name = L"edit_button";
        this->edit_button->Size = System::Drawing::Size(118, 44);
        this->edit_button->TabIndex = 21;
        this->edit_button->Text = L"Применить";
        this->edit_button->UseVisualStyleBackColor = true;
        this->edit_button->Click += gcnew System::EventHandler(this,
&EditFlights::edit_button_Click);
        //
        // fnum_textbox
        //
        this->fnum_textbox->Location = System::Drawing::Point(203, 68);
        this->fnum_textbox->Name = L"fnum_textbox";
        this->fnum_textbox->Size = System::Drawing::Size(100, 20);
        this->fnum_textbox->TabIndex = 22;
        //
        // name_textbox
        //
        this->name_textbox->Location = System::Drawing::Point(203, 94);
        this->name_textbox->Name = L"name_textbox";
        this->name_textbox->Size = System::Drawing::Size(100, 20);
        this->name_textbox->TabIndex = 23;
        //
        // dest_textbox
        //

```

```

this->dest_textbox->Location = System::Drawing::Point(203, 120);
this->dest_textbox->Name = L"dest_textbox";
this->dest_textbox->Size = System::Drawing::Size(100, 20);
this->dest_textbox->TabIndex = 24;
//
// price_textbox
//
this->price_textbox->Location = System::Drawing::Point(203,
400);

this->price_textbox->Name = L"price_textbox";
this->price_textbox->Size = System::Drawing::Size(100, 20);
this->price_textbox->TabIndex = 25;
//
// error_label
//
this->error_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->error_label->ForeColor = System::Drawing::Color::Red;
this->error_label->Location = System::Drawing::Point(17, 432);
this->error_label->Name = L"error_label";
this->error_label->Size = System::Drawing::Size(286, 37);
this->error_label->TabIndex = 26;
this->error_label->TextAlign =
System::Drawing::ContentAlignment::MiddleCenter;
//
// fnum_checkbox
//
this->fnum_checkbox->AutoSize = true;
this->fnum_checkbox->Location = System::Drawing::Point(11, 71);
this->fnum_checkbox->Name = L"fnum_checkbox";
this->fnum_checkbox->Size = System::Drawing::Size(15, 14);
this->fnum_checkbox->TabIndex = 27;
this->fnum_checkbox->UseVisualStyleBackColor = true;
//
// name_checkbox
//
this->name_checkbox->AutoSize = true;
this->name_checkbox->Location = System::Drawing::Point(11, 97);
this->name_checkbox->Name = L"name_checkbox";
this->name_checkbox->Size = System::Drawing::Size(15, 14);
this->name_checkbox->TabIndex = 28;
this->name_checkbox->UseVisualStyleBackColor = true;
//
// dest_checkbox
//
this->dest_checkbox->AutoSize = true;
this->dest_checkbox->Location = System::Drawing::Point(11, 123);
this->dest_checkbox->Name = L"dest_checkbox";
this->dest_checkbox->Size = System::Drawing::Size(15, 14);
this->dest_checkbox->TabIndex = 29;
this->dest_checkbox->UseVisualStyleBackColor = true;
//
// days_checkbox
//
this->days_checkbox->AutoSize = true;
this->days_checkbox->Location = System::Drawing::Point(11, 154);
this->days_checkbox->Name = L"days_checkbox";
this->days_checkbox->Size = System::Drawing::Size(15, 14);
this->days_checkbox->TabIndex = 30;
this->days_checkbox->UseVisualStyleBackColor = true;
//

```

```

// dep_time_checkbox
//
this->dep_time_checkbox->AutoSize = true;
this->dep_time_checkbox->Location = System::Drawing::Point(11,
350);

this->dep_time_checkbox->Name = L"dep_time_checkbox";
this->dep_time_checkbox->Size = System::Drawing::Size(15, 14);
this->dep_time_checkbox->TabIndex = 31;
this->dep_time_checkbox->UseVisualStyleBackColor = true;
//
// arr_time_checkbox
//
this->arr_time_checkbox->AutoSize = true;
this->arr_time_checkbox->Location = System::Drawing::Point(11,
377);

this->arr_time_checkbox->Name = L"arr_time_checkbox";
this->arr_time_checkbox->Size = System::Drawing::Size(15, 14);
this->arr_time_checkbox->TabIndex = 32;
this->arr_time_checkbox->UseVisualStyleBackColor = true;
//
// price_checkbox
//
this->price_checkbox->AutoSize = true;
this->price_checkbox->Location = System::Drawing::Point(11,
403);

this->price_checkbox->Name = L"price_checkbox";
this->price_checkbox->Size = System::Drawing::Size(15, 14);
this->price_checkbox->TabIndex = 33;
this->price_checkbox->UseVisualStyleBackColor = true;
//
// check_label
//
this->check_label->AutoSize = true;
this->check_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->check_label->Location = System::Drawing::Point(8, 41);
this->check_label->Name = L"check_label";
this->check_label->Size = System::Drawing::Size(184, 17);
this->check_label->TabIndex = 34;
this->check_label->Text = L"Выберите нужные поля";
//
// from_array_radio
//
this->from_array_radio->AutoSize = true;
this->from_array_radio->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 9,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
static_cast<System::Byte>(204)));
this->from_array_radio->Location = System::Drawing::Point(11,
533);

this->from_array_radio->Name = L"from_array_radio";
this->from_array_radio->Size = System::Drawing::Size(169, 19);
this->from_array_radio->TabIndex = 35;
this->from_array_radio->Text = L"По выбранным номерам";
this->from_array_radio->UseVisualStyleBackColor = true;
//
// filtered_radio
//
this->filtered_radio->AutoSize = true;
this->filtered_radio->Checked = true;

```

```

        this->filtered_radio->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 9,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->filtered_radio->Location = System::Drawing::Point(11,
508);

        this->filtered_radio->Name = L"filtered_radio";
        this->filtered_radio->Size = System::Drawing::Size(193, 19);
        this->filtered_radio->TabIndex = 36;
        this->filtered_radio->TabStop = true;
        this->filtered_radio->Text = L"Все подходящие под фильтр";
        this->filtered_radio->UseVisualStyleBackColor = true;
        //
        // apply_label
        //
        this->apply_label->AutoSize = true;
        this->apply_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->apply_label->Location = System::Drawing::Point(12, 479);
        this->apply_label->Name = L"apply_label";
        this->apply_label->Size = System::Drawing::Size(235, 17);
        this->apply_label->TabIndex = 37;
        this->apply_label->Text = L"Какие записи нужно изменить";
        //
        // to_edit_textbox
        //
        this->to_edit_textbox->Location = System::Drawing::Point(8,
558);

        this->to_edit_textbox->Name = L"to_edit_textbox";
        this->to_edit_textbox->Size = System::Drawing::Size(308, 20);
        this->to_edit_textbox->TabIndex = 38;
        //
        // EditFlights
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(323, 640);
        this->Controls->Add(this->to_edit_textbox);
        this->Controls->Add(this->apply_label);
        this->Controls->Add(this->filtered_radio);
        this->Controls->Add(this->from_array_radio);
        this->Controls->Add(this->check_label);
        this->Controls->Add(this->price_checkbox);
        this->Controls->Add(this->arr_time_checkbox);
        this->Controls->Add(this->dep_time_checkbox);
        this->Controls->Add(this->days_checkbox);
        this->Controls->Add(this->dest_checkbox);
        this->Controls->Add(this->name_checkbox);
        this->Controls->Add(this->fnum_checkbox);
        this->Controls->Add(this->error_label);
        this->Controls->Add(this->price_textbox);
        this->Controls->Add(this->dest_textbox);
        this->Controls->Add(this->name_textbox);
        this->Controls->Add(this->fnum_textbox);
        this->Controls->Add(this->edit_button);
        this->Controls->Add(this->price_label);
        this->Controls->Add(this->arr_time_textbox);
        this->Controls->Add(this->arr_time_label);

```

```

        this->Controls->Add(this->dep_time_textbox);
        this->Controls->Add(this->dep_time_label);
        this->Controls->Add(this->checkbox_7);
        this->Controls->Add(this->checkbox_6);
        this->Controls->Add(this->checkbox_5);
        this->Controls->Add(this->checkbox_4);
        this->Controls->Add(this->checkbox_3);
        this->Controls->Add(this->checkbox_2);
        this->Controls->Add(this->checkbox_1);
        this->Controls->Add(this->days_label);
        this->Controls->Add(this->dest_label);
        this->Controls->Add(this->name_label);
        this->Controls->Add(this->fnum_label);
        this->Controls->Add(this->editing_label);
        this->MaximizeBox = false;
        this->MinimizeBox = false;
        this->Name = L"EditFlights";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"Изменение рейсов";
        this->ResumeLayout(false);
        this->PerformLayout();
    }
#pragma endregion
    private: System::Void edit_button_Click(System::Object^ sender,
System::EventArgs^ e) {
        Flight_filter flight_exapmle; // Структура для хранения информации
        // Запишем все поля, на которые нужно применить изменение
        for (int i = 0; i < FIELDS_NUM; i++) flight_exapmle.apply[i] = 0;
        if (this->fnum_checkbox->Checked) flight_exapmle.apply[0] =
1;
        if (this->name_checkbox->Checked) flight_exapmle.apply[1] =
1;
        if (this->dest_checkbox->Checked) flight_exapmle.apply[2] =
1;
        if (this->days_checkbox->Checked) flight_exapmle.apply[3] =
1;
        if (this->dep_time_checkbox->Checked) flight_exapmle.apply[4] = 1;
        if (this->arr_time_checkbox->Checked) flight_exapmle.apply[5] = 1;
        if (this->price_checkbox->Checked) flight_exapmle.apply[6] =
1;
        if (flight_exapmle.apply[0] + flight_exapmle.apply[1] +
flight_exapmle.apply[2]
        + flight_exapmle.apply[3] + flight_exapmle.apply[4]
        + flight_exapmle.apply[5] + flight_exapmle.apply[6] == 0) {
            this->error_label->Text = "Нужно выбрать хотя бы одно поле";
            return;
        }
        // Получаем и проверяем fnum
        if (flight_exapmle.apply[0]) {
            try {
                flight_exapmle.fnum = Convert::ToInt32(this->fnum_textbox-
>Text);
            }
            catch (...) {
                this->error_label->Text = L"Некорректный номер рейса";
                return;
            }
        }
        if (flight_exapmle.fnum <= 0) {
            this->error_label->Text = L"Некорректный номер рейса";
            return;
        }
    }

```

```

    }

    // Получаем и проверяем name
    if (flight_exapmle.apply[1]) {
        copy_char_from_string(flight_exapmle.name, TEXT_LEN, this-
>name_textbox->Text);
        if (!flight_exapmle.name[0]) {
            this->error_label->Text = L"Некорректный тип самолёта";
            return;
        }
    }

    // Получаем и проверяем dest
    if (flight_exapmle.apply[2]) {
        copy_char_from_string(flight_exapmle.dest, TEXT_LEN, this-
>dest_textbox->Text);
        if (!flight_exapmle.dest[0]) {
            this->error_label->Text = L"Некорректный пункт
назначения";
            return;
        }
    }

    // Получаем и проверяем days
    if (flight_exapmle.apply[3]) {
        int day = 0;
        if (this->checkbox_1->Checked) flight_exapmle.days[day++] = 1;
        if (this->checkbox_2->Checked) flight_exapmle.days[day++] = 2;
        if (this->checkbox_3->Checked) flight_exapmle.days[day++] = 3;
        if (this->checkbox_4->Checked) flight_exapmle.days[day++] = 4;
        if (this->checkbox_5->Checked) flight_exapmle.days[day++] = 5;
        if (this->checkbox_6->Checked) flight_exapmle.days[day++] = 6;
        if (this->checkbox_7->Checked) flight_exapmle.days[day++] = 7;
        if (day == 0) {
            this->error_label->Text = L"Нужно выбрать хотя бы один
день отправления";
            return;
        }
        for (int i = day; i < 7; i++) flight_exapmle.days[i] = 0;
    }

    // Получаем и проверяем dep_time
    if (flight_exapmle.apply[4]) {
        int dep_time_try = convert_string_time(this->dep_time_textbox-
>Text);
        if (dep_time_try == -1){
            this->error_label->Text = L"Некорректное время вылета";
            return;
        }
        flight_exapmle.dep_time = dep_time_try;
    }

    // Получаем и проверяем arr_time
    if (flight_exapmle.apply[5]) {
        int arr_time_try = convert_string_time(this->arr_time_textbox-
>Text);
        if (arr_time_try == -1) {
            this->error_label->Text = L"Некорректное время прилёта";
            return;
        }
        flight_exapmle.arr_time = arr_time_try;
    }

    // Получаем и проверяем price
    if (flight_exapmle.apply[6]) {
        try {

```

```

        flight_exapmle.price = Convert::ToDouble(this-
>price_textbox->Text->Replace(".", ","));
    }
    catch (...) {
        this->error_label->Text = L"Некорректная цена билета";
        return;
    }
    if (flight_exapmle.price <= 0) {
        this->error_label->Text = L"Некорректная цена билета";
        return;
    }
}
int* to_edit; // Указатель на массив с номерами записей,
которые нужно поменять
int last_ind = 0; // Индекс последнего эл-та массива
// Получим номера для изменения
if (this->from_array_radio->Checked) {
    String^ numbers = this->to_edit_textbox->Text;
    // Разделители: пробел, запятая, точка с запятой, табуляция
    array<wchar_t>^ delimiters = { L' ', L',', L';', L'\t' };
    array<String^>^ parts = numbers->Split(delimiters,
StringSplitOptions::RemoveEmptyEntries);
    if (numbers->Length == 0 || parts->Length == 0) {
        this->error_label->Text = "Некорректно указаны номера";
        return;
    }
    // Выделяем память
    to_edit = (int*)calloc(parts->Length + 1, sizeof(int));
    // Конвертируем все part в int
    for each (String ^ part in parts) {
        try {
            int num = Convert::ToInt32(part);
            to_edit[last_ind++] = num;
        }
        catch (...) {
            free(to_edit);
            this->error_label->Text = "Некорректно указаны
номера";
            return;
        }
    }
    // Сортируем пузырьком полученный массив
    for (int k = 1; k < last_ind; k++)
        for (int i = 0; i < last_ind - k; i++)
            if (to_edit[i] > to_edit[i + 1]) {
                int buffer = to_edit[i];
                to_edit[i] = to_edit[i + 1];
                to_edit[i + 1] = buffer;
            }
    to_edit[last_ind] = 0;
}
this->error_label->Text = "";
edit_with_example(flight_exapmle, this->filtered_radio->Checked,
to_edit, last_ind);
if (this->from_array_radio->Checked) free(to_edit);
this->Close();
}
};
}

```

П.Б.13 Код DeleteFlights.h

```
#pragma once
```



```

#include <msclr/marshal.h>
#include <windows.h>
#include <stdlib.h>

#include "structure.h"
#include "file_funcs.h"

namespace MarkflightsUI {
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;

    /// <summary>
    /// Сводка для DeleteFlights
    /// </summary>
    public ref class DeleteFlights : public System::Windows::Forms::Form
    {
    public:
        DeleteFlights(void)
        {
            InitializeComponent();
        }

    protected:
        /// <summary>
        /// Освободить все используемые ресурсы.
        /// </summary>
        ~DeleteFlights()
        {
            if (components)
            {
                delete components;
            }
        }
    private: System::Windows::Forms::Button^ del_button;
    private: System::Windows::Forms::RadioButton^ from_array_radio;
    private: System::Windows::Forms::RadioButton^ filtered_radio;
    private: System::Windows::Forms::Label^ apply_label;
    private: System::Windows::Forms::TextBox^ to_del_textbox;
    private: System::Windows::Forms::Label^ deleting_label;
    private: System::Windows::Forms::Label^ error_label;
    private:
        /// <summary>
        /// Обязательная переменная конструктора.
        /// </summary>
        System::ComponentModel::Container^ components;

#pragma region Windows Form Designer generated code
        /// <summary>
        /// Требуемый метод для поддержки конструктора – не изменяйте
        /// содержимое этого метода с помощью редактора кода.
        /// </summary>
        void InitializeComponent(void)
        {
            this->del_button = (gcnew System::Windows::Forms::Button());
            this->from_array_radio = (gcnew
System::Windows::Forms::RadioButton());
            this->filtered_radio = (gcnew
System::Windows::Forms::RadioButton());
            this->apply_label = (gcnew System::Windows::Forms::Label());

```

```

        this->to_del_textbox = (gcnew
System::Windows::Forms::TextBox());
        this->deleting_label = (gcnew System::Windows::Forms::Label());
        this->error_label = (gcnew System::Windows::Forms::Label());
        this->SuspendLayout();
        //
        // del_button
        //
        this->del_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->del_button->Location = System::Drawing::Point(109, 183);
        this->del_button->Name = L"del_button";
        this->del_button->Size = System::Drawing::Size(118, 44);
        this->del_button->TabIndex = 21;
        this->del_button->Text = L"Применить";
        this->del_button->UseVisualStyleBackColor = true;
        this->del_button->Click += gcnew System::EventHandler(this,
&DeleteFlights::edit_button_Click);
        //
        // from_array_radio
        //
        this->from_array_radio->AutoSize = true;
        this->from_array_radio->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->from_array_radio->Location = System::Drawing::Point(11,
86);
        this->from_array_radio->Name = L"from_array_radio";
        this->from_array_radio->Size = System::Drawing::Size(186, 21);
        this->from_array_radio->TabIndex = 35;
        this->from_array_radio->Text = L"По выбранным номерам";
        this->from_array_radio->UseVisualStyleBackColor = true;
        //
        // filtered_radio
        //
        this->filtered_radio->AutoSize = true;
        this->filtered_radio->Checked = true;
        this->filtered_radio->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->filtered_radio->Location = System::Drawing::Point(11, 59);
        this->filtered_radio->Name = L"filtered_radio";
        this->filtered_radio->Size = System::Drawing::Size(216, 21);
        this->filtered_radio->TabIndex = 36;
        this->filtered_radio->TabStop = true;
        this->filtered_radio->Text = L"Все подходящие под фильтр";
        this->filtered_radio->UseVisualStyleBackColor = true;
        //
        // apply_label
        //
        this->apply_label->AutoSize = true;
        this->apply_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->apply_label->Location = System::Drawing::Point(8, 39);
        this->apply_label->Name = L"apply_label";
        this->apply_label->Size = System::Drawing::Size(225, 17);
        this->apply_label->TabIndex = 37;

```

```

        this->apply_label->Text = L"Какие записи нужно удалить";
        //
        // to_del_textbox
        //
        this->to_del_textbox->Location = System::Drawing::Point(11,
120);

        this->to_del_textbox->Name = L"to_del_textbox";
        this->to_del_textbox->Size = System::Drawing::Size(308, 20);
        this->to_del_textbox->TabIndex = 38;
        //
        // deleting_label
        //
        this->deleting_label->AutoSize = true;
        this->deleting_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->deleting_label->Location = System::Drawing::Point(90, 9);
        this->deleting_label->Name = L"deleting_label";
        this->deleting_label->Size = System::Drawing::Size(157, 20);
        this->deleting_label->TabIndex = 0;
        this->deleting_label->Text = L"Удаление рейсов";
        //
        // error_label
        //
        this->error_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->error_label->ForeColor = System::Drawing::Color::Red;
        this->error_label->Location = System::Drawing::Point(12, 143);
        this->error_label->Name = L"error_label";
        this->error_label->Size = System::Drawing::Size(307, 37);
        this->error_label->TabIndex = 39;
        this->error_label->TextAlign =
System::Drawing::ContentAlignment::MiddleCenter;
        //
        // DeleteFlights
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(323, 238);
        this->Controls->Add(this->error_label);
        this->Controls->Add(this->to_del_textbox);
        this->Controls->Add(this->apply_label);
        this->Controls->Add(this->filtered_radio);
        this->Controls->Add(this->from_array_radio);
        this->Controls->Add(this->del_button);
        this->Controls->Add(this->deleting_label);
        this->MaximizeBox = false;
        this->MinimizeBox = false;
        this->Name = L"DeleteFlights";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"Удаление рейсов";
        this->ResumeLayout(false);
        this->PerformLayout();

    }
#pragma endregion

```

```

        private: System::Void edit_button_Click(System::Object^ sender,
System::EventArgs^ e) {
            int* to_del;          // Указатель на массив с номерами записей, которые
нужно удалить
            int last_ind = 0;    // Индекс последнего эл-та массива

            // Получим номера для изменения
            if (this->from_array_radio->Checked) {
                String^ numbers = this->to_del_textbox->Text;
                // Разделители: пробел, запятая, точка с запятой, табуляция
                array<wchar_t>^ delimiters = { L' ', L',', L';', L'\t' };
                array<String^>^ parts = numbers->Split(delimiters,
StringSplitOptions::RemoveEmptyEntries);
                if (numbers->Length == 0 || parts->Length == 0) {
                    this->error_label->Text = "Некорректно указаны номера";
                    return;
                }
                // Выделяем память под to_del
                to_del = (int*)calloc(parts->Length + 1, sizeof(int));
                // Переводим все part в int
                for each (String ^ part in parts) {
                    try {
                        int num = Convert::ToInt32(part);
                        to_del[last_ind++] = num;
                    }
                    catch (...) {
                        free(to_del);
                        this->error_label->Text = "Некорректно указаны
номера";
                        return;
                    }
                }
                // Сортируем пузырьком полученный массив
                for (int k = 1; k < last_ind; k++)
                    for (int i = 0; i < last_ind - k; i++)
                        if (to_del[i] > to_del[i + 1]) {
                            int buffer = to_del[i];
                            to_del[i] = to_del[i + 1];
                            to_del[i + 1] = buffer;
                        }
                to_del[last_ind] = 0;
            }
            this->error_label->Text = "";
            delete_flights(this->filtered_radio->Checked, to_del, last_ind);
            if (this->from_array_radio->Checked) free(to_del);
            this->Close();
        }
    };
}

```

П.Б.14 Код ChangePath.h

```

#pragma once

#include <msclr/marshal.h>
#include "file_funcs.h"
#include <windows.h>

namespace MarkflightsUI {
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;

```

```

using namespace System::Drawing;

/// <summary>
/// Сводка для ChangePath
/// </summary>
public ref class ChangePath : public System::Windows::Forms::Form
{
public:
    ChangePath(void)
    {
        InitializeComponent();
        // Получаем путь до файла
        char file_path[FILE_NAME_LEN];
        if (get_file(file_path) == 1) {
            MessageBox::Show(L"Слишком длинный путь, запись произойдёт
по пути " DIR_NAME "/" FILE_NAME);
            return;
        }
        this->cur_path_textbox->Text = gcnew String(file_path);
    }

protected:
    /// <summary>
    /// Освободить все используемые ресурсы.
    /// </summary>
    ~ChangePath()
    {
        if (components)
        {
            delete components;
        }
    }

private: System::Windows::Forms::Button^ choose_path_button;
private: System::Windows::Forms::Label^ changing_label;
private: System::Windows::Forms::RadioButton^ new_file_radio;
private: System::Windows::Forms::RadioButton^ exist_file_radio;
private: System::Windows::Forms::Label^ cur_path_label;
private: System::Windows::Forms::Label^ new_path_label;
private: System::Windows::Forms::TextBox^ cur_path_textbox;
private: System::Windows::Forms::TextBox^ new_path_textbox;
private: System::Windows::Forms::CheckBox^ copy_file_checkbox;
private: System::Windows::Forms::CheckBox^ delete_file_checkbox;
private: System::Windows::Forms::Button^ save_button;
private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container ^components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        this->choose_path_button = (gcnew
System::Windows::Forms::Button());
        this->changing_label = (gcnew System::Windows::Forms::Label());
        this->new_file_radio = (gcnew
System::Windows::Forms::RadioButton());
        this->exist_file_radio = (gcnew
System::Windows::Forms::RadioButton());
    }

```

```

        this->cur_path_label = (gcnew System::Windows::Forms::Label());
        this->new_path_label = (gcnew System::Windows::Forms::Label());
        this->cur_path_textbox = (gcnew
System::Windows::Forms::TextBox());
        this->new_path_textbox = (gcnew
System::Windows::Forms::TextBox());
        this->copy_file_checkbox = (gcnew
System::Windows::Forms::CheckBox());
        this->delete_file_checkbox = (gcnew
System::Windows::Forms::CheckBox());
        this->save_button = (gcnew System::Windows::Forms::Button());
        this->SuspendLayout();
        //
        // choose_path_button
        //
        this->choose_path_button->Location = System::Drawing::Point(200,
90);

        this->choose_path_button->Name = L"choose_path_button";
        this->choose_path_button->Size = System::Drawing::Size(75, 44);
        this->choose_path_button->TabIndex = 0;
        this->choose_path_button->Text = L"Создать";
        this->choose_path_button->UseVisualStyleBackColor = true;
        this->choose_path_button->Click += gcnew
System::EventHandler(this, &ChangePath::choose_path_button_Click);
        //
        // changing_label
        //
        this->changing_label->AutoSize = true;
        this->changing_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->changing_label->Location = System::Drawing::Point(30, 9);
        this->changing_label->Name = L"changing_label";
        this->changing_label->Size = System::Drawing::Size(234, 20);
        this->changing_label->TabIndex = 1;
        this->changing_label->Text = L"Изменение пути до файла";
        //
        // new_file_radio
        //
        this->new_file_radio->AutoSize = true;
        this->new_file_radio->Checked = true;
        this->new_file_radio->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 9,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->new_file_radio->Location = System::Drawing::Point(19, 90);
        this->new_file_radio->Name = L"new_file_radio";
        this->new_file_radio->Size = System::Drawing::Size(149, 19);
        this->new_file_radio->TabIndex = 2;
        this->new_file_radio->TabStop = true;
        this->new_file_radio->Text = L"Создать новый файл";
        this->new_file_radio->UseVisualStyleBackColor = true;
        this->new_file_radio->Click += gcnew System::EventHandler(this,
&ChangePath::radio_button_Click);
        //
        // exist_file_radio
        //
        this->exist_file_radio->AutoSize = true;
        this->exist_file_radio->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 9,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));

```

```

115);
    this->exist_file_radio->Location = System::Drawing::Point(19,
    this->exist_file_radio->Name = L"exist_file_radio";
    this->exist_file_radio->Size = System::Drawing::Size(164, 19);
    this->exist_file_radio->TabIndex = 3;
    this->exist_file_radio->Text = L"Выбрать существующий";
    this->exist_file_radio->UseVisualStyleBackColor = true;
    this->exist_file_radio->Click += gcnew
System::EventHandler(this, &ChangePath::radio_button_Click);
    //
    // cur_path_label
    //
    this->cur_path_label->AutoSize = true;
    this->cur_path_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 9,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
    static_cast<System::Byte>(204)));
    this->cur_path_label->Location = System::Drawing::Point(12, 39);
    this->cur_path_label->Name = L"cur_path_label";
    this->cur_path_label->Size = System::Drawing::Size(146, 15);
    this->cur_path_label->TabIndex = 4;
    this->cur_path_label->Text = L"Текущий путь до файла:";
    //
    // new_path_label
    //
    this->new_path_label->AutoSize = true;
    this->new_path_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 9,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
    static_cast<System::Byte>(204)));
    this->new_path_label->Location = System::Drawing::Point(12,
146);
    this->new_path_label->Name = L"new_path_label";
    this->new_path_label->Size = System::Drawing::Size(169, 15);
    this->new_path_label->TabIndex = 5;
    this->new_path_label->Text = L"Полученный путь до файла:";
    //
    // cur_path_textbox
    //
    this->cur_path_textbox->Location = System::Drawing::Point(12,
59);
    this->cur_path_textbox->Name = L"cur_path_textbox";
    this->cur_path_textbox->ReadOnly = true;
    this->cur_path_textbox->Size = System::Drawing::Size(276, 20);
    this->cur_path_textbox->TabIndex = 6;
    //
    // new_path_textbox
    //
    this->new_path_textbox->Location = System::Drawing::Point(12,
164);
    this->new_path_textbox->Name = L"new_path_textbox";
    this->new_path_textbox->ReadOnly = true;
    this->new_path_textbox->Size = System::Drawing::Size(276, 20);
    this->new_path_textbox->TabIndex = 7;
    //
    // copy_file_checkbox
    //
    this->copy_file_checkbox->AutoSize = true;
    this->copy_file_checkbox->Location = System::Drawing::Point(12,
200);
    this->copy_file_checkbox->Name = L"copy_file_checkbox";
    this->copy_file_checkbox->Size = System::Drawing::Size(278, 17);
    this->copy_file_checkbox->TabIndex = 8;

```

```

        this->copy_file_checkbox->Text = L"Скопировать данные из
текущего файла в новый";
        this->copy_file_checkbox->UseVisualStyleBackColor = true;
        //
        // delete_file_checkbox
        //
        this->delete_file_checkbox->AutoSize = true;
        this->delete_file_checkbox->Location =
System::Drawing::Point(12, 223);
        this->delete_file_checkbox->Name = L"delete_file_checkbox";
        this->delete_file_checkbox->Size = System::Drawing::Size(144,
17);

        this->delete_file_checkbox->TabIndex = 9;
        this->delete_file_checkbox->Text = L"Удалить текущий файл";
        this->delete_file_checkbox->UseVisualStyleBackColor = true;
        //
        // save_button
        //
        this->save_button->Location = System::Drawing::Point(107, 246);
        this->save_button->Name = L"save_button";
        this->save_button->Size = System::Drawing::Size(92, 39);
        this->save_button->TabIndex = 10;
        this->save_button->Text = L"Сохранить";
        this->save_button->UseVisualStyleBackColor = true;
        this->save_button->Click += gcnew System::EventHandler(this,
&ChangePath::save_button_Click);
        //
        // ChangePath
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(300, 295);
        this->Controls->Add(this->save_button);
        this->Controls->Add(this->delete_file_checkbox);
        this->Controls->Add(this->copy_file_checkbox);
        this->Controls->Add(this->new_path_textbox);
        this->Controls->Add(this->cur_path_textbox);
        this->Controls->Add(this->new_path_label);
        this->Controls->Add(this->cur_path_label);
        this->Controls->Add(this->exist_file_radio);
        this->Controls->Add(this->new_file_radio);
        this->Controls->Add(this->changing_label);
        this->Controls->Add(this->choose_path_button);
        this->MaximizeBox = false;
        this->MinimizeBox = false;
        this->Name = L"ChangePath";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterParent;
        this->Text = L"Путь до файла";
        this->ResumeLayout(false);
        this->PerformLayout();

    }
#pragma endregion
private: System::Void radio_button_Click(System::Object^ sender, System::EventArgs^
e) {
    // Меняем текст на кнопке в зависимости от radiobutton-ов
    if (this->new_file_radio->Checked) this->choose_path_button->Text =
L"Создать";
    else this->choose_path_button->Text = L"Выбрать";
}

```



```

        this->new_path_textbox->Text = "";
    }
private: System::Void choose_path_button_Click(System::Object^ sender,
System::EventArgs^ e) {
    String^ base_path = Application::StartupPath; // Путь до .exe
    try {
        if (this->new_file_radio->Checked) {
            // Создать новый файл - используем SaveFileDialog
            SaveFileDialog^ saveDialog = gcnew SaveFileDialog();
            saveDialog->Title = L"Создать новый файл";
            saveDialog->Filter = L"CSV файлы (*.csv)|*.csv";
            saveDialog->DefaultExt = L"csv";
            saveDialog->AddExtension = true;
            if (saveDialog->ShowDialog(this) ==
System::Windows::Forms::DialogResult::OK) {
                this->new_path_textbox->Text = saveDialog->FileName-
>Replace(base_path + "\\ ", "");
            }
        }
        else {
            // Выбрать существующий файл - используем OpenFileDialog
            OpenFileDialog^ openDialog = gcnew OpenFileDialog();
            openDialog->Title = L"Выберите существующий CSV файл";
            openDialog->Filter = L"CSV файлы (*.csv)|*.csv";
            if (openDialog->ShowDialog(this) ==
System::Windows::Forms::DialogResult::OK) {
                this->new_path_textbox->Text = openDialog->FileName-
>Replace(base_path + "\\ ", "");
            }
        }
    }
    catch (Exception^ ex) {
        MessageBox::Show(this, L"Ошибка: " + ex->Message);
    }
}
private: System::Void save_button_Click(System::Object^ sender, System::EventArgs^
e) {
    if (this->new_path_textbox->Text == "") {
        MessageBox::Show(L"Нужно выбрать файл");
        return;
    }
    FILE* settings = fopen(DIR_NAME "/" CONF_NAME, "a+");
    if (!settings) {
        MessageBox::Show(L"Нет прав для изменения пути");
        return;
    }
    rewind(settings);

    // Получим путь до файла
    char new_file_path[FILE_NAME_LEN];
    copy_char_from_string(new_file_path, FILE_NAME_LEN, this->new_path_textbox-
>Text);

    // Создадим файл, если его нет
    if (this->new_file_radio->Checked) {
        FILE* new_file = fopen(new_file_path, "w");
        if (new_file == NULL) {
            MessageBox::Show(L"Нет прав для создания файла");
            return;
        }
        fclose(new_file);
    }
}

```

```

        // Заменяем его в файле
        char password[PASSWORD_LEN];
        char old_file_path[FILE_NAME_LEN];
        if (fgets(password, PASSWORD_LEN, settings) == NULL) { // Если файл пустой
            strcpy(password, "root");
            strcpy(old_file_path, DIR_NAME "/" FILE_NAME);
            fclose(settings);
        }
        else if (fgets(old_file_path, FILE_NAME_LEN, settings) == NULL) { // Если в
файле есть только пароль
            strcpy(old_file_path, DIR_NAME "/" FILE_NAME);
            fclose(settings);
        }
        else fclose(settings);
        settings = fopen(DIR_NAME "/" CONF_NAME, "w");
        // Сохраняем новый путь в файл
        fprintf(settings, "%s%s", password, new_file_path);
        fclose(settings);

        // Если нужно скопировать данные из старого файла в новый
        if (this->copy_file_checkbox->Checked) {
            FILE* new_file = fopen(new_file_path, "a");
            FILE* old_file = fopen(old_file_path, "r");
            if (old_file == NULL || new_file == NULL) {
                MessageBox::Show(L"Не получилось скопировать данные из файла"
+ gcnew String(old_file_path) + L" в файл " + gcnew
String(new_file_path));
            }
            else {
                char read_buffer[TEXT_LEN * FIELDS_NUM];
                while (fgets(read_buffer, TEXT_LEN * FIELDS_NUM, old_file) !=
NULL) {
                    fputs(read_buffer, new_file);
                }
            }
            if (old_file != NULL) fclose(old_file);
            if (new_file != NULL) fclose(new_file);
        }
        // Если нужно удалить старый файл
        if (this->delete_file_checkbox->Checked) {
            remove(old_file_path);
        }
        this->Close();
    }
};
}

```

П.Б.15 Код UserSite.h

```

#pragma once

#include "AdminSite.h"

namespace MarkflightsUI {
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;

    /// <summary>
    /// Сводка для UserSite
    /// </summary>

```

```

//public ref class UserSite : public System::Windows::Forms::Form
public ref class UserSite : public AdminSite // Наследуем от AdminSite
{
public:
    UserSite(void)
    {
        // Выполняется код конструктора AdminSite, который автоматически
заполняет таблицу
        InitializeComponent();
    }
protected:
    /// <summary>
    /// Освободить все используемые ресурсы.
    /// </summary>
    ~UserSite()
    {
        if (components)
        {
            delete components;
        }
    }
private: System::Windows::Forms::DataGridView^ data_base;
private: System::Windows::Forms::Label^ mode_label;
private: System::Windows::Forms::Button^ update_button;
private: System::Windows::Forms::Button^ set_filters_button;
private: System::Windows::Forms::Button^ exit_button;
private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container^ components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        this->data_base = (gcnew
System::Windows::Forms::DataGridView());
        this->mode_label = (gcnew System::Windows::Forms::Label());
        this->update_button = (gcnew System::Windows::Forms::Button());
        this->set_filters_button = (gcnew
System::Windows::Forms::Button());
        this->exit_button = (gcnew System::Windows::Forms::Button());

        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>data_base))->BeginInit();
        this->SuspendLayout();
        //
        // data_base
        //
        this->data_base->AllowUserToAddRows = false;
        this->data_base->AllowUserToDeleteRows = false;
        this->data_base->AllowUserToResizeColumns = false;
        this->data_base->AllowUserToResizeRows = false;
        this->data_base->BackColor =
System::Drawing::SystemColors::ButtonHighlight;
        this->data_base->ColumnHeadersHeightSizeMode =
System::Windows::Forms::DataGridViewColumnHeadersHeightSizeMode::AutoSize;
        this->data_base->Location = System::Drawing::Point(12, 12);
        this->data_base->Name = L"data_base";
    }

```

```

        this->data_base->ReadOnly = true;
        this->data_base->RowHeadersVisible = false;
        this->data_base->Size = System::Drawing::Size(803, 500);
        this->data_base->TabIndex = 1;
        //
        // mode_label
        //
        this->mode_label->AutoSize = true;
        this->mode_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->mode_label->Location = System::Drawing::Point(26, 530);
        this->mode_label->Name = L"mode_label";
        this->mode_label->Size = System::Drawing::Size(190, 20);
        this->mode_label->TabIndex = 2;
        this->mode_label->Text = L"Режим пользователя";
        //
        // update_button
        //
        this->update_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->update_button->Location = System::Drawing::Point(235,
525);
        this->update_button->Name = L"update_button";
        this->update_button->Size = System::Drawing::Size(186, 33);
        this->update_button->TabIndex = 9;
        this->update_button->Text = L"Обновить таблицу";
        this->update_button->UseVisualStyleBackColor = true;
        this->update_button->Click += gcnew System::EventHandler(this,
&UserSite::update_button_Click);
        //
        // set_filters_button
        //
        this->set_filters_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->set_filters_button->Location = System::Drawing::Point(427,
525);
        this->set_filters_button->Name = L"set_filters_button";
        this->set_filters_button->Size = System::Drawing::Size(186, 33);
        this->set_filters_button->TabIndex = 10;
        this->set_filters_button->Text = L"Настроить фильтры";
        this->set_filters_button->UseVisualStyleBackColor = true;
        this->set_filters_button->Click += gcnew
System::EventHandler(this, &UserSite::set_filters_button_Click);
        //
        // exit_button
        //
        this->exit_button->BackColor =
System::Drawing::Color::FromArgb(static_cast<System::Int32>(static_cast<System::Byte>
(255)), static_cast<System::Int32>(static_cast<System::Byte>(192)),
        static_cast<System::Int32>(static_cast<System::Byte>(192)));
        this->exit_button->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 10,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->exit_button->Location = System::Drawing::Point(619, 525);
        this->exit_button->Name = L"exit_button";

```

```

        this->exit_button->Size = System::Drawing::Size(186, 33);
        this->exit_button->TabIndex = 11;
        this->exit_button->Text = L"Выйти в главное меню";
        this->exit_button->UseVisualStyleBackColor = false;
        this->exit_button->Click += gcnew System::EventHandler(this,
&UserSite::exit_button_Click);
        //
        // UserSite
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(831, 586);
        this->Controls->Add(this->exit_button);
        this->Controls->Add(this->set_filters_button);
        this->Controls->Add(this->update_button);
        this->Controls->Add(this->mode_label);
        this->Controls->Add(this->data_base);
        this->MaximizeBox = false;
        this->Name = L"UserSite";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"Режим пользователя";

        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>data_base))->EndInit();
        this->ResumeLayout(false);
        this->PerformLayout();

    }
#pragma endregion
};
}

```

П.Б.16 Код StartGame.h

```

#pragma once

#include "TheZondbie.h"

namespace MarkflightsUI {
    using namespace System;
    using namespace System::IO;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;

    /// <summary>
    /// Сводка для StartGame
    /// </summary>
    public ref class StartGame : public System::Windows::Forms::Form
    {
    public:
        StartGame(void)
        {
            InitializeComponent();
            try {

```

```

        if (File::Exists("game_data/score.txt")) {
            StreamReader^ reader = gcnew
StreamReader("game_data/score.txt");
            String^ line = reader->ReadLine();
            reader->Close();
            this->best_time_label->Text = "Лучшее время: " +
line;
            this->best_time_label->Visible = 1;
        }
    }
    catch (...) {}
}

protected:
    /// <summary>
    /// Освободить все используемые ресурсы.
    /// </summary>
    ~StartGame()
    {
        if (components)
        {
            delete components;
        }
        // Показываем главную форму
        for each (Form ^ form in Application::OpenForms) {
            if (form->GetType()->Name == "FlightMenu") {
                form->Show();
                break;
            }
        }
    }

private: System::Windows::Forms::Label^ lore_label;
private: System::Windows::Forms::Button^ start_button;
private: System::Windows::Forms::Label^ goal_label;
private: System::Windows::Forms::Label^ best_time_label;
private: System::Windows::Forms::Button^ exit_button;
private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container ^components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        System::ComponentModel::ComponentResourceManager^ resources =
(gcnew System::ComponentModel::ComponentResourceManager(StartGame::typeid));
        this->lore_label = (gcnew System::Windows::Forms::Label());
        this->start_button = (gcnew System::Windows::Forms::Button());
        this->goal_label = (gcnew System::Windows::Forms::Label());
        this->best_time_label = (gcnew System::Windows::Forms::Label());
        this->exit_button = (gcnew System::Windows::Forms::Button());
        this->SuspendLayout();
        //
        // lore_label
        //
        this->lore_label->AutoSize = true;
    }

```

```

        this->lore_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->lore_label->Location = System::Drawing::Point(12, 9);
        this->lore_label->Name = L"lore_label";
        this->lore_label->Size = System::Drawing::Size(711, 160);
        this->lore_label->TabIndex = 0;
        this->lore_label->Text = resources-
>GetString(L"lore_label.Text");
        //
        // start_button
        //
        this->start_button->Location = System::Drawing::Point(16, 192);
        this->start_button->Name = L"start_button";
        this->start_button->Size = System::Drawing::Size(183, 46);
        this->start_button->TabIndex = 1;
        this->start_button->Text = L"Начать защиты";
        this->start_button->UseVisualStyleBackColor = true;
        this->start_button->Click += gcnew System::EventHandler(this,
&StartGame::start_button_Click);
        //
        // goal_label
        //
        this->goal_label->AutoSize = true;
        this->goal_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->goal_label->Location = System::Drawing::Point(12, 169);
        this->goal_label->Name = L"goal_label";
        this->goal_label->Size = System::Drawing::Size(386, 20);
        this->goal_label->TabIndex = 2;
        this->goal_label->Text = L"Постарайтесь прожить как можно
дольше...";
        //
        // best_time_label
        //
        this->best_time_label->AutoSize = true;
        this->best_time_label->Font = (gcnew
System::Drawing::Font(L"Microsoft Sans Serif", 12,
System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->best_time_label->Location = System::Drawing::Point(446,
169);
        this->best_time_label->Name = L"best_time_label";
        this->best_time_label->Size = System::Drawing::Size(127, 20);
        this->best_time_label->TabIndex = 3;
        this->best_time_label->Text = L"Лучшее время: ";
        this->best_time_label->Visible = false;
        //
        // exit_button
        //
        this->exit_button->BackColor =
System::Drawing::Color::FromArgb(static_cast<System::Int32>(static_cast<System::Byte>
(255)), static_cast<System::Int32>(static_cast<System::Byte>(128)),
        static_cast<System::Int32>(static_cast<System::Byte>(128)));
        this->exit_button->Location = System::Drawing::Point(16, 245);
        this->exit_button->Name = L"exit_button";
        this->exit_button->Size = System::Drawing::Size(183, 46);
        this->exit_button->TabIndex = 4;
        this->exit_button->Text = L"Вернуться в меню";

```

```

        this->exit_button->UseVisualStyleBackColor = false;
        this->exit_button->Click += gcnew System::EventHandler(this,
&StartGame::exit_button_Click);
        //
        // StartGame
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode =
System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(729, 299);
        this->Controls->Add(this->exit_button);
        this->Controls->Add(this->best_time_label);
        this->Controls->Add(this->goal_label);
        this->Controls->Add(this->start_button);
        this->Controls->Add(this->lore_label);
        this->MaximizeBox = false;
        this->Name = L"StartGame";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"The Zondbie";
        this->ResumeLayout(false);
        this->PerformLayout();

    }
#pragma endregion
private: System::Void start_button_Click(System::Object^ sender, System::EventArgs^
e) {
    TheZondbie^ p = gcnew TheZondbie();
    p->Show();
    this->Hide();
}
private: System::Void exit_button_Click(System::Object^ sender, System::EventArgs^
e) {
    for each (Form ^ form in Application::OpenForms) {
        if (form->GetType()->Name == "FlightMenu") {
            form->Show();
            break;
        }
    }
    this->Close();
}
};
}

```

П.Б.17 Код TheZondbie.h

```

#pragma once

#include <fstream>
#include <ctime>

namespace MarkflightsUI {
    using namespace System;
    using namespace System::IO;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;
    using namespace System::Collections::Generic;
    using namespace System::Media;

```



```

public ref class TheZombie : public System::Windows::Forms::Form
{
private:
    // Игровые объекты
    List<PictureBox^>^ zombies;
    Timer^ game_timer;
    Timer^ spawn_timer;
    Timer^ animation_timer;
    Timer^ world_time;

    // Состояние игры
    int bullets_left;           // Осталось пуль (0-2)
    int is_gun_open;            // Открыто ли ружьё
    int best_score;             // Лучший результат
    float zombie_speed;         // Текущая скорость зомби
    int animation_frame;        // Кадр анимации (1 или 2)
    int time_passed;            // Сколько времени прошло

    // Элементы UI
    PictureBox^ zombieBox;

    // Константы
    static const int ZOMBIE_SIZE_X = 61;
    static const int ZOMBIE_SIZE_Y = 111;
    static const int MAX_BULLETS = 2;
    static const float BASE_SPEED = 2.0f;

    // Путь к файлу рекордов
    String^ best_score_file;

    // Звуки
    SoundPlayer^ shoot_sound;
    SoundPlayer^ open_Peet_sound;
    SoundPlayer^ new_bullet_sound;
    SoundPlayer^ zombie_death_sound;
    SoundPlayer^ game_over_sound;

public:
    TheZombie(void)
    {
        InitializeComponent();
        InitializeGame();
    }

protected:
    ~TheZombie()
    {
        if (components)
        {
            delete components;
        }
        game_timer->Stop();
        spawn_timer->Stop();
        animation_timer->Stop();
        world_time->Stop();
        // Проигрываем звук проигрыша
        try {
            game_over_sound->Play();
        }
        catch (...) {}
        // Последнее сообщение
        MessageBox::Show("Вы держались как могли...");
    }
}

```

```

        // Пробуем записать результат игры
        int best = 0;
        try {
            if (File::Exists("game_data/score.txt")) {
                StreamReader^ reader = gcnew
StreamReader("game_data/score.txt");
                best = Convert::ToInt32(reader->ReadLine());
                reader->Close();
            }

        }
        catch (...) {}
        if (best < time_passed) {
            try {
                StreamWriter^ writer = gcnew
StreamWriter("game_data/score.txt");
                writer->WriteLine(time_passed);
                writer->Close();
            }
            catch (Exception^ ex) {
                MessageBox::Show(L"Ошибка сохранения: " + ex->Message);
            }
        }
        // Показываем стартовую форму при закрытии
        for each (Form ^ form in Application::OpenForms) {
            if (form->GetType()->Name == "StartGame") {
                form->Show();
                break;
            }
        }
    }

private: System::Windows::Forms::PictureBox^ street_box;
private: System::Windows::Forms::PictureBox^ Peet_closed_box;
private: System::Windows::Forms::PictureBox^ Peet_open_box;
private: System::Windows::Forms::PictureBox^ bullet_ready_left_box;
private: System::Windows::Forms::PictureBox^ bullet_ready_right_box;
private: System::Windows::Forms::PictureBox^ bullet_done_left_box;
private: System::Windows::Forms::PictureBox^ bullet_done_right_box;
private: System::Windows::Forms::Label^ time_word_label;
private: System::Windows::Forms::Label^ time_label;
private: System::Windows::Forms::Button^ exit_button;

private:
    /// <summary>
    /// Обязательная переменная конструктора.
    /// </summary>
    System::ComponentModel::Container^ components;

#pragma region Windows Form Designer generated code
    /// <summary>
    /// Требуемый метод для поддержки конструктора – не изменяйте
    /// содержимое этого метода с помощью редактора кода.
    /// </summary>
    void InitializeComponent(void)
    {
        System::ComponentModel::ComponentResourceManager^ resources = (gcnew
System::ComponentModel::ComponentResourceManager(TheZondbie::typeid));
        this->street_box = (gcnew System::Windows::Forms::PictureBox());
        this->Peet_closed_box = (gcnew System::Windows::Forms::PictureBox());
        this->Peet_open_box = (gcnew System::Windows::Forms::PictureBox());
        this->bullet_ready_left_box = (gcnew
System::Windows::Forms::PictureBox());

```

```

        this->bullet_ready_right_box = (gcnew
System::Windows::Forms::PictureBox());
        this->bullet_done_left_box = (gcnew
System::Windows::Forms::PictureBox());
        this->bullet_done_right_box = (gcnew
System::Windows::Forms::PictureBox());
        this->time_word_label = (gcnew System::Windows::Forms::Label());
        this->time_label = (gcnew System::Windows::Forms::Label());
        this->exit_button = (gcnew System::Windows::Forms::Button());
        (cli::safe_cast<System::ComponentModel::ISupportInitialize>(this-
>street_box))->BeginInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize>(this-
>Peet_closed_box))->BeginInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize>(this-
>Peet_open_box))->BeginInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize>(this-
>bullet_ready_left_box))->BeginInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize>(this-
>bullet_ready_right_box))->BeginInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize>(this-
>bullet_done_left_box))->BeginInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize>(this-
>bullet_done_right_box))->BeginInit();
        this->SuspendLayout();
        //
        // street_box
        //
        this->street_box->Image =
(cli::safe_cast<System::Drawing::Image>(resources-
>GetObject(L"street_box.Image")));
        this->street_box->Location = System::Drawing::Point(240, 13);
        this->street_box->Name = L"street_box";
        this->street_box->Size = System::Drawing::Size(599, 427);
        this->street_box->SizeMode =
System::Windows::Forms::PictureBoxSizeMode::Zoom;
        this->street_box->TabIndex = 0;
        this->street_box->TabStop = false;
        //
        // Peet_closed_box
        //
        this->Peet_closed_box->Image =
(cli::safe_cast<System::Drawing::Image>(resources-
>GetObject(L"Peet_closed_box.Image")));
        this->Peet_closed_box->Location = System::Drawing::Point(12, 12);
        this->Peet_closed_box->Name = L"Peet_closed_box";
        this->Peet_closed_box->Size = System::Drawing::Size(222, 50);
        this->Peet_closed_box->SizeMode =
System::Windows::Forms::PictureBoxSizeMode::StretchImage;
        this->Peet_closed_box->TabIndex = 1;
        this->Peet_closed_box->TabStop = false;
        this->Peet_closed_box->Click += gcnew System::EventHandler(this,
&TheZondbie::Peet_closed_Click);
        //
        // Peet_open_box
        //
        this->Peet_open_box->Image =
(cli::safe_cast<System::Drawing::Image>(resources-
>GetObject(L"Peet_open_box.Image")));
        this->Peet_open_box->Location = System::Drawing::Point(12, 13);
        this->Peet_open_box->Name = L"Peet_open_box";
        this->Peet_open_box->Size = System::Drawing::Size(222, 108);
        this->Peet_open_box->SizeMode =
System::Windows::Forms::PictureBoxSizeMode::StretchImage;

```

```

        this->Peet_open_box->TabIndex = 2;
        this->Peet_open_box->TabStop = false;
        this->Peet_open_box->Visible = false;
        this->Peet_open_box->Click += gcnew System::EventHandler(this,
&TheZondbie::Peet_open_Click);
        //
        // bullet_ready_left_box
        //
        this->bullet_ready_left_box->Image =
(cli::safe_cast<System::Drawing::Image^>(resources-
>GetObject(L"bullet_ready_left_box.Image"))));
        this->bullet_ready_left_box->Location = System::Drawing::Point(17, 127);
        this->bullet_ready_left_box->Name = L"bullet_ready_left_box";
        this->bullet_ready_left_box->Size = System::Drawing::Size(93, 93);
        this->bullet_ready_left_box->SizeMode =
System::Windows::Forms::PictureBoxSizeMode::AutoSize;
        this->bullet_ready_left_box->TabIndex = 3;
        this->bullet_ready_left_box->TabStop = false;
        this->bullet_ready_left_box->Visible = false;
        //
        // bullet_ready_right_box
        //
        this->bullet_ready_right_box->Image =
(cli::safe_cast<System::Drawing::Image^>(resources-
>GetObject(L"bullet_ready_right_box.Image"))));
        this->bullet_ready_right_box->Location = System::Drawing::Point(136,
127);
        this->bullet_ready_right_box->Name = L"bullet_ready_right_box";
        this->bullet_ready_right_box->Size = System::Drawing::Size(93, 93);
        this->bullet_ready_right_box->SizeMode =
System::Windows::Forms::PictureBoxSizeMode::AutoSize;
        this->bullet_ready_right_box->TabIndex = 4;
        this->bullet_ready_right_box->TabStop = false;
        this->bullet_ready_right_box->Visible = false;
        //
        // bullet_done_left_box
        //
        this->bullet_done_left_box->Image =
(cli::safe_cast<System::Drawing::Image^>(resources-
>GetObject(L"bullet_done_left_box.Image"))));
        this->bullet_done_left_box->Location = System::Drawing::Point(17, 127);
        this->bullet_done_left_box->Name = L"bullet_done_left_box";
        this->bullet_done_left_box->Size = System::Drawing::Size(93, 93);
        this->bullet_done_left_box->SizeMode =
System::Windows::Forms::PictureBoxSizeMode::AutoSize;
        this->bullet_done_left_box->TabIndex = 5;
        this->bullet_done_left_box->TabStop = false;
        this->bullet_done_left_box->Visible = false;
        this->bullet_done_left_box->Click += gcnew System::EventHandler(this,
&TheZondbie::bullet_done_left_Click);
        //
        // bullet_done_right_box
        //
        this->bullet_done_right_box->Image =
(cli::safe_cast<System::Drawing::Image^>(resources-
>GetObject(L"bullet_done_right_box.Image"))));
        this->bullet_done_right_box->Location = System::Drawing::Point(136,
127);
        this->bullet_done_right_box->Name = L"bullet_done_right_box";
        this->bullet_done_right_box->Size = System::Drawing::Size(93, 93);
        this->bullet_done_right_box->SizeMode =
System::Windows::Forms::PictureBoxSizeMode::AutoSize;
        this->bullet_done_right_box->TabIndex = 6;

```

```

        this->bullet_done_right_box->TabStop = false;
        this->bullet_done_right_box->Visible = false;
        this->bullet_done_right_box->Click += gcnew System::EventHandler(this,
&TheZondbie::bullet_done_right_Click);
        //
        // time_word_label
        //
        this->time_word_label->AutoSize = true;
        this->time_word_label->Font = (gcnew System::Drawing::Font(L"Microsoft
Sans Serif", 12, System::Drawing::FontStyle::Bold,
System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->time_word_label->Location = System::Drawing::Point(13, 344);
        this->time_word_label->Name = L"time_word_label";
        this->time_word_label->Size = System::Drawing::Size(73, 20);
        this->time_word_label->TabIndex = 7;
        this->time_word_label->Text = L"Время: ";
        //
        // time_label
        //
        this->time_label->AutoSize = true;
        this->time_label->Font = (gcnew System::Drawing::Font(L"Microsoft Sans
Serif", 12, System::Drawing::FontStyle::Bold, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
        this->time_label->Location = System::Drawing::Point(91, 344);
        this->time_label->Name = L"time_label";
        this->time_label->Size = System::Drawing::Size(19, 20);
        this->time_label->TabIndex = 8;
        this->time_label->Text = L"0";
        //
        // exit_button
        //
        this->exit_button->BackColor =
System::Drawing::Color::FromArgb(static_cast<System::Int32>(static_cast<System::Byte>
>(255)), static_cast<System::Int32>(static_cast<System::Byte>(128)),
        static_cast<System::Int32>(static_cast<System::Byte>(128)));
        this->exit_button->Location = System::Drawing::Point(12, 401);
        this->exit_button->Name = L"exit_button";
        this->exit_button->Size = System::Drawing::Size(129, 34);
        this->exit_button->TabIndex = 9;
        this->exit_button->Text = L"Сдаться";
        this->exit_button->UseVisualStyleBackColor = false;
        this->exit_button->Click += gcnew System::EventHandler(this,
&TheZondbie::exit_button_Click);
        //
        // TheZondbie
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13);
        this->AutoScaleMode = System::Windows::Forms::AutoScaleMode::Font;
        this->AutoSizeMode =
System::Windows::Forms::AutoSizeMode::GrowAndShrink;
        this->ClientSize = System::Drawing::Size(843, 447);
        this->Controls->Add(this->exit_button);
        this->Controls->Add(this->time_label);
        this->Controls->Add(this->time_word_label);
        this->Controls->Add(this->bullet_done_right_box);
        this->Controls->Add(this->bullet_done_left_box);
        this->Controls->Add(this->bullet_ready_right_box);
        this->Controls->Add(this->bullet_ready_left_box);
        this->Controls->Add(this->Peet_closed_box);
        this->Controls->Add(this->street_box);
        this->Controls->Add(this->Peet_open_box);
        this->DoubleBuffered = true;

```

```

        this->MaximizeBox = false;
        this->MinimizeBox = false;
        this->Name = L"TheZondbie";
        this->StartPosition =
System::Windows::Forms::FormStartPosition::CenterScreen;
        this->Text = L"TheZondbie";
        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>street_box))->EndInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>Peet_closed_box))->EndInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>Peet_open_box))->EndInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>bullet_ready_left_box))->EndInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>bullet_ready_right_box))->EndInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>bullet_done_left_box))->EndInit();
        (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this-
>bullet_done_right_box))->EndInit();
        this->ResumeLayout(false);
        this->PerformLayout();

    }
#pragma endregion
private:
    void InitializeGame() {
        // Инициализация списков
        zondbies = gcnew List<PictureBox^>();
        // Инициализация переменных
        bullets_left = MAX_BULLETS;
        is_gun_open = 0;
        zondbie_speed = BASE_SPEED;
        animation_frame = 1;
        time_passed = 0;
        // Таймер игрового цикла
        game_timer = gcnew Timer();
        game_timer->Interval = 16; // ~60 FPS
        game_timer->Tick += gcnew EventHandler(this, &TheZondbie::GameTick);
        game_timer->Start();
        // Таймер спавна зомби
        spawn_timer = gcnew Timer();
        spawn_timer->Interval = 1500; // Каждые 1.5 секунды
        spawn_timer->Tick += gcnew EventHandler(this,
&TheZondbie::SpawnZondbie);
        spawn_timer->Start();
        // Таймер анимации зомби
        animation_timer = gcnew Timer();
        animation_timer->Interval = 200; // 5 FPS для анимации
        animation_timer->Tick += gcnew EventHandler(this,
&TheZondbie::AnimationTick);
        animation_timer->Start();
        // Таймер времени
        world_time = gcnew Timer();
        world_time->Interval = 1000;
        world_time->Tick += gcnew EventHandler(this, &TheZondbie::TimeTick);
        world_time->Start();
        // Создание звуков
        shoot_sound = gcnew SoundPlayer("game_data/shoot_sound.wav");
        open_Peet_sound = gcnew SoundPlayer("game_data/open_Peet_sound.wav");
        new_bullet_sound = gcnew SoundPlayer("game_data/new_bullet_sound.wav");
        zondbie_death_sound = gcnew
SoundPlayer("game_data/zondbie_death_sound.wav");

```

```

        game_over_sound = gcnew SoundPlayer("game_data/game_over_sound.wav");
    }
    void SpawnZondbie(Object^ sender, EventArgs^ e) {
        PictureBox^ zondbie = gcnew PictureBox();
        zondbie->Size = Drawing::Size(ZONDBIE_SIZE_X, ZONDBIE_SIZE_Y);
        zondbie->Tag = 0; // 0 - не уничтожен
        // Случайный выбор стороны
        Random^ rand = gcnew Random();
        int from_left = rand->Next(0, 2) == 0;
        if (from_left) {
            zondbie->Location = Point(-ZONDBIE_SIZE_X, 0);
            zondbie->Tag = 1; // 1 - идёт слева направо
        }
        else {
            zondbie->Location = Point(street_box->Width, 0);
            zondbie->Tag = -1; // -1 - идёт справа налево
        }
        // Устанавливаем картинку
        UpdateZondbieImage(zondbie);
        zondbie->Click += gcnew EventHandler(this, &TheZondbie::Zondbie_Click);
        street_box->Controls->Add(zondbie);
        zondbies->Add(zondbie);
    }
    void UpdateZondbieImage(PictureBox^ zondbie) {
        int direction = (int)zondbie->Tag;
        String^ side = (direction == 1 || direction == -1) ?
            (direction != 1 ? L"right" : L"left") : L"left";
        String^ imagePath = String::Format(L"game_data/Zondbie_{0}_{1}.png",
            side, (animation_frame % 2) + 1);
        try {
            zondbie->Image = Image::FromFile(imagePath);
        }
        catch (...) {
            // Если картинки нет - закрашиваем цветом
            zondbie->BackColor = Color::Green;
        }
    }

    // Обработки таймеров
    void GameTick(Object^ sender, EventArgs^ e) {
        // Двигаем всех зомби
        for (int i = zondbies->Count - 1; i >= 0; i--)
        {
            PictureBox^ zondbie = zondbies[i];
            int direction = (int)zondbie->Tag;
            Point pos = zondbie->Location;
            // Получаем центр street_box по X
            int centerX = street_box->Width / 2;
            // Расстояние от зомби до центра (по модулю)
            int distanceToCenter = Math::Abs(pos.X + ZONDBIE_SIZE_X / 2 -
centerX);
            // Максимальное смещение по Y (насколько низко может опуститься
зомби)

            int maxYOffset = street_box->Height - ZONDBIE_SIZE_Y;
            // Y меняется пропорционально расстоянию до центра
            // Чем ближе к центру (distanceToCenter меньше), тем больше Y
            pos.Y = maxYOffset - (distanceToCenter * maxYOffset / centerX);
            // Движение по X
            pos.X += (int)(zondbie_speed * direction);
            // Ограничиваем Y, чтобы зомби не выходил за пределы
            if (pos.Y < 0) pos.Y = 0;
            if (pos.Y > maxYOffset) pos.Y = maxYOffset;
            zondbie->Location = pos;
        }
    }

```

```

        // Проверка, дошёл ли зомби до центра
        if ((direction == 1 && pos.X >= centerX - ZONDBIE_SIZE_X / 2) ||
            (direction == -1 && pos.X <= centerX + ZONDBIE_SIZE_X / 2)) {
            // Зомби дошёл - конец игры
            this->Close();
            return;
        }
        // Удаляем зомби за пределами экрана
        if (pos.X < -ZONDBIE_SIZE_X || pos.X > street_box->Width +
ZONDBIE_SIZE_X) {
            street_box->Controls->Remove(zondbie);
            zondbies->RemoveAt(i);
            delete zondbie;
        }
    }
}

void AnimationTick(Object^ sender, EventArgs^ e) {
    // Меняем кадр анимации
    animation_frame++;
    // Обновляем картинки всех зомби
    for each (PictureBox ^ zondbie in zondbies)
        UpdateZondbieImage(zondbie);
}

void TimeTick(Object^ sender, EventArgs^ e) {
    time_passed++;
    time_label->Text = time_passed.ToString();
}

// Нажатия игрока
void Zondbie_Click(Object^ sender, EventArgs^ e) {
    PictureBox^ zondbie = safe_cast<PictureBox^>(sender);
    // Проверяем, можно ли стрелять
    if (is_gun_open || bullets_left <= 0) return;
    try {
        Random^ rand = gcnew Random();
        if (rand->Next(0, 2))
            shoot_sound->Play();
        else
            zondbie_death_sound->Play();
    }
    catch (...) {}
    // Убиваем зомби
    street_box->Controls->Remove(zondbie);
    zondbies->Remove(zondbie);
    delete zondbie;
    bullets_left--;
    zondbie_speed += 0.05f; // Увеличиваем скорость
}

void Peet_closed_Click(Object^ sender, EventArgs^ e) {
    if (bullets_left) return;
    try {
        open_Peet_sound->Play();
    }
    catch (...) {}
    // Переключаем состояние ружья и патронов
    is_gun_open = 1;
    Peet_closed_box->Visible = 0;
    Peet_open_box->Visible = 1;
    bullet_done_left_box->Visible = 1;
    bullet_done_right_box->Visible = 1;
}

void Peet_open_Click(Object^ sender, EventArgs^ e) {
    if (bullets_left != 2) return;
}

```



```

        try {
            open_Peet_sound->Play();
        }
        catch (...) {}
        // Переключаем состояние ружья и патронов
        is_gun_open = 0;
        Peet_closed_box->Visible = 1;
        Peet_open_box->Visible = 0;
        bullet_ready_left_box->Visible = 0;
        bullet_ready_right_box->Visible = 0;
    }
    void bullet_done_left_Click(Object^ sender, EventArgs^ e) {
        try {
            new_bullet_sound->Play();
        }
        catch (...) {}
        // Добавляем патрон и меняем картинку
        bullets_left++;
        bullet_done_left_box->Visible = 0;
        bullet_ready_left_box->Visible = 1;
    }
    void bullet_done_right_Click(Object^ sender, EventArgs^ e) {
        try {
            new_bullet_sound->Play();
        }
        catch (...) {}
        // Добавляем патрон и меняем картинку
        bullets_left++;
        bullet_done_right_box->Visible = 0;
        bullet_ready_right_box->Visible = 1;
    }
    private: System::Void exit_button_Click(System::Object^ sender,
        System::EventArgs^ e) {
        this->Close();
    }
};
}

```

Приложение В

Руководство системного администратора

П.В.1 Системные требования

Для корректной работы программного обеспечения «Mark Flights» необходимы следующие аппаратные и программные средства:

- ОС Windows XP SP3 или выше;
- Оперативная память 30МБ или выше;
- Жесткий диск: 10МБ или выше для хранения программы и дополнительное место для хранения вводимых в базу данных;
- Наличие стандартных устройств ввода-вывода – клавиатура и монитор (дисплей);
- .NET Framework версии 2.0 или выше;
- Разрешение экрана 1200 x 900 или выше.

П.В.2 Установка и запуск

Программа не требует специальной установки. Для запуска необходимо выполнить следующие действия:

1. Скопировать папку с исполняемым файлом MarkflightsUI.exe на жёсткий диск компьютера.
2. Запустить файл MarkflightsUI.exe двойным щелчком мыши.

П.В.3 Структура файлов данных

Программа использует следующие файлы, хранящиеся в папке flights_data (создаётся автоматически при первом запуске в директории с исполняемым файлом):

- **table.csv** - Основной файл базы данных с информацией об авиарейсах. Формат — CSV с разделителем «;»;

- **settings.txt** - Файл конфигурации. Содержит пароль администратора и путь к файлу table.csv;
- **filters.csv** - Файл с сохранёнными фильтрами поиска;

П.В.4 Управление файлами данных

Администратор может изменить путь к файлу базы данных с помощью кнопки «Изменить путь до файла» в главной форме. При этом доступны следующие опции:

- Создать новый CSV-файл;
- Выбрать существующий CSV-файл;
- Скопировать данные из текущего файла в новый;
- Удалить старый файл.

Ручное редактирование файлов не рекомендуется, так как может нарушить работу программы.

П.В.5 Пароль администратора

Пароль администратора хранится в файле flights_data/settings.txt в открытом виде. Для обеспечения безопасности рекомендуется ограничить доступ к этому файлу средствами операционной системы. Смена пароля возможна только путём редактирования файла settings.txt (первая строка).

Приложение Г

Руководство пользователя

П.Г.1 Начало работы

Для запуска программы дважды щёлкните левой кнопкой мыши по файлу MarkflightsUI.exe. После запуска откроется главное меню.

П.Г.2 Главное меню

В главном меню доступны следующие варианты:

- **«Администратор»** — переход к форме авторизации. Доступен только при наличии пароля;
- **«Пользователь»** — переход к базе данных в режиме просмотра;
- **«Запустить игру»** — запуск дополнительного игрового модуля «Зомби-шутер».

П.Г.3 Режим администратора

Для входа в режим администратора необходимо ввести пароль, известный только администратору (по умолчанию — уточнить у системного администратора).

Основное окно администратора содержит таблицу со списком авиарейсов и панель управления с кнопками:

- **«Обновить таблицу»** — Перечитать данные из файла и обновить отображение;
- **«Настроить фильтры»** — Открыть окно для создания и редактирования фильтров поиска;
- **«Добавить рейс»** — Добавить новую запись в базу данных;
- **«Изменение рейсов»** — Изменить выбранные или отфильтрованные записи;
- **«Удаление рейсов»** — Удалить выбранные или отфильтрованные записи;

- **«Изменить путь до файла»** — Сменить файл базы данных;
- **«Выйти в главное меню»** — Завершить работу с базой данных и вернуться в главное меню.

П.Г.4 Работа с фильтрами

Форма «Настройка фильтров» позволяет задать условия поиска. Для каждого поля можно добавить до двух условий, объединённых логическими операторами «И» или «ИЛИ». Для полей с числами и временем доступны операторы сравнения ($>$, $<$, $=$, $\geq$, $\leq$).

После применения фильтров таблица автоматически отображает только записи, удовлетворяющие заданным условиям.

П.Г.5 Добавление, изменение и удаление записей

Для добавления новой записи нажмите «Добавить рейс» и заполните все поля в открывшейся форме. Поля времени используют маску ввода «00:00», дни недели выбираются флажками.

Для изменения или удаления записей выберите соответствующий режим. Возможно:

- Применить действие ко всем записям, подходящим под текущий фильтр;
- Применить действие к конкретным номерам записей, указанным через запятую или пробел.

П.Г.6 Игра «Зомби-шутер»

В главном меню нажмите кнопку «Запустить игру». Игровой процесс:

- Цель — продержаться как можно дольше, отстреливая наступающих зомби;
- Управление:
 - Нажмите на закрытое ружьё, чтобы открыть его;
 - Зарядите две пули, нажимая на пустые патроны;

- Стреляйте по зомби кликом мыши.
- Сложность — скорость зомби увеличивается с каждым убитым;
- Окончание игры — зомби добрался до нижней центральной точки игрового поля или игрок нажал «Сдаться»;
- Рекорды — лучшее время выживания сохраняется и отображается при следующем запуске игры.

П.Г.7 Завершение работы

Для выхода из программы нажмите кнопку «Выйти в главное меню» в формах работы с базой данных, а затем закройте главное меню стандартными средствами Windows (крестик в правом верхнем углу

Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Алтайский государственный технический университет
имени И.И. Ползунова»

Факультет информационных технологий
Кафедра «Прикладная математика»

ОТЗЫВ

на выполнение учебной (ознакомительной) практики
студенту группы ПИ-54 Дворникову Марку Сергеевичу

Тема курсового проекта: «Разработка программного обеспечения для
организации списка данных об авиарейсах».

За время выполнения проекта студент Дворников М.С. изучил и применил на практике базовые знания языка Си и приобрел практический навык разработки консольных приложений по работе с наборами структурированных данных. Основные характеристики выполненной работы приведены ниже.

1. Разработанная программа соответствует поставленной задаче – да / нет.
2. Программа работоспособна - да / нет / частично.
- 3 Программа реализует все поставленные задачи - да / нет.
4. Код соответствует требованиям структурного программирования - да / нет.
5. Код хорошо документирован, читабелен - да / нет.
6. Программа имеет дружелюбный для пользователя интерфейс - да / нет.
7. Оформление отчёта соответствует требованиям - да / нет / частично.
8. Практика выполнена и сдана в установленные сроки - да / нет.
9. Отмеченные достоинства:

10. Отмеченные недостатки:

Работа оценена на «_____» (____)

Руководитель проекта _____ Егорова Е.В., доцент

Дата «__»_____20__ г.